ISTQB® Certified Tester

Foundation Level Specialist — **Performance Testing** (CT-PT, Syllabus 2018)

CHAPTER 4
Performance Testing Tasks

Complete Study Guide

Chapter fact	Value
Syllabus sections covered	4.1 Planning (4.1.1–4.1.3) · 4.2 Analysis, Design and Implementation (4.2.1–4.2.8) · 4.3 Execution · 4.4 Analyzing Results and Reporting
Learning objectives	13 examinable (PTFL-4.1.1 – PTFL-4.4.1) — more than the other four chapters combined
Recommended teaching time	475 minutes — more than Chapters 1, 2, 3 and 5 together (400 minutes)
Questions in the exam	**20 of 40 questions · 20 of 40 points — exactly half the paper.** K1 = 0 · **K2 = 10 · K3 = 4 · K4 = 6**. Every objective is guaranteed; **seven objectives get two questions each**
Exam format	40 questions · 40 points · **90 minutes** (113 with the 25% non-native-language allowance) · pass mark 65% = **26 points**
Keywords	concurrency · load profile · load generation · operational profile · ramp-down · ramp-up · system of systems · system throughput · test plan · think time · virtual user
Official sample-exam questions	**#18 – #37**. Eight of them use the **regional health-records scenario** introduced in Chapter 3

Built exclusively from the ISTQB® CT-PT 2018 syllabus, the official Sample Exam (questions and answers), and the ISTQB® exam structure tables.

Study this single file to master Chapter 4 — half the exam.

Contents

The page numbers above are filled in by Word: right-click the table of contents and choose "Update Field" → "Update entire table".

How to use this guide

Section 1 explains all thirteen learning objectives in syllabus order — read it once, slowly, over more than one sitting. **Section 2** compresses the same material into recall-ready notes. **Section 3** holds the tables you should be able to reproduce from memory; **table 3.2 (protocol groups)**, **table 3.5 (who hears what)**, **table 3.7 (the K4 arithmetic)** and **table 3.8 (script hazards)** carry the most marks. **Section 4** is the question bank — attempt every question with the green answer box covered, then read the rationale even when you were right. **Section 5** converts that into exam tactics. **Section 6** is for daily spaced repetition. **Section 7** is what you read on the morning of the exam.

What makes Chapter 4 unusual: it is half the exam. Twenty of the forty questions and twenty of the forty points live here, along with **all four K3 questions** and **six of the nine K4 questions**. The pass mark is **26 of 40** — so a perfect score on every other chapter still leaves you **six marks short**. Realistically, **Chapter 4 deserves about half your total study time**.

Where the marks actually are. Seven objectives are worth two marks each — 4.1.3 · 4.2.1 · 4.2.2 · 4.2.6 · 4.2.7 · 4.2.8 · 4.3.1 — which is **14 of the 20 marks**, and most of that is **K2 recall**: protocol groups, think time, script structure, ramp-up, environment fidelity. The remaining six marks come one each from the analytical objectives **4.1.1 · 4.1.2 · 4.2.3 · 4.2.4 · 4.2.5 · 4.4.1**. **Learn the seven two-mark objectives first.**

Three marks come from a single worked scenario. Objectives **4.2.3, 4.2.4 and 4.2.5** are all examined on the same emergency-room-doctors problem: what is **missing from the operational profile**, what is the **proper load profile**, and what is the **highest concurrency**. The arithmetic is plain multiplication — **2500 doctors × 10 accesses × 6 updates = 150,000 transactions a day** — but two things trip people up: the load-profile option figures are **band totals, not hourly rates**, and **concurrency counts sessions, not the transactions inside them** (**2,000**, not 12,000). Table 3.7 works it through line by line.

Two option sets are reused. Sample Exam **#20 and #21** offer an **identical four options**, once for business stakeholders and once for technical ones; **#22 and #23** do the same for protocols. Do not assume the second answer must be a different letter.

Suggested schedule (two weeks, roughly half your study time): Days 1–2 — Section 1 for **4.1**, then **4.2.1 and 4.2.2**. Days 3–4 — Section 1 for **4.2.3 to 4.2.5**, then work table 3.7 with a blank sheet until the arithmetic is automatic. Days 5–6 — Section 1 for **4.2.6 to 4.2.8**. Day 7 — Section 1 for **4.3 and 4.4**, plus Section 2 end to end. Days 8–9 — Section 4 Part A, all twenty official questions. Days 10–11 — Section 4 Part B. Day 12 — Section 5. Days 13 onwards — Section 6 flashcards daily, Section 7 the night before.

Section 1 — Syllabus Explanation

Chapter 4 carries **13 learning objectives** across four sub-chapters (4.1 Planning · 4.2 Analysis, Design and Implementation · 4.3 Execution · 4.4 Analyzing Results and Reporting). The official ISTQB Exam Structure Table allocates **20 of the 40 exam questions** to Chapter 4 — **half the paper, and 20 of the 40 points**. Recommended teaching time is **475 minutes**, more than the other four chapters combined.

You cannot pass CT-PT without Chapter 4. The pass mark is **26 of 40**. Even a perfect score on Chapters 1, 2, 3 and 5 gives you **20** — you still need **6 marks from this chapter** just to pass, and realistically you need far more. Chapter 4 also carries **all four K3 questions** and **six of the nine K4 questions** on the paper.

LO ID	K	Objective (syllabus statement)	Qs	Pts
PTFL-4.1.1	**K4**	Derive performance test objectives from relevant information.	**1**	1
PTFL-4.1.2	**K4**	Outline a performance test plan which considers the performance objectives for a given project.	**1**	1
PTFL-4.1.3	**K4** †	Create a presentation that enables various stakeholders to understand the rationale behind the planned performance testing.	**2**	2
PTFL-4.2.1	K2	Give examples of typical protocols encountered in performance testing.	**2**	2
PTFL-4.2.2	K2	Understand the concept of transactions in performance testing.	**2**	2
PTFL-4.2.3	**K4**	Analyze operational profiles for system usage.	**1**	1
PTFL-4.2.4	**K4**	Create load profiles derived from operational profiles for given performance objectives.	**1**	1
PTFL-4.2.5	**K4**	Analyze throughput and concurrency when developing performance tests.	**1**	1
PTFL-4.2.6	K2	Understand the basic structure of a performance test script.	**2**	2
PTFL-4.2.7	**K3**	Implement performance test scripts consistent with the plan and load profiles.	**2**	2
PTFL-4.2.8	K2	Understand the activities involved in preparing for performance test execution.	**2**	2
PTFL-4.3.1	K2	Understand the principal activities in running performance test scripts.	**2**	2
PTFL-4.4.1	**K4**	Analyze and report performance test results and implications.	**1**	1
Chapter 4 total	—	**K1 = 0 · K2 = 10 · K3 = 4 · K4 = 6**	**20**	**20**

A documented discrepancy you should know about

† PTFL-4.1.3 is listed as K4 in the syllabus and in the official Sample Exam Answers, but as K3 in the ISTQB Exam Structure Tables (v1.16). The Exam Structure Table's chapter arithmetic — **K2 = 10, K3 = 4, K4 = 6** for Chapter 4 — only balances if PTFL-4.1.3's two questions are counted as **K3**. This affects nothing you can control: the two questions are the same either way (Sample Exam **#20** and **#21**). It is noted here so that you are not confused if you compare the two documents. **Two further inconsistencies exist in the same table:** the K-level timings sum to **100 minutes** while the exam is **90 minutes**, and a footer line reads **"TOTAL 60 minutes for 40 questions"**. The authoritative figure is the summary row: **40 questions · 40 points · 90 minutes · pass at 26**.

Exam allocation you must know

Chapter 4 = **20 questions / 20 points** of 40 questions and 40 points — **exactly half the paper**. In the official Sample Exam these are questions **#18 to #37**. **Eight of those twenty use the regional health-records scenario** first seen in Chapter 3. **All four K3 questions on the paper are here (PTFL-4.2.7 ×2, plus the two PTFL-4.1.3 items as classified by the structure table), and six of the nine K4 questions.**

Where the marks actually are within Chapter 4

Six objectives are worth two marks each — **4.1.3 · 4.2.1 · 4.2.2 · 4.2.6 · 4.2.7 · 4.2.8 · 4.3.1** (seven, in fact, totalling **14 of the 20**). The remaining **six marks** come one each from the harder analytical objectives: **4.1.1 · 4.1.2 · 4.2.3 · 4.2.4 · 4.2.5 · 4.4.1**. **The two-mark objectives are mostly K2 recall and are far cheaper per mark.** Learn those seven first; they are 70% of the chapter.

PTFL-4.1.1 (K4) — Deriving Performance Test Objectives

Original syllabus statement

Derive performance test objectives from relevant information.

Stakeholders own the objectives. "*Stakeholders may include **users and people with a business or technical background**. They may have **different objectives** relating to performance testing. **Stakeholders set the objectives, the terminology to be used and the criteria for determining whether the objective has been achieved.***" Note that all three — objectives, terminology, criteria — come from the stakeholders, not the tester.

The distinction the official question is built on

"*It is a **good practice to distinguish between user-based and technical objectives**.*" **User-based objectives** focus primarily on **end-user satisfaction and business goals**. "*Generally, **users are less concerned about feature types or how a product gets delivered. They just want to be able to do what they need to do.***" **Technical objectives** focus on **operational aspects** and provide answers to questions regarding **a system's ability to scale**, or **under what conditions degraded performance may become apparent**. **In the official exam, "the system must be able to scale to 10 million patient records with no degradation" is the technical objective; a three-second response time for 100 concurrent users is user-based.**

The three key objectives of performance testing. "*Key objectives of performance testing include **identifying potential risks, finding opportunities for improvement, and identifying necessary changes**.*"

The three questions to ask stakeholders. "*When gathering information from the various stakeholders, the following questions should be answered*" — and these three are quotable:

#	Question to answer when gathering information from stakeholders
1	**What transactions will be executed in the performance test and what average response time is expected?**
2	**What system metrics are to be captured (e.g., memory usage, network throughput) and what values are expected?**
3	**What performance improvements are expected from these tests compared to previous test cycles?**

Exam focus. One **K4** question, on the health-records scenario: "*Which of the following is a **technical objective** for performance that could be applicable to this project?*" The answer is **the system must be able to scale to 10 million patient records with no degradation in the performance**, because **scalability is a technical objective for performance**. The rationale rejects the others precisely: the **three-second response time with 100 concurrent users** is **a user-based objective**; **performing at or above the level of the legacy system** is **also a user-based objective and not a very clear one at that**; and the **disaster-recovery switchover** option is **primarily a robustness test rather than a performance test**.

The K4 answer pattern for PTFL-4.1.1

Ask **whose satisfaction is being measured**. If the sentence would make sense to a **user or the business** — "I get my answer in three seconds", "it is at least as good as the old system" — it is **user-based**. If it is about **capacity to grow, conditions of degradation, or operational limits**, it is **technical**. And beware of options that are not performance objectives at all: **robustness, availability and failover** belong to other quality characteristics.

Related concepts: stakeholder objectives, terminology, achievement criteria, user-based objectives, end-user satisfaction, business goals, technical objectives, operational aspects, scalability, degraded performance, potential risks, opportunities for improvement, necessary changes, expected average response time, expected metric values, improvement against previous test cycles, robustness.

PTFL-4.1.2 (K4) — The Performance Test Plan (PTP)

Original syllabus statement

Outline a performance test plan which considers the performance objectives for a given project.

What the PTP is. "*The **Performance Test Plan (PTP)** is a document **created prior to any performance testing occurring**. The PTP **should be referred to by the Test Plan**, which also includes **relevant scheduling information**. It **continues to be updated once performance testing begins**.*"

The twelve sections of a PTP. "*The following information should be supplied in a PTP*" — this is the single longest list in the syllabus, and the exam expects you to recognise every heading.

The Performance Test Plan contents

Section	What the syllabus requires in it
Objective	**Describes the goals, strategies and methods for the performance test.** It **enables a quantifiable answer to the central question of the adequacy and the readiness of the system to perform under load.**
Test Objectives	**Overall test objectives for performance efficiency to be achieved by the SUT are listed for each type of stakeholder** (see §4.1.1).
System Overview	**A brief description of the SUT** providing **the context for the measurement of the performance test parameters**, including **a high-level description of the functionality being tested under load**.
Types of Performance Testing to be Conducted	The types **are listed (see §1.2) along with a description of the purpose of each type**.
Acceptance Criteria	Performance testing determines the **responsiveness, throughput, reliability and/or scalability** of the system under a given workload. **In general, response time is a user concern, throughput is a business concern, and resource utilization is a system concern.** Criteria should be set for all relevant measures and related back to: **overall performance test objectives · Service Level Agreements (SLAs) · baseline values**. A **baseline** is **a set of metrics used to compare current and previously achieved performance measurements**, enabling **improvements to be demonstrated and/or acceptance criteria to be confirmed**. It **may be necessary to first create the baseline using sanitized data from a database, where possible.**
Test Data	**User account data** (e.g., accounts available for simultaneous log in) · **user input data** (the data a user would enter to perform a business process) · **database** (the pre-populated database). The **creation process** must address: **data extraction from production data · importing data into the SUT · creation of new data · creation of backups that can be used to restore the data when new cycles of testing are performed · data masking or anonymizing**.
System Configuration	**Specific system architecture including servers** (web, database, load balancer) · **definition of multiple tiers** · **computing hardware details** (CPU cores, RAM, SSD, HDD) **including versions · software details** (applications, operating systems, databases, enterprise services) **including versions · external systems that operate with the SUT**, their configuration and version · **SUT build / version identifier**.
Test Environment	**Often a separate environment that mimics production, but at a smaller scale.** Must include **how the results will be extrapolated to apply to the larger production environment**. **With some systems the production environment becomes the only viable option for testing, but in this case the specific risks of this type of testing must be discussed. Testing tools sometimes reside outside the test environment and may require special access rights.** Tests **may also be conducted with a component part of the system capable of operating without other components** — **often cheaper** and possible **as soon as the component is developed**.
Test Tools	**Which test tools (and versions) will be used in scripting, executing and monitoring** — normally **tool(s) used to simulate user transactions · tools to provide load from multiple points within the system architecture (points of presence) · tools to monitor system performance**.
Profiles	**Operational profiles provide a repeatable step-by-step flow through the application for a particular**

Section	What the syllabus requires in it
	usage of the system. Aggregating these operational profiles results in a load profile (commonly referred to as a scenario).
Relevant Metrics	**Taking too many measurements can make analysis difficult as well as negatively impact the application's actual performance**, so it is important to **identify the measurements and metrics that are most relevant to accomplish the objectives**. The syllabus supplies a table (reproduced in Section 3).
Risks	**Areas not measured as part of the performance testing**, plus **limitations to the performance testing** (e.g., **external interfaces that cannot be simulated, insufficient load, inability to monitor servers**). **Limitations of the test environment may also produce risks** (e.g., **insufficient data, scaled down environment**).

Three sentences from the PTP that are examined elsewhere too

1. "Response time is a user concern, throughput is a business concern, and resource utilization is a system concern." This one line ties **PTFL-4.1.1** (user-based vs technical objectives) to **PTFL-2.1.1** (business vs technical metrics) and to **PTFL-4.4.1** (what to report to whom). **2. Data masking is mandatory under GDPR** for production data containing personally identifiable information — **however, in performance testing, data masking adds risk to the performance tests as it may not have the same data characteristics as seen in real-world use. 3. A baseline is a set of metrics used to compare current and previously achieved performance measurements** — the same idea the six risks in **PTFL-2.1.1** warn about losing.

Exam focus. One **K4** question, on the health-records scenario: "*You have been asked to write a performance test plan. Which of the following is the information you will need to deal with the **most critical performance objective**?*" The answer is **who can access what data and how often will they do it**. The official rationale is a compressed lesson in plan-building: **because data retrieval time is critical to the project, you need to know who can access that data (so you can get the user profiles created), what data they will access (so you can be sure you have the right data available) and how often they will access it (so you can set up an operational profile)**. The distractors fail because: **authentication and authorization is more related to security than performance testing**; **data storage may be something to investigate if retrieval proves slow, but we don't need to know it to conduct the tests**; and **what happens after retrieval is after the fact — the scenario just discusses presenting the information to the requestor**.

The K4 answer pattern for PTFL-4.1.2

Identify the stated critical objective, then choose the option whose information feeds directly into building the profiles and data for that objective. In the official question the objective is **three-second retrieval**, so the winning information is **who, what and how often** — the three inputs to an **operational profile** (**PTFL-4.2.3**). Options about **security**, **storage capacity** or **downstream use** are all plausible project questions that do not feed the test.

Related concepts: Performance Test Plan (PTP), test plan reference, objective, test objectives per stakeholder type, system overview, types of performance testing, acceptance criteria, responsiveness, throughput, reliability, scalability, SLA, baseline, sanitized data, test data, user account data, user input data, pre-populated database, data extraction, data import, backups and restoration, data masking, anonymization, GDPR, system configuration, tiers, versions, SUT build identifier, test environment, extrapolation, production testing risks, special access rights, component-level testing, test tools, points of presence, operational profile, load profile, scenario, relevant metrics, risks and limitations.

PTFL-4.1.3 (K4 / K3) — Communicating About Performance Testing

Original syllabus statement

Create a presentation that enables various stakeholders to understand the rationale behind the planned performance testing.

Why this objective exists. "*The tester **must be capable of communicating to all stakeholders the rationale behind the performance testing approach and the activities to be undertaken** (as detailed in the Performance Test Plan). **The subjects to***

be addressed in this communication may vary considerably between stakeholders *depending on whether they have a* ***"business / user-facing" interest*** *or a more* ***"technology / operations-facing" focus***."

This objective is worth two marks, and the official exam spends both on the same distinction: the **same four options** are offered twice, once asking what to tell **business** stakeholders and once asking what to tell **technical** stakeholders.

Business focus versus technology focus

Stakeholders with a business focus — seven factors	Stakeholders with a technology focus — six factors
Less interested in the distinctions between functional and non-functional quality characteristics.	**The planned approach to generating required load profiles must be explained**, and **the expected involvement of technical stakeholders made clear**.
Technical issues concerning tooling, scripting and load generation are generally of secondary interest.	**Detailed steps in the setup and execution of the performance tests must be explained to show the relation of the testing to the architectural risks.**
The connection between product risks and performance test objectives must be clearly stated.	**Steps required to make performance tests repeatable must be communicated** — these may include **organizational aspects (e.g., participation of key staff) as well as technical issues**.
Stakeholders must be made aware of the balance between the cost of planned performance tests and how representative the performance testing results will be, compared to production conditions.	**Where test environments are to be shared, the scheduling of performance tests must be communicated to ensure the test results will not be adversely impacted.**
The repeatability of planned performance tests must be communicated — will the test be **difficult to repeat** or can it be **repeated with a minimum of effort**?	**Mitigations of the potential impact on actual users, if performance testing needs to be executed in the production environment, must be communicated and accepted.**
Project risks must be communicated — constraints and dependencies concerning the setup of the tests, infrastructure requirements (hardware, tools, data, bandwidth, test environment, resources) and dependencies on key staff.	**Technical stakeholders must be clear about their tasks and when they are scheduled.**
The high-level activities must be communicated (see §4.2 and §4.3) **together with a broad plan containing costs, time schedule and milestones.**	—

The two official questions, decoded together

Both questions offer the **same four options**. **To business stakeholders (#20), the answer is the COST option:** "*The performance test system will cost* ***$240,000*** *which will include the hardware and setup. This system will allow us to create a test system that is* ***representative of the production system*** *and will allow us to* ***simulate production-like conditions***." — it hits **cost versus representativeness** exactly. **To technical stakeholders (#21), the answer is the PROCEDURE option:** "*Once configured, the system will require* ***data loading****... then a* ***small set of sample scripts to verify the output****... then the performance test script which* ***ramps up users at a rate of 10 per minute*** *until we reach the target number of concurrent users. This load will then be* ***maintained for 2 hours***." — it hits **detailed steps in the setup and execution**. **The other two options are wrong for BOTH audiences.** The production-testing option is **incomplete: if testing will be conducted in production, mitigation plans must also be made so that any impact to production is understood**. The risk-list option is **a list of risks but doesn't include a list of how the tests will help mitigate these risks**.

The answer pattern for PTFL-4.1.3 — worth two marks

Money and risk-to-objective linkage go to the business. Steps, ramps, schedules and tooling go to the technical audience. And regardless of audience, an option is wrong if it is **incomplete** — production testing without **mitigation plans**, or risks without **how the testing mitigates them**.

Related concepts: stakeholder communication, business focus, technology focus, product risks, project risks, cost versus representativeness, repeatability, infrastructure requirements, dependencies on key staff, milestones, load profile generation approach, architectural risks, shared test environments, scheduling, production testing mitigation, task clarity.

PTFL-4.2.1 (K2) — Typical Communication Protocols

Original syllabus statement

Give examples of typical protocols encountered in performance testing.

Why protocols matter. "*Communication protocols* ***define a set of communications rules between computers and systems***. ***Designing tests properly to target specific parts of the system requires understanding protocols.***"

The OSI framing. "*Communication protocols are often described by the* ***Open Systems Interconnection (OSI) model layers*** *(see* ***ISO/IEC 7498-1****), although* ***some protocols may fall outside of this model***. ***For performance testing, protocols from Layer 5 (Session Layer) to Layer 7 (Application Layer) are most commonly used.***" And: "*Generally speaking,* ***the level of the OSI layer which is most in focus relates to the level of the architecture being tested***. *When testing some* ***low-level, embedded architecture****, for example,* ***the lower numbered layers of the OSI model will be mostly in focus***."

The protocol catalogue — learn the group, not just the name

Group	Protocols named by the syllabus
Database	ODBC · JDBC · other vendor-specific protocols
Web	HTTP · HTTPS · HTML
Web Service	SOAP · REST
Network	DNS · FTP · IMAP · LDAP · POP3 · SMTP · Windows Sockets · CORBA
Mobile	TruClient · SMP · MMS
Remote Access	Citrix ICA · RTE
SOA	MQSeries · JSON · WSCL

Both official questions are pure group-matching

#22 — testing the speed of the response of a web service → HTTP. The rationale: **HTTP is an example of a web protocol**; **ODBC is a database protocol**; **REST is a web service protocol**; **SMTP is a network protocol**. **#23 — testing the speed with which database requests are sent and received → ODBC.** Same four options, same four classifications. **Note the subtlety in #22:** the stem says *web service*, and **REST** is on the list as a **web service** protocol — yet the official answer is **HTTP**. The question is asking which protocol is **being used** to carry the test traffic, and the official classification treats **HTTP as the protocol in use**. **Learn the group labels exactly as the syllabus assigns them, and answer from the classification, not from architectural reasoning.**

Architecture and protocol choice. "*It is important to* ***understand the overall system architecture*** *because performance tests can be executed* ***on an individual system component (e.g., web server, database server) or on a whole system via end-to-end testing***. ***Traditional 2-tier applications built with a client-server model*** *specify the* ***"client" as the GUI and primary user interface****, and the* ***"server" as the backend database***. *These applications* ***require the use of a protocol such as ODBC to access the database***. *With the evolution of* ***web-based applications and multi-tiered architectures****,* ***many servers are involved in processing information that is ultimately rendered to the user's browser***."

"*Depending on the part of the system that is targeted for testing,* ***an understanding is required of the appropriate protocol to be used***. *Therefore, if the need is to* ***perform end-to-end testing emulating user activity from the browser****,* ***a web protocol such as HTTP/HTTPS will be employed***. *In this way,* ***interaction with the GUI can be bypassed and the tests can focus on the communication and activities of the backend servers***."

Related concepts: communication protocol, OSI model, ISO/IEC 7498-1, session layer, application layer, ODBC, JDBC, HTTP, HTTPS, HTML, SOAP, REST, DNS, FTP, IMAP, LDAP, POP3, SMTP, Windows Sockets, CORBA, TruClient, SMP, MMS, Citrix ICA, RTE, MQSeries, JSON, WSCL, two-tier client-server, multi-tier architecture, end-to-end testing, GUI bypass.

PTFL-4.2.2 (K2) — Transactions

Original syllabus statement

Understand the concept of transactions in performance testing.

The definition. "*Transactions describe the **set of activities performed by a system from the point of initiation to when one or more processes (requests, operations, or operational processes) have been completed**. **The response time of transactions can be measured for the purpose of evaluating system performance.** During a performance test these measurements are used to **identify any components that require correction or optimization***."

The think-time equation — memorise it exactly

"*Simulated transactions **can include think time** to better reflect the timing of a real user taking an action (e.g., **pressing the "SEND" button**). **The transaction response time plus the think time equals the elapsed time for that transaction.***" **elapsed time = transaction response time + think time.** The same idea appears again in **PTFL-4.2.5** as the throughput formula's denominator: **[processing time] + [think time]**.

What transaction times reveal under load. "*The transaction response times collected during the performance test **show how this measurement changes under different loads** imposed on the system. **Analysis may show no degradation under load while other measurements may show severe degradation.** By **ramping up load and measuring the underlying transaction times, it is possible to correlate the cause of degradation with the response times of one or more transactions.***"

Nesting. "*Transactions **can also be nested so that individual and aggregate activities can be measured**. This can be helpful, for example, when understanding the performance efficiency of an **online ordering system**. The tester may want to measure the **discrete steps in the order process** (e.g., **search for item, add item to cart, pay for item, confirm order**) **as well as the order process as a whole**. **By nesting transactions, both sets of information can be gathered in one test.***"

The two official questions

#24 — simulating the time a real user takes to read a prompt and enter data → think time. *Wait time*, *latency time* and *reading time* are all incorrect: **think time** is the syllabus term. **#25 — the value of nesting transactions → it allows the tester to measure a series of discrete transactions.** The rationale: **the discrete transactions can be nested together to provide response information for a series of transactions that would normally be performed together while still allowing measurement of the discrete transactions as well.** The distractors fail because: **parent and child transactions may not accomplish anything meaningful in terms of system usage**; nesting **will not affect the reporting time — it may increase reporting time**; and nesting **has nothing to do with bypassing network communications**.

Related concepts: transaction, point of initiation, transaction response time, think time, elapsed time, correction and optimization, degradation under load, ramping up load, correlation of degradation, nested transactions, discrete steps, aggregate measurement, online ordering system.

PTFL-4.2.3 (K4) — Identifying Operational Profiles

Original syllabus statement

Analyze operational profiles for system usage.

The definition. "*Operational profiles **specify distinct patterns of interaction with an application** such as from **users or other system components**. **Multiple operational profiles may be specified for a given application. They may be combined to create a desired load profile** for achieving particular performance test objectives.*" (The PTP section adds: an operational profile provides **a repeatable step-by-step flow through the application for a particular usage of the system**.)

Three principal steps: identify the data · gather the data · evaluate the data to construct the profiles.

Step 1 — Identify the data to be gathered

#	Data to gather or estimate where users interact with the SUT
1	**Different types of user personas and their roles** (e.g., **standard user, registered member, administrator, user groups with specific privileges**).
2	**Different generic tasks performed by those users/roles** (e.g., **browsing a web site for information, searching a web site for a particular product, performing role-specific activities**). **Note that these tasks are generally best modeled at a high level of abstraction** (e.g., **at the level of business processes or major user stories**).
3	**Estimated numbers of users for each role/task per unit of time over a given time period.** This information **will also be useful for subsequently building load profiles**.

Item 3 is the official exam answer

The official question describes an emergency-room doctor operational profile in full — **10 accesses per 10-hour shift, 6 patient records per access, printing, entering notes** — and asks **what is missing to construct the operational profile for this user class**. The answer: **the number of users across the system for this role is not known**. The rationale: **at this point, we need to know how many users of this type will be accessing the system. We know the concurrency because we can take the number of users and the accesses per shift to figure this out.** The distractors: **interviews are not needed for this instance because we already have the information**; **batch processing of the patient upload information** and **other system components (x-ray upload, lab results reporting)** are **considerations for setting up the load profile but not needed for the operational profile**.

Step 2 — Gather the data (four sources)

Source	What the syllabus says about it
Interviews or workshops with stakeholders	With **product owners, sales managers and (potential) end users**. These discussions **often reveal the principal operational profiles of users** and answer the fundamental question **"Who is this application intended for"**.
Functional specifications and requirements	**A valuable source of information about intended usage patterns.** Where expressed as **user stories**, the standard format **directly enables types of users to be identified** (**As a <type of user>, I want <some capability> so that <some benefit>**). Similarly, **UML Use Case diagrams and descriptions identify the "actor"** for the use case.
Usage data and metrics from similar applications	May support **identification of user types** and provide **initial indications of the expected numbers of users**. **Access to automatically monitored data (e.g., from a web master's administration tool) is recommended**, including **monitoring logs and data taken from usage of the current operational system where an update to that system is planned**.
Monitoring the behavior of users	**Performing predefined tasks with the application** may give insights into the **types of operational profiles to be modeled**. **Coordinating this task with any planned usability tests (especially if a usability lab is available) is recommended.**

Step 3 — Construct the operational profiles (three rules)

#	Rule
1	**A top-down approach is taken. Relatively simple broad operational profiles are initially created and only broken down further if this is needed to achieve performance test objectives.**
2	**Particular user profiles may be singled out as relevant** if they involve tasks that are **executed frequently**, **require critical (high risk) or frequent transactions between different system components**, or **potentially demand large volumes of data to be transferred**.
3	**Operational profiles are reviewed and refined with the principal stakeholders before being used for the creation of load profiles.**

Non-user systems also need operational profiles. "*The system under test is **not always subjected to loads imposed by the user**. Operational profiles may also be required for performance testing of the following types of system (**please note this list is not exhaustive**)*":

System type	Where the focus lies
Off-line batch processing systems	The focus lies **principally on the throughput of the batch processing system and its ability to complete within a given time period**. Operational profiles focus on **the types of processing which are demanded of the batch processes**. The syllabus example is a **stock trading system** (online and batch), whose profiles may include **payment transactions, verifying credentials, and checking compliance of legal conditions for particular types of stock transactions — each resulting in different paths being taken through the overall batch process**. **The steps for online user-based systems can also be applied in the batch processing context.**
Systems of systems	Components within a **multi-system (which may also be embedded) environment** respond to **different types of input from other systems or components**. This may require **modeling of several different operational profiles** to represent the types of input provided by those **supplier systems**, and **may involve detailed analysis (e.g., of buffers and queues) together with the system architects and based on system and interface specifications**.

Related concepts: operational profile, repeatable step-by-step flow, user persona, role, generic task, level of abstraction, business process, user story, UML use case, actor, web master's administration tool, monitoring logs, usability lab, top-down approach, high-risk transactions, data volume, stakeholder review, off-line batch processing, batch window, stock trading system, systems of systems, buffers and queues, interface specifications.

PTFL-4.2.4 (K4) — Creating Load Profiles

> **Original syllabus statement**
> Create load profiles derived from operational profiles for given performance objectives.

The definition. "*A load profile **specifies the activity which a component or system being tested may experience in production**. It consists of **a designated number of instances that will perform the actions of predefined operational profiles over a specified time period**. **Where the instances are users, the term "virtual users" is commonly applied.***"

The four inputs required to create a realistic and repeatable load profile

#	Required information
1	**The performance testing objective** (e.g., **to evaluate system behavior under stress loads**).
2	**Operational profiles which accurately represent individual usage patterns** (see §4.2.3).
3	**Known throughput and concurrency issues** (see §4.2.5).
4	**The quantity and time distribution with which the operational profiles are to be executed** such that the SUT experiences the desired load.

The four time distributions

Distribution	Syllabus definition and example
Ramp-ups	**Steadily increasing load** (e.g., **add one virtual user per minute**).
Ramp-downs	**Steadily decreasing load.**
Steps	**Instantaneous changes in load** (e.g., **add 100 virtual users every five minutes**).

Distribution	Syllabus definition and example
Predefined distributions	e.g., **volume mimics daily or seasonal business cycles**.

The syllabus's own worked example — constructing a "stress" load profile. The objective is **generating stress conditions (at or above the expected maximum for a system to handle)**. Three diagrams are combined:

Layer	What it contains
Top	A load profile consisting of a **step input of 100 virtual users**. These users perform the activities defined by **Operational Profile 1 over the entire duration of the test**. **This is typical of many performance load profiles that represent a background load.**
Middle	A load profile consisting of a **ramp-up to 220 virtual users**, **maintained for two hours before ramping down**. Each virtual user performs activities defined in **Operational Profile 2**.
Combined	**The system under test is subjected to a three-hour period of stress.**

The official K4 calculation — worked in full

The scenario. Emergency-room doctors **access the system 10 times per shift (a shift is 10 hours)** and **update 6 patient records for each access**. Shifts: **7am–5pm (day, 1000 doctors) · 2pm–midnight (evening, 1000 doctors) · 9pm–7am (night, 500 doctors)**. **Assume an even distribution of system access across a shift. Step 1 — total transactions. 2500 doctors × 10 accesses × 6 record updates = 150,000 transactions per day. Step 2 — transactions per doctor per hour.** 10 accesses over a 10-hour shift = **1 access per hour**, and each access = 6 updates, so **6 transactions per doctor per hour**. **Step 3 — doctors on duty, hour by hour (24-hour clock). 01:00–06:00 = 500 · 07:00–13:00 = 1000 · 14:00–16:00 = 2000 · 17:00–20:00 = 1000 · 21:00–23:00 = 1500 · 24:00 = 500.** That is **7 hours at 500 · 11 hours at 1000 · 3 hours at 1500 · 3 hours at 2000**. **Step 4 — transactions per band. 7 h × 500 × 6 = 21,000 · 11 h × 1000 × 6 = 66,000 · 3 h × 1500 × 6 = 27,000 · 3 h × 2000 × 6 = 36,000. Total 150,000 ✔ The answer. "Stepped ramp-up with 7 hours at 21,000 transactions, 3 hours at 27,000 transactions, 3 hours at 36,000 transactions, 11 hours at 66,000 transactions."**

Read the option numbers correctly

The figures in the winning option are band TOTALS, not hourly rates. "7 hours at 21,000 transactions" means **21,000 transactions across those seven hours**, not 21,000 per hour. This is the single most confusing thing about the official question, and recognising it is what makes the arithmetic land on 150,000. Check any load-profile option by **summing its bands and comparing against the daily total** — if the sum is not 150,000, the option is wrong.

Related concepts: load profile, virtual user, instances, operational profile aggregation, scenario, ramp-up, ramp-down, steps, predefined distribution, background load, stress load profile, daily and seasonal business cycles, stepped ramp-up, band totals.

PTFL-4.2.5 (K4) — Analyzing Throughput and Concurrency

Original syllabus statement

Analyze throughput and concurrency when developing performance tests.

The framing. "*It is important to understand **different aspects of workload: throughput and concurrency**. **To model operational and load profiles properly, both aspects should be taken into consideration.***"

System throughput

"***System throughput** is a measure of the **number of transactions of a given type that the system processes in a unit of time** — for example, the **number of orders per hour** or the **number of HTTP requests per second**. **System throughput should be***

distinguished from network throughput, which is the amount of data moved over the network *(§2.1).* ***System throughput defines load on the system.***"

Why user counts are a poor measure of load — the 60x example

"***Quite often the number of concurrent users is used to define the load for interactive systems instead of throughput.*** *This is partially true because* ***that number is often easier to find****, and partially because* ***it is the way load testing tools define load****.* ***Without defining operational profiles — what each user is doing and how intensely (which also is throughput for one user) — the number of users is not a good measure of load.***" The syllabus's own arithmetic: **if there are 500 users running short queries each minute, we have a throughput of 30,000 queries per hour. If the same 500 users are running the same queries, but one per hour, the throughput is 500 queries per hour. So there are the same 500 users, but a 60x difference between loads and at least a 60x difference in the hardware requirements for the system.**

The throughput formula — the only formula in the syllabus

System throughput = [number of virtual users] / ([processing time] + [think time]) "*Workload modeling is usually done by considering the* ***number of virtual users (execution threads)*** *and the* ***think time (delays between user actions). However, system throughput is also defined by processing time, and that time may increase as load increases.***" **The consequence sentence: "*****So when the processing time increases, throughput may significantly decrease even if everything else stays the same.***" Note that the denominator is the same **elapsed time** defined in **PTFL-4.2.2**.

Batch throughput. "*System throughput is an* ***important aspect when testing batch processing systems****. In this case, the throughput is typically* ***measured according to the number of transactions that can be accomplished within a given time frame*** *(e.g.,* ***a nightly batch processing window****).*"

Concurrency

"***Concurrency*** *is a measure of the* ***number of simultaneous / parallel threads of execution. For interactive systems, it may be a number of simultaneous / parallel users. Concurrency is usually modeled in load testing tools by setting the number of virtual users.***"

"***Concurrency is an important measure. It represents the number of parallel sessions, each of which may use its own resources. Even if throughput is the same, the amount of resources used may differ depending on concurrency.***"

Closed versus open systems (queuing theory)

Closed system	Open system
Typical test setups are closed systems, **where the number of users in the system is set (fixed population). If all users are waiting for the system's response in a closed system, no new users can arrive.**	**Many public systems are open systems — new users are arriving all the time even if all the current users are waiting for the system's response.**

The official K4 concurrency question — worked

Same doctors scenario. **What is the highest number of concurrent accesses on the system from these doctors?** Answer: **2,000**. The rationale: **the highest concurrent usage is when 2000 doctors are working from 2:00–5:00 in the afternoon. Each doctor has only one access per hour according to the question and that access has up to 6 updates. The trap:** the tempting answer is **12,000** (2000 doctors × 6 updates). But the question asks for **concurrent accesses**, and **each doctor makes one access per hour** — the six updates happen **within** that single access. **Concurrency counts sessions, not the work inside them.**

Related concepts: workload, system throughput, network throughput, transactions per unit of time, orders per hour, HTTP requests per second, virtual users, execution threads, think time, processing time, throughput formula, batch processing window, concurrency, parallel sessions, resource usage per session, queuing theory, closed system, fixed population, open system, arrival of new users.

PTFL-4.2.6 (K2) — Basic Structure of a Performance Test Script

Original syllabus statement

Understand the basic structure of a performance test script.

What a script must do. "*A performance test script should* ***simulate a user or component activity that contributes to the load on the system under test*** *(which may be* ***the whole system or one of its components****). It* ***initiates requests to the server in a proper order and at a given pace. The best way to create performance test scripts depends on the load generation approach used.***"

The three ways to create a script

Approach	What the syllabus says
Recording at the protocol level	**The traditional way — record communication between the client and the system or component on the protocol level and then play it back after the script has been parameterized and documented. The parameterization results in a scalable and maintainable script, but the task of parameterization may be time consuming.**
Recording at the GUI level	**Typically involves capturing GUI actions of a single client with a test execution tool and running that script with the load generation tool to represent multiple clients.**
Programming	**May be done using protocol requests (e.g., HTTP requests), GUI actions, or API calls.** In the case of programming scripts, **the exact sequence of requests sent to and received from the real system must be determined, which may be not trivial.**

What a script is made of. "*Usually a script is* ***one or several sections of code (written in a generic programming language with some extensions or in a specialized language) or an object, which may be presented to a user by the tool in a GUI****. In both cases the script will include* ***server requests creating load (e.g., HTTP requests)*** *and* ***some programming logic around them specifying how exactly these requests would be invoked*** *(e.g.,* ***in what order, at what moment, with what parameters, what should be checked****).* ***The more sophisticated the logic, the more need for using powerful programming languages.***"

The four structural elements

Element	Syllabus content
Overall structure	**Often the script has an initialization section (where everything gets prepared for the main part), main sections that may be executed multiple times, and a clean-up section (where necessary steps are taken to finish the test properly).**
Data collection	**To collect response times, timers should be added to the script to measure how long a request or a combination of requests takes. The timed requests should match a meaningful unit of logical work —** for example, **a business transaction for adding an item to an order or submitting an order. It is important to understand what exactly is measured: in the case of protocol-level scripts it is server and network response time only, while GUI scripts measure end-to-end time** (although what exactly is measured depends on the technology used).
Result verification and error handling	**Even in the best load testing tools, default error handling tends to be minimal (such as checking the HTTP request return code), so it is recommended to add additional checks to verify what the requests actually return. Also if any cleanup is required in case of an error, it likely will need to be implemented manually. A good practice is to verify that the script is doing what it is supposed to do using indirect methods — for example, checking the database to verify that the proper information was added.**
Other logic	**Scripts may include other logic specifying rules concerning when and how server requests will be made.** One example is **setting synchronization points**, done by **specifying that the script should wait for an event at that point before proceeding**. Synchronization points **may be used to ensure that a specific action is invoked concurrently or to coordinate work between several scripts**.

The sentence that reframes the whole objective

"***Performance testing scripts are software, so creating a performance testing script is a software development activity. It should include quality assurance and tests to verify that the script works as expected with the whole range of input data.***"

The two official questions

#29 — an advantage of conducting performance tests at the protocol level → it's scalable because the client is bypassed. The rationale: **this is a scalable method because it bypasses the UI so you can run many of these tests at the same time without requiring the client**. The distractors: protocol scripting **can be quite difficult without using a tool to capture the protocol level communication**; it is **not** the best way to evaluate the total user experience **because the UI is bypassed**; and **data correlation is more difficult because the correlation done at the UI is bypassed**. **#30 — the best method for verifying that a script added users to a system → use the script to verify that the users exist in the database.** The rationale: **scripts should contain their own checks that the process has worked, and verifying via the database directly is a more efficient way to do it (and more maintainable) than having the script do the verification through the application (which may also be wrong)**. The distractors: **no errors doesn't mean that it worked — it could just mean that the error handling is also broken**; manual checking **could be a very time-consuming approach**; and verifying through the application fails because **the application may have a defect that causes it to not find the new users**.

Related concepts: performance test script, protocol-level recording, GUI-level recording, programming, parameterization, scalable and maintainable scripts, initialization section, main section, clean-up section, timers, meaningful unit of logical work, business transaction, server and network response time, end-to-end time, default error handling, HTTP return code, indirect verification, database verification, synchronization point, concurrent invocation, script quality assurance.

PTFL-4.2.7 (K3) — Implementing Performance Test Scripts

Original syllabus statement

Implement performance test scripts consistent with the plan and load profiles.

This is a K3 objective — apply. "*Performance test scripts are implemented* ***based on the PTP and the load profiles***. *While* ***technical details of implementation will differ depending on the approach and tool(s) used, the overall process remains the same***. *A performance script is created* ***using an Integrated Development Environment (IDE) or script editor***, *to* ***simulate a user or component behavior***. ***Usually the script is created to simulate a specific operational profile*** *(although* ***it is often possible to combine several operational profiles in one script with conditional statements****).*"

Record or program? "*As the sequence of requests is determined, the script* ***may be recorded or programmed depending on the approach***. ***Recording usually ensures that it exactly simulates the real system, while programming relies on knowledge of the proper request sequence.***"

Correlation — the highest-value concept in this objective

"*If* ***recording on the protocol level*** *is used,* ***an essential step after recording in most cases is replacing all recorded internal identifiers that define context***. ***These identifiers must be made into variables that can be changed between runs with appropriate values that are extracted from the request responses*** *(e.g.,* ***a user identifier that is acquired at login and must be supplied for all subsequent transactions****).* ***This is a part of script parameterization, sometimes referred to as 'correlation'.***"

Two sentences that carry a mark each

1. The word means something different here. "*In that context the word correlation* ***has a different meaning than when used in statistics*** *(where it means* ***relationship between two or more things****).*" **Advanced load testing tools may do some correlation automatically, so it may be transparent in some cases — but in more complex cases, manual correlation or adding new correlation rules may be required. 2. The failure mode.** "***Incorrect correlation or lack of correlation is the main reason why recorded scripts fail to playback.***" **This is the answer to official question #31.**

Parameterization — and why re-using one user is dangerous

"***Running multiple virtual users with the same user name and accessing the same set of data*** *(as usually happens during playback of a recorded script without any further modification beyond necessary correlation)* ***is an easy way to get misleading results.***" The syllabus then gives **two opposite failure modes**, and both are examinable:

Failure mode	Mechanism	Direction of the error
Caching	**The data could be completely cached (copied from disk to memory for faster access)**	**Results would be much better than in production** (where such data may be **read from a disk**)
Concurrency / locking	**Using the same users and/or data can cause concurrency issues (e.g., if data is locked when a user is updating it)**	**Results would be much worse than in production**, as the software **would wait for the lock to free before the next user could lock the data for update**

"*So* ***scripts and test harnesses should be parameterized*** *(i.e.,* ***fixed or recorded data should be replaced with values from a list of possible choices****), so that* ***each virtual user uses a proper set of data****.* ***The term "proper" here means different enough to avoid problems with caching and concurrency****, which is* ***specific for the system, data, and test requirements****. This further parameterization* ***depends on the data in the system and the way the system works with this data, so it usually is done manually, although many tools provide assistance here.***"

Uniqueness. "*There are cases where* ***some data must be parameterized for the test to work more than once*** *— for example,* ***when an order is created and the order name must be unique. Unless the order's name is parameterized, the test will fail as soon as it tries to create an order with an existing (recorded) name.***"

Think times and combination. "***To match operational profiles, think times should be inserted and/or adjusted (if recorded) to generate a proper number of requests / throughput*** *as discussed in §4.2.5.*" And: "***When scripts for separate operational profiles are created, they are combined into a scenario implementing the whole load profile. The load profile controls how many virtual users are started using each script, when, and with what parameters. The exact implementation details depend on the specific load testing tool or harness.***"

The two official K3 questions

#31 — the script logs in successfully but subsequent patient look-ups fail with "user is not known to the system" → the script is not capturing and re-using the system identifier for the user. The rationale: **the problem is most likely with the system identifier that is given to that particular user when they login. When they try to do another transaction, that unique identifier needs to be passed to the system so it can provide access to the transaction.** This is **correlation**, exactly as defined above. **#32 — the first search took 5.00 seconds but subsequent identical searches respond in 0.01 seconds → you need to be sure the cache is cleared because the query results are probably being cached.** The rationale: **this is a symptom of caching and can be avoided by either varying the data or by clearing the cache.** The distractors: systems **tend to get slower, not faster, unless there is a caching issue**; and **it's not the user's information that is being re-used but rather the search results**.

Related concepts: IDE, script editor, operational profile simulation, conditional statements, recording versus programming, correlation, internal identifiers, context variables, response extraction, automatic correlation, correlation rules, playback failure, parameterization, caching, disk versus memory, concurrency issues, data locking, unique data, order name, think time adjustment, scenario, load profile control.

PTFL-4.2.8 (K2) — Preparing for Performance Test Execution

Original syllabus statement

Understand the activities involved in preparing for performance test execution.

The three main preparation activities. "*The main activities for preparing to execute the performance tests include:* ***setting up the system under test · deploying the environment · setting up the load generation and monitoring tools and making sure that all the necessary information will be collected***."

> **The non-linearity sentence — worth a mark on its own**
>
> "*It is important to ensure the test environment is* ***as close to the production environment as possible****. If this is not possible, then* ***there must be a clear understanding of the differences and how the test results will be projected on the production environment****. Ideally,* ***the true production environment and data would be used****, but* ***testing in a scaled-down environment still may help mitigate a number of performance risks***." "*It is important to remember that* ***performance is a non-linear function of the environment****, so* ***the further the environment is from production standard, the more difficult it becomes to make accurate projections for production performance****.* ***The lack of reliability of the projections and the increased risk level grow as the test system looks less like production.***" **This is the answer to official question #33.**

The three most important parts of the environment. "***The most important parts of the test environment are data, hardware and software configuration, and network configuration.***"

On data. "***The size and structure of the data could affect load test results dramatically. Using a small sample set of data or a sample set with a different data complexity for performance tests can give misleading results****, particularly when the production system will use a large set of data.* ***It is difficult to predict how much the data size affects performance before real testing is performed. The closer the test data is to the production data in size and structure, the more reliable the test results will be.***" And: "***If data is generated or altered during the test, it may be necessary to restore the original data before the next test cycle to ensure that the system is in the proper state.***"

Service virtualization. "*If some parts of the system or some of the data is* ***unavailable for performance tests for whatever reason, a workaround should be implemented****. For example,* ***a stub may be implemented to replace and emulate a third party component responsible for credit card processing****.* ***That process is often referred to as "service virtualization"*** *and* ***there are special tools available to assist with that process. The use of such tools are highly recommended to isolate the system under test.***"

The three deployment options — and when each is preferred

Deployment option	Preferred when
Traditional internal (and external) test labs	**To test the effect of a performance improvement (performance optimization) — using an isolated lab environment may be a better option to see even small variations introduced by the change.**
Cloud as an environment using Infrastructure as a Service (IaaS) — when some or all of the system is deployed to the cloud	**To minimize costs when performance testing is limited in time**, creating a test environment in the cloud **may be a more economical solution.**
Cloud as an environment using Software as a Service (SaaS) — when vendors provide the load testing service	**To load test the whole production environment end-to-end** to make sure the system will handle the load without any major issues — **testing from the cloud or a service may be more appropriate. (Note that this only works for SUTs that can be reached from a cloud.)**

Configuration. "*Whatever approach to deployment is used,* ***both hardware and software should be configured to meet the test objective and plan****.* ***If the environment matches production, it should be configured in the same way. However, if there are differences, the configuration may have to be adjusted to accommodate these differences.*** *For example,* ***if test machines have less physical memory than the production machines, software memory parameters (such as Java heap size) may need to be adjusted to avoid memory paging***."

Network. "***Proper configuration / emulation of the network is important for global and mobile systems.*** *For* ***global systems*** *(one which has* ***users or processing distributed world-wide****) one approach may be to* ***deploy load generators in places where users are located****. For* ***mobile systems****,* ***network emulation remains the most viable option due to the variances in the network types that can be used****.* ***Some load testing tools have built-in network emulation tools and there are standalone tools for network emulation.***"

Load generators must themselves be monitored — the second official question

"*Depending on the load, specific tool / load generation approach, and machine configuration,* ***more than one load generation machine may be needed. To verify the setup, machines involved in load generation should be monitored too. This will help avoid a situation where the load is not maintained properly because one of the load generators is running slowly.***" Official question #34 asks what the concern is when using **a properly configured load generation tool** to build background load. The answer: **the load generator may experience performance problems and will not be able to maintain a steady load — particularly if you are running multiples on one machine**. The distractors fail because **invalid data and production impact will not occur if the tool is configured properly**, and **log readability is not an issue as the load generator is not usually used to generate the performance results — it just creates the load on the system so the performance tests can be conducted**.

Tool configuration and validation. "*Load testing tools need to be configured to create the appropriate load. For example,* ***specific browser emulation parameters may be set or IP spoofing (simulating that each virtual user has a different IP address) may be used***." And: "***Before tests are executed, the environment and setup must be validated.*** *This is usually done by* ***conducting a controlled set of tests and verifying the outcome of the tests as well as checking that the monitoring tools are tracking the important information.*** *To verify that the test works as designed,* ***a variety of techniques may be used, including log analysis and verifying database content***. *Preparing for the test includes* ***checking that required information gets logged, the system is in the proper state, etc.*** *For example,* ***if the test changes the state of the system significantly (add / change information in database), it may be necessary to return the system to the original state before repeating the test.***"

Metrics. "*The* ***load generation tools should be properly deployed*** *and the* ***monitoring tools should be configured to collect all necessary metrics for the test***. ***The list of metrics depends on the test objectives, but it is recommended to collect at least basic metrics for all tests*** *(see §2.1.2).*"

Related concepts: SUT setup, environment deployment, load generation and monitoring tool setup, production-likeness, projection of results, non-linear function of the environment, data size and structure, data restoration, service virtualization, stub, third-party component, IaaS, SaaS, test lab, Java heap size, memory paging, network emulation, global systems, load generator distribution, multiple load generation machines, load generator monitoring, IP spoofing, browser emulation, environment validation, controlled test set, log analysis, database content verification, system state restoration.

PTFL-4.3.1 (K2) — Execution

Original syllabus statement

Understand the principal activities in running performance test scripts.

What execution consists of. "*Performance test execution involves* ***generation of a load against the SUT according to a load profile*** *(usually implemented by performance testing scripts invoked according to a given scenario),* ***monitoring all parts of the environment****, and* ***collecting and keeping all results and information related to the test****.* ***Usually advanced load testing tools / harnesses perform these tasks automatically*** *(after, of course, proper configuration). They generally* ***provide a console to enable performance data to be monitored during the test and permit necessary adjustments to be made*** *(see §5.1).* ***However, depending on the tool used, the SUT, and the specific tests being executed, some manual steps may be needed.***"

Steady state, ramp-up and ramp-down — the vocabulary that decides one mark

"*Performance tests are* ***usually focused on a steady state of the system****, i.e.,* ***when the system's behavior is stable*** *— for example,* ***when all simulated users / threads are initiated and are performing work as designed. When the load is changing (for example, when new users are added), the system's behavior is changing and it becomes more difficult to monitor and analyze test results. The stage of getting to the steady state is often referred to as the ramp-up, and the stage of finishing the test is often referred to as the ramp-down.***"

Transient states. "*It is* ***sometimes important to test transient states, when the system's behavior is changing****. This may apply, for example, to* ***the concurrent logging of a large number of users*** *or* ***spike tests****.* ***When testing transient states it is important to understand the need for careful monitoring and analysis of the results, as some standard approaches — such as monitoring averages — may be very misleading.***"

The two official questions

#35 — the purpose of a ramp-up period → to allow the system to achieve a steady state before taking measurements. The rationale: **the system must have started up and achieved a steady state for valid steady state performance metrics to be gathered.** The distractors: the goal is **to get the number of users to the point desired for the test, not to exceed that number**; **ramp up is increasing load, not decreasing it getting ready for shut down**; and checking the monitoring tools **has nothing to do with ramp up and should be tested before any official tests are started**. **#36 — a method for testing transient states → spike testing.** The rationale: **this is achieved with spike tests which force the system to transition between a low load to a high load in a short period of time.** Note how this reaches back into **PTFL-1.2.1**.

During the ramp-up. "***During the ramp-up it is advisable to implement incremental load states to monitor the impact of the steadily increasing load on the system's response. This ensures that sufficient time is allocated for the ramp-up and that the system is able to handle the load. Once the steady state has been reached, it is a good practice to monitor that both the load and the system's responses are stable and that random variations (which always exist) are not substantial.***"

Failure handling. "***It is important to specify how failures should be handled to make sure that no system issues are introduced.*** *For example,* ***it may be important for the user to logout when a failure occurs to ensure that all resources associated with that user are released.***"

Monitoring and time synchronization. "***If monitoring is built into the load testing tool and it is properly configured, it usually starts at the same time as the test execution. However, if stand-alone monitoring tools are used, monitoring should be started separately and the necessary information collected such that subsequent analysis can be carried out together with the test results.*** *The same is true for log analysis.* ***It is essential to time-synchronize all tools used, so that all information related to a specific test execution cycle can be located.***"

Watching the run. "***Test execution is often monitored using the performance test tool's console and real-time log analysis to check for issues and errors in both the test and the SUT. This helps to avoid needlessly continuing with running large-scale tests, which might even impact other systems if things go wrong*** *(e.g., if failures occur, components fail or the generated loads*

*are too low or high). **These tests can be expensive to run and it may be necessary to stop the test or make some on-the-fly adjustments to the performance test or the system configuration if the test deviates from the expected behavior.***"

The cross-check technique

"***One technique for verifying load tests which are communicating directly on the protocol level is to run several GUI-level (functional) scripts or even to execute similar operational profiles manually in parallel to the running load test. This checks that response times reported during the test only differ from the response times measured manually at the GUI level by the time spent on the client side.***" This is the practical consequence of the **PTFL-4.2.6** rule that **protocol-level scripts measure server and network response time only, while GUI scripts measure end-to-end time**.

Automated execution. "*In some cases when running performance testing in an automated way (for example, **as a part of Continuous Integration**, as discussed in §3.4) **checks must be done automatically, since manual monitoring and intervention may not be possible**. In this case, **the test set up should be able to recognize any deviations or problems and issue an alert (usually while properly completing the test). This approach is easier to implement for regression performance tests when the system's behavior is generally known, but may be more difficult with exploratory performance tests or large-scale expensive performance tests that may need adjustments to be made dynamically during the test.***"

Related concepts: load generation according to a load profile, scenario, environment monitoring, results collection, load testing console, manual steps, steady state, ramp-up, ramp-down, transient state, concurrent login, spike test, misleading averages, incremental load states, random variation, failure handling, resource release on logout, stand-alone monitoring, time synchronization, real-time log analysis, on-the-fly adjustment, protocol versus GUI cross-check, client-side time, Continuous Integration, automated alerting, regression versus exploratory performance tests.

PTFL-4.4.1 (K4) — Analyzing Results and Reporting

Original syllabus statement

Analyze and report performance test results and implications.

The analysis sequence. "*§4.1.2 discussed the various metrics in a performance test plan. **Defining these up front determines what must be measured for each test run.** After completion of a test cycle, **data should be collected for the defined metrics**. **When analyzing the data it is first compared to the performance test objective.** Once the behavior is understood, **conclusions can be drawn which provide a meaningful summary report that includes recommended actions**.*"

Where recommended actions land. "*These actions may include **changing physical components (e.g., hardware, routers)**, **changing software (e.g., optimizing applications and database calls)**, and **altering the network (e.g., load balancing, routing)**.*"

The seven data items typically analyzed — in syllabus order

Data item	What the syllabus says about it
Status of simulated (e.g., virtual) users	**This needs to be examined first. It is normally expected that all simulated users have been able to accomplish the tasks specified in the operational profile. Any interruption to this activity would mimic what an actual user may experience. This makes it very important to first see that all user activity is completed since any errors encountered may influence the other performance data.**
Transaction response time	**Can be measured in multiple ways, including minimum, maximum, average, and a percentile (e.g., 90th). The minimum and maximum readings show the extremes of the system performance. The average performance is not necessarily indicative of anything other than the mathematical average and can often be skewed by outliers. The 90th percentile is often used as a goal since it represents the majority of users attaining a specific performance threshold. It is not recommended to require 100% compliance with the performance objectives as the resources required may be too large and the net effect to the users will often be minor.**
Transactions per second	**Provides information on how much work was done by the system (system throughput).**

Data item	What the syllabus says about it
Transaction failures	**Used when analyzing transactions per second. Failures indicate the expected event or process did not complete, or did not execute. Any failures encountered are a cause for concern and the root cause must be investigated. Failed transactions may also result in invalid transactions per second data since a failed transaction will take far less time than a completed one.**
Hits (or requests) per second	**Provides a sense of the number of hits to a server by the simulated users during each second of the test.**
Network throughput	**Usually measured in bits by time interval, as in bits per second. Represents the amount of data the simulated users receive from the server every second.**
HTTP responses	**Measured per second and include possible response codes such as: 200, 302, 304, 404** — the latter indicating that **a page is not found**.

"***Although much of this information can be presented in tables, graphical representations make it easier to view the data and identify trends.***"

The seven analysis techniques

#	Technique
1	**Comparing results to stated requirements**
2	**Observing trends in results**
3	**Statistical quality control techniques**
4	**Identifying errors**
5	**Comparing expected and actual results**
6	**Comparing the results to prior test results**
7	**Verifying proper functioning of components (e.g., servers, networks)**

Correlation between metrics

"***Identifying correlation between metrics can help us understand at what point system performance begins to degrade.*** *For example,* ***what number of transactions per second were processed when the CPU reached 90% capacity and the system slowed? Analysis can help identify the root cause of the performance degradation or failure, which in turn will facilitate correction. Confirmation testing will help determine if the corrective action addressed the root cause.***"

Reporting

"***Analysis results are consolidated and compared against the objectives stated in the performance test plan.*** *These may be reported* ***in the overall test status report together with other test results, or included in a dedicated report for performance testing. The level of detail reported should match the needs of the stakeholders. The recommendations based on these results typically address software release criteria (including target environment) or required performance improvements.***"

A typical performance testing report — the four sections

Section	Contents
Executive Summary	**Completed once all performance testing has been done and all results have been analyzed and understood.** The goal is to **present concise and understandable conclusions, findings, and recommendations for management with the goal of an actionable outcome.**
Test Results	May include: **a summary providing an explanation and elaboration of the results · results of a baseline**

Section	Contents
	test that serves as a "snapshot" of system performance at a given time and forms the basis of comparison with subsequent tests — including **the date/time the test started, the concurrent user goal, the throughput measured, and key findings** (key findings may include **overall error rate measured, response time and average throughput**) · **a high-level diagram showing any architectural components that could (or did) impact test objectives · a detailed analysis (tables and charts) showing response times, transaction rates, error rates and performance analysis**, including **a description of what was observed, such as at what point a stable application became unstable and the source of failures (e.g., web server, database server)**.
Test Logs / Information Recorded	**A log of each test run should be recorded**, typically including: **date/time of test start · test duration · scripts used for test (including script mix if multiple scripts are used) and relevant script configuration data · test data file(s) used by the test · name and location of data/log files created during test · HW/SW configuration tested (especially any changes between runs) · average and peak CPU and RAM utilization on web and database servers · notes on achieved performance · defects identified.**
Recommendations	May include: **technical changes recommended, such as reconfiguring hardware or software or network infrastructure · areas identified for further analysis (e.g., analysis of web server logs to help identify root causes of issues and/or errors) · additional monitoring required of gateways, servers, and networks so that more detailed data can be obtained for measuring performance characteristics and trends (e.g., degradation)**.

The official K4 reporting question — and its unusual rationale

The scenario. Health records, **three-second** target. The measured results: **< 3 seconds: 85% of the time · < 5 seconds: 90% · < 10 seconds: 95% · < 60 seconds: 100%. How should you present the results to the stakeholders? The answer key gives C:** "*The response time for **90% of the tests is probably acceptable, but more tuning is needed to bring down the 95% response time***." **The official rationale is unusually candid, and begins by discussing A: "*A is correct but this is also a problem with the requirements being unclear.** Does it have to be **less than 3 seconds 100% of the time**? Or is it acceptable if it's **95%**? As stated though, **the system has failed the test and the requirements should be revisited**. **C might be a good result if the requirements had been better defined.** Right now, they say that **100% must be less than 3 seconds**. Realistically though, **the 90% time should be investigated to see if there are a few outliers that are bringing it down or if it is always at the 5 second mark. More tuning should be attempted though because this is a critical system and performance could be life or death impacting. B is not correct because the goal didn't state 85% of the time was acceptable. D is not correct because tuning to get to below 3 seconds 100% of the time will probably not be possible.*" **What to take from this:** the syllabus's own §2.4 warning — **stakeholders are notoriously bad at articulating performance requirements** and **testers often receive limited guidance to define the expected test results** — is the subtext of the entire question. **In the exam, choose the option that reports the result honestly against the stated objective, recommends further tuning, and does not invent an acceptance level the requirements never stated.**

The percentile rule that decides reporting questions

The 90th percentile is often used as a goal since it represents the majority of users attaining a specific performance threshold and **it is not recommended to require 100% compliance with the performance objectives as the resources required may be too large and the net effect to the users will often be minor**. Any option demanding **100% compliance** is therefore suspect — and so is any option that **declares success at a percentage the requirements never mentioned**.

Related concepts: metric definition up front, comparison to test objectives, summary report, recommended actions, physical components, routers, database call optimization, load balancing, routing, virtual user status, transaction response time, minimum, maximum, average, 90th percentile, outlier skew, 100% compliance, transactions per second, system throughput, transaction failures, root cause, hits per second, network throughput, bits per second, HTTP response codes 200/302/304/404, graphical representation, statistical quality control, confirmation testing, executive summary, baseline snapshot, architectural diagram, test log contents, script mix, peak CPU and RAM, recommendations, gateways.

Section 2 — Key Concepts

Concise, exam-ready notes drawn from the syllabus text. Every item is traceable to Chapter 4 of the CT-PT syllabus (pages 32–54).

2.1 Planning (4.1)

- **Stakeholders set the objectives, the terminology to be used and the criteria for determining whether the objective has been achieved.**
- **User-based objectives** focus on **end-user satisfaction and business goals**; users are **less concerned about feature types or how a product gets delivered — they just want to be able to do what they need to do**. **Technical objectives** focus on **operational aspects**: **a system's ability to scale**, or **under what conditions degraded performance may become apparent**.
- **Key objectives of performance testing: identifying potential risks, finding opportunities for improvement, and identifying necessary changes.**
- **Three stakeholder questions:** what **transactions** will be executed and what **average response time** is expected · what **system metrics** are to be captured and what **values** are expected · what **performance improvements** are expected **compared to previous test cycles**.
- **The PTP is created prior to any performance testing**, is **referred to by the Test Plan** (which holds **scheduling**), and **continues to be updated once performance testing begins**.
- **Twelve PTP sections:** Objective · Test Objectives · System Overview · Types of Performance Testing to be Conducted · Acceptance Criteria · Test Data · System Configuration · Test Environment · Test Tools · Profiles · Relevant Metrics · Risks.
- **Response time is a user concern, throughput is a business concern, and resource utilization is a system concern.**
- **A baseline is a set of metrics used to compare current and previously achieved performance measurements**; it **may be necessary to first create the baseline using sanitized data from a database**.
- **Data masking is mandatory under GDPR** for production data with personally identifiable information, **however in performance testing it adds risk as the masked data may not have the same data characteristics as seen in real-world use**.
- **The test environment is often a separate environment that mimics production at a smaller scale**; the PTP must state **how results will be extrapolated**. **Production may be the only viable option, but then the specific risks must be discussed.**
- **Communication:** business focus wants **product risk to objective linkage · cost versus representativeness · repeatability · project risks and dependencies · high-level activities with costs, schedule and milestones**, and finds **tooling, scripting and load generation of secondary interest**. Technology focus wants **the load-profile generation approach and their own involvement · detailed setup and execution steps related to architectural risks · repeatability steps including organizational aspects · scheduling of shared environments · mitigations if testing in production · clarity about their tasks and timing**.

2.2 Analysis, design and implementation (4.2)

- **Protocols from OSI Layer 5 (Session) to Layer 7 (Application) are most commonly used** in performance testing; **the OSI layer in focus relates to the level of the architecture being tested — lower layers for low-level embedded architectures**.
- **Groups: Database** = ODBC, JDBC · **Web** = HTTP, HTTPS, HTML · **Web Service** = SOAP, REST · **Network** = DNS, FTP, IMAP, LDAP, POP3, SMTP, Windows Sockets, CORBA · **Mobile** = TruClient, SMP, MMS · **Remote Access** = Citrix ICA, RTE · **SOA** = MQSeries, JSON, WSCL.
- **Traditional 2-tier client-server** = GUI client + backend database, **requiring a protocol such as ODBC**. **End-to-end testing from the browser uses HTTP/HTTPS**, which **bypasses GUI interaction and focuses on backend server communication**.
- **A transaction is the set of activities performed from the point of initiation to when one or more processes have been completed. Transaction response time + think time = elapsed time.**
- **Nesting transactions** lets **individual and aggregate activities** be measured **in one test** — e.g., **search, add to cart, pay, confirm** plus the whole order process.

- **Operational profiles specify distinct patterns of interaction**, from **users or other system components**; **multiple profiles may be combined into a load profile**.
- **Identify: user personas and roles · generic tasks (at a high level of abstraction, e.g., business processes or major user stories) · estimated numbers of users for each role/task per unit of time over a given time period**.
- **Gather: interviews/workshops** (product owners, sales managers, end users) · **functional specifications and requirements** (user stories give the **type of user**; **UML use cases give the actor**) · **usage data from similar applications** (web master's administration tool, monitoring logs) · **monitoring user behavior** (coordinate with **usability tests**).
- **Construct: top-down**, broad first, broken down **only if needed to achieve objectives** · single out profiles that are **executed frequently**, involve **critical (high risk) or frequent inter-component transactions**, or **demand large data volumes · review and refine with principal stakeholders before creating load profiles**.
- **Non-user profiles: off-line batch processing** (**throughput and completion within a given time period**) and **systems of systems** (**inputs from supplier systems**, **analysis of buffers and queues with system architects**).
- **A load profile specifies the activity a system may experience in production — a designated number of instances performing predefined operational profiles over a specified time period**; instances that are users are **virtual users**.
- **Four inputs: the performance testing objective · accurate operational profiles · known throughput and concurrency issues · the quantity and time distribution**. **Four distributions: ramp-ups · ramp-downs · steps · predefined distributions**.
- **System throughput = number of transactions of a given type processed in a unit of time**, and **defines load on the system**. **Distinguish it from network throughput, the amount of data moved over the network.**
- **System throughput = [number of virtual users] / ([processing time] + [think time]) — when processing time increases, throughput may significantly decrease even if everything else stays the same**.
- **The number of users is not a good measure of load without operational profiles — 500 users at one query per minute is 30,000 queries per hour; at one per hour it is 500 — a 60x difference in load and hardware requirements**.
- **Concurrency is the number of simultaneous / parallel threads of execution**, modeled by **setting the number of virtual users**. **Even if throughput is the same, resource usage may differ depending on concurrency.**
- **Closed system** = **fixed population**; **if all users are waiting, no new users can arrive**. **Open system** = **new users arrive all the time even if current users are waiting — many public systems are open**.
- **Script creation: protocol-level recording** (traditional; **parameterization gives a scalable and maintainable script but may be time consuming**) · **GUI-level recording · programming** (**the exact sequence must be determined, which may be not trivial**).
- **Script structure: initialization · main sections executed multiple times · clean-up**. **Timers** must match **a meaningful unit of logical work**. **Protocol-level scripts measure server and network response time only; GUI scripts measure end-to-end time. Default error handling is minimal**, so **add checks**; verify **indirectly**, e.g., **checking the database**. **Synchronization points** make the script **wait for an event**, ensuring **concurrent invocation or coordination between scripts**.
- **Performance testing scripts are software — creating one is a software development activity** requiring **quality assurance and tests across the whole range of input data**.
- **Correlation** = replacing **recorded internal identifiers that define context** with **variables extracted from request responses**. **It means something different from statistical correlation. Incorrect correlation or lack of correlation is the main reason why recorded scripts fail to playback.**
- **Re-using one user and one data set is misleading in two opposite directions: caching makes results much better than production**; **locking and concurrency issues make them much worse**.
- **Parameterize** so each virtual user uses **a proper set of data — different enough to avoid caching and concurrency problems**. Some data **must** be parameterized for the test to run twice — e.g., **a unique order name**.
- **Think times should be inserted and/or adjusted to generate the proper number of requests / throughput. Scripts for separate operational profiles are combined into a scenario implementing the whole load profile.**
- **Preparation = setting up the SUT · deploying the environment · setting up load generation and monitoring tools.**
- **Performance is a non-linear function of the environment — the further from production, the harder accurate projection becomes**, and **the lack of reliability and the risk level grow as the test system looks less like production**.

- **The most important parts of the test environment are data, hardware and software configuration, and network configuration. The closer the test data is to production data in size and structure, the more reliable the results.**
- **Service virtualization** replaces unavailable parts with **stubs** (e.g., **a third party credit card processing component**); **special tools are highly recommended to isolate the SUT**.
- **Deployment: test labs** for seeing **small variations from an optimization · cloud (SaaS/service)** for **end-to-end production load testing** (only if the SUT is reachable from a cloud) **· cloud (IaaS)** to **minimize costs when testing is limited in time**.
- **Configuration differences may need compensating adjustments** — e.g., **Java heap size when test machines have less physical memory, to avoid memory paging**.
- **Network emulation matters for global and mobile systems** — **deploy load generators where users are** for global systems; **emulation is the most viable option for mobile**.
- **More than one load generation machine may be needed**, and **load generators must themselves be monitored** to avoid **the load not being maintained because one generator is running slowly**.
- **IP spoofing** simulates **each virtual user having a different IP address**.
- **Validate the environment and setup before executing** — **a controlled set of tests**, **log analysis**, **verifying database content**, and **restoring the system state if the test changed it**.

2.3 Execution (4.3)

- Execution = **generating load according to a load profile · monitoring all parts of the environment · collecting and keeping all results and information**.
- **Advanced tools do this automatically** and provide **a console to monitor performance data during the test and permit necessary adjustments**; **some manual steps may still be needed**.
- **Steady state** = **the system's behavior is stable**, e.g., **all simulated users/threads initiated and performing work as designed**. **Ramp-up** = getting to steady state. **Ramp-down** = finishing the test.
- **Transient states** matter for **concurrent logging of a large number of users** and **spike tests**; **standard approaches such as monitoring averages may be very misleading**.
- **During ramp-up, implement incremental load states** to monitor the impact of steadily increasing load, ensuring **sufficient time is allocated** and **the system can handle the load**. Once steady, check that **load and responses are stable and random variations are not substantial**.
- **Specify how failures should be handled** — e.g., **logout on failure so all resources associated with that user are released**.
- **Time-synchronize all tools** so all information for a test cycle can be located; **stand-alone monitoring must be started separately**.
- **Monitor with the console and real-time log analysis** to avoid **needlessly continuing large-scale tests** that **might impact other systems**; **stop or adjust on the fly if the test deviates**.
- **Cross-check protocol-level tests** by running **GUI-level scripts or manual operational profiles in parallel** — response times should differ only by **the time spent on the client side**.
- **In CI, checks must be automatic** and the setup must **recognize deviations and issue an alert, usually while properly completing the test**; this is **easier for regression performance tests than for exploratory or large-scale expensive tests**.

2.4 Analysis and reporting (4.4)

- **Defining metrics up front determines what must be measured for each test run**; **analysis first compares data to the performance test objective**.
- **Recommended actions** may change **physical components (hardware, routers)**, **software (optimizing applications and database calls)** or **the network (load balancing, routing)**.
- **Seven data items: virtual user status (examined first) · transaction response time · transactions per second · transaction failures · hits per second · network throughput · HTTP responses**.
- **Virtual user status is examined first because any errors encountered may influence the other performance data.**

- **The average can often be skewed by outliers**; **the 90th percentile is often used as a goal since it represents the majority of users attaining a specific performance threshold**; **100% compliance is not recommended as the resources required may be too large and the net effect to the users will often be minor**.
- **Failed transactions may result in invalid transactions-per-second data since a failed transaction takes far less time than a completed one.**
- **HTTP response codes named: 200, 302, 304, 404.**
- **Seven techniques:** compare to **stated requirements** · observe **trends** · **statistical quality control** · **identify errors** · compare **expected and actual** · compare to **prior test results** · **verify proper functioning of components**.
- **Correlation between metrics** shows **at what point performance begins to degrade** — e.g., **transactions per second when CPU reached 90% capacity**. **Confirmation testing determines whether the corrective action addressed the root cause.**
- **Report sections: Executive Summary** (concise conclusions, findings and recommendations for management, **with the goal of an actionable outcome**) · **Test Results** (summary · **baseline snapshot** with **date/time, concurrent user goal, throughput measured, key findings** · **high-level architecture diagram** · **detailed tables and charts**, including **at what point a stable application became unstable and the source of failures**) · **Test Logs** · **Recommendations**.
- **Test log contents: date/time of test start** · **test duration** · **scripts used including script mix and configuration data** · **test data files** · **name and location of data/log files created** · **HW/SW configuration especially changes between runs** · **average and peak CPU and RAM utilization on web and database servers** · **notes on achieved performance** · **defects identified.**
- **Recommendations may cover: technical changes (hardware, software, network infrastructure)** · **areas for further analysis (e.g., web server log analysis for root causes)** · **additional monitoring of gateways, servers and networks for more detailed data on performance characteristics and trends such as degradation**.

2.5 Chapter 4 keyword glossary

The syllabus lists **eleven keywords** for Chapter 4 — more than any other chapter. All are examinable at **K1**.

Keyword	Definition to memorise
Concurrency	**A measure of the number of simultaneous / parallel threads of execution.** For interactive systems it may be **a number of simultaneous / parallel users**, and is **usually modeled in load testing tools by setting the number of virtual users**.
Load profile	**Specifies the activity which a component or system being tested may experience in production**, consisting of **a designated number of instances that will perform the actions of predefined operational profiles over a specified time period**. Also called a **scenario**.
Load generation	The **modeling, generation and submission of representative system loads** to the SUT — via **UI, crowds, API or captured communication protocols** (Chapter 1).
Operational profile	**Specifies distinct patterns of interaction with an application**, from users or other system components; **a repeatable step-by-step flow through the application for a particular usage of the system**.
Ramp-down	**Steadily decreasing load**; also **the stage of finishing the test**.
Ramp-up	**Steadily increasing load** (e.g., **add one virtual user per minute**); also **the stage of getting to the steady state**, during which **incremental load states** should be implemented.
System of systems	A **multi-system (which may also be embedded) environment** whose components **respond to different types of input from other systems or components**, requiring **several different operational profiles** and possibly **detailed analysis of buffers and queues**.
System throughput	**The number of transactions of a given type that the system processes in a unit of time. It defines load on the system** and **must be distinguished from network throughput. = [virtual users] / ([processing time] + [think time]).**
Test plan	The document that **refers to the PTP** and **includes relevant scheduling information**. The **PTP** itself is **created prior to any performance testing** and **continues to be updated once testing begins**.

Keyword	Definition to memorise
Think time	**Delays between user actions**, inserted to **better reflect the timing of a real user taking an action**. **Transaction response time + think time = elapsed time.**
Virtual user	The term applied to **load profile instances that are users**; **concurrency is usually modeled by setting the number of virtual users**, also described as **execution threads**.

Section 3 — Tables and Summaries

3.1 The 20 marks, objective by objective

LO	K	Qs	Sample Exam #	What the question actually tests
4.1.1	K4	1	#18	**User-based versus technical objective** — scalability is technical
4.1.2	K4	1	#19	**Which information feeds the critical objective** — who, what, how often
4.1.3	K4/ K3	**2**	#20, #21	**Business audience versus technical audience** — same four options, twice
4.2.1	K2	**2**	#22, #23	**Protocol group matching** — web / database / web service / network
4.2.2	K2	**2**	#24, #25	**Think time**, and **the value of nesting transactions**
4.2.3	K4	1	#26	**What is missing from an operational profile** — the number of users for the role
4.2.4	K4	1	#27	**Building the load profile** — the 150,000-transaction calculation
4.2.5	K4	1	#28	**Peak concurrency** — 2,000, not 12,000
4.2.6	K2	**2**	#29, #30	**Protocol-level scalability**, and **verifying via the database**
4.2.7	**K3**	**2**	#31, #32	**Correlation failure**, and **caching**
4.2.8	K2	**2**	#33, #34	**Non-production environments increase risk**, and **load generators can slow down**
4.3.1	K2	**2**	#35, #36	**Purpose of ramp-up**, and **spike tests exercise transient states**
4.4.1	K4	1	#37	**Reporting percentile results honestly against an unclear requirement**

3.2 Protocol group lookup

Protocol	Group
ODBC · JDBC (and vendor-specific)	**Database**
HTTP · HTTPS · HTML	**Web**
SOAP · REST	**Web Service**
DNS · FTP · IMAP · LDAP · POP3 · SMTP · Windows Sockets · CORBA	**Network**
TruClient · SMP · MMS	**Mobile**
Citrix ICA · RTE	**Remote Access**
MQSeries · JSON · WSCL	**SOA**

> **The one protocol answer that surprises people**
>
> When the stem says **"testing the speed of the response of a web service"**, the official answer is **HTTP** — a **web** protocol — not **REST**, even though REST is listed under **Web Service**. Answer these questions by asking **which protocol carries the traffic under test**, and accept the syllabus's own group labels without arguing with them.

3.3 The PTP at a glance — twelve sections

#	Section	The one thing to remember
1	**Objective**	**Goals, strategies and methods** — enabling **a quantifiable answer to the adequacy and readiness of the system to perform under load**
2	**Test Objectives**	**Listed for each type of stakeholder**
3	**System Overview**	**Context for measurement** plus a **high-level description of the functionality being tested under load**
4	**Types of Performance Testing**	**Listed with the purpose of each type**
5	**Acceptance Criteria**	**Response time = user concern · throughput = business concern · resource utilization = system concern.** Related to **objectives, SLAs and baseline values**
6	**Test Data**	**User account data · user input data · database**; creation covers **extraction, import, new data, backups, masking/anonymizing (GDPR — but masking adds risk)**
7	**System Configuration**	**Servers, tiers, hardware, software, external systems — all with versions — plus the SUT build identifier**
8	**Test Environment**	**Mimics production at a smaller scale**; must state **how results will be extrapolated**; production is a **last resort with risks discussed**
9	**Test Tools**	**Tools and versions** for **scripting, executing and monitoring**, including **points of presence**
10	**Profiles**	**Operational profiles aggregate into a load profile (a scenario)**
11	**Relevant Metrics**	**Too many measurements make analysis difficult and can negatively impact the application's actual performance**
12	**Risks**	**What is not measured**, **limitations of the testing**, and **limitations of the test environment**

3.4 The PTP metrics table (syllabus §4.1.2)

Type	Metric
Virtual User Status	**# Passed · # Failed**
Transaction Response Time	**Minimum · Maximum · Average · 90% Percentile**
Transactions Per Second	**# Passed / second · # Failed / second · # Total / second**
Hits (e.g., on database or web server)	**Hits / second** — Minimum · Maximum · Average · Total
Throughput	**Bits / second** — Minimum · Maximum · Average · Total
HTTP Responses Per Second	**Responses / second** — Minimum · Maximum · Average · Total; **response by HTTP response codes**
CPU usage (performance monitoring)	**% of available CPU used**
Memory usage (performance monitoring)	**% of available memory used**

3.5 Who hears what — the two-mark communication table

Message	Business focus	Technology focus
Cost of the test system, and how representative of production it will be	✔️ **(official answer)**	—
The connection between product risks and performance test objectives	✔️	—

Message	Business focus	Technology focus
Repeatability — difficult, or repeatable with minimum effort	✔️	✔️ **(the steps required)**
Project risks: constraints, dependencies, infrastructure, key staff	✔️	—
High-level activities plus costs, schedule and milestones	✔️	—
Detailed setup and execution steps, ramp rates and durations	—	✔️ **(official answer)**
The planned approach to generating load profiles, and their involvement	—	✔️
Relation of the testing to the architectural risks	—	✔️
Scheduling of shared test environments	—	✔️
Mitigations if testing must run in production	—	✔️ **(and required for either audience to make the option complete)**
Tooling, scripting and load generation detail	**Secondary interest**	✔️

3.6 Operational profile versus load profile

	Operational profile	Load profile
Definition	**Distinct patterns of interaction with an application**, from **users or other system components**; **a repeatable step-by-step flow for a particular usage**	**The activity a component or system may experience in production — a designated number of instances performing predefined operational profiles over a specified time period**
Answers	**What does one kind of user do, and in what order?**	**How many of them, doing it when?**
Needs	**Personas and roles · generic tasks at a high level of abstraction · estimated numbers of users per role/task per unit of time**	**The testing objective · accurate operational profiles · known throughput and concurrency issues · the quantity and time distribution**
Built by	**Top-down**, broad first, **broken down only if needed**, then **reviewed and refined with principal stakeholders**	**Aggregating operational profiles**, applying **ramp-ups, ramp-downs, steps or predefined distributions**
Also called	—	**Scenario**

3.7 The K4 arithmetic, step by step (the doctors scenario)

Step	Working	Result
Total doctors	**1000 day + 1000 evening + 500 night**	**2,500**
Transactions per doctor per day	**10 accesses × 6 record updates**	**60**
Total transactions per day	**2,500 × 60**	**150,000**
Accesses per doctor per hour	**10 accesses ÷ 10-hour shift**	**1**
Transactions per doctor per hour	**1 access × 6 updates**	**6**
Hours at 500 doctors	**01:00–06:00 and 24:00**	**7 hours → 21,000**
Hours at 1000 doctors	**07:00–13:00 and 17:00–20:00**	**11 hours → 66,000**
Hours at 1500 doctors	**21:00–23:00** (evening + night overlap)	**3 hours → 27,000**
Hours at 2000 doctors	**14:00–16:00** (day + evening overlap)	**3 hours → 36,000**

Step	Working	Result
Check	**21,000 + 66,000 + 27,000 + 36,000**	**150,000** ✔️
Peak concurrency	**The largest headcount, one access each per hour**	**2,000 — not 12,000**

3.8 Script structure and the two implementation hazards

Topic	What to remember
Overall structure	**Initialization · main sections executed multiple times · clean-up**
Timers	Must match **a meaningful unit of logical work** — e.g., a business transaction
What is measured	**Protocol-level scripts: server and network response time only. GUI scripts: end-to-end time.**
Error handling	**Default handling is minimal (e.g., HTTP return code) — add checks**; **cleanup on error is usually manual**; **verify indirectly, e.g., check the database**
Synchronization points	**Wait for an event** to **ensure concurrent invocation or coordinate work between several scripts**
Correlation	Replace **recorded internal identifiers that define context** with **variables extracted from request responses**. **Incorrect or missing correlation is the main reason recorded scripts fail to playback.**
Parameterization	Replace **fixed or recorded data with values from a list of choices** so each virtual user uses **a proper set of data — different enough to avoid caching and concurrency problems**. Some data **must** be unique (e.g., **order name**).
Caching hazard	**Results much BETTER than production**, because **data is copied from disk to memory**
Locking hazard	**Results much WORSE than production**, because **the software waits for the lock to free**

3.9 Execution vocabulary

Term	Meaning	Why it is examined
Ramp-up	**The stage of getting to the steady state**	**Its purpose is to allow the system to achieve a steady state before taking measurements** — not to exceed the target load
Steady state	**When the system's behavior is stable** — all simulated users initiated and performing work as designed	**Performance tests are usually focused on it**
Ramp-down	**The stage of finishing the test**	Distractors offer it as the purpose of ramp-up
Transient state	**When the system's behavior is changing**	Tested by **spike tests** and **concurrent logging of a large number of users**; **monitoring averages may be very misleading**
Incremental load states	Implemented **during the ramp-up** to monitor the impact of steadily increasing load	Ensures **sufficient ramp-up time** and that **the system can handle the load**
Time synchronization	**Essential across all tools used**	So **all information related to a specific test execution cycle can be located**

3.10 What to analyse, in order, and what each tells you

#	Data item	The point
1	**Virtual user status**	**Examine first — any errors encountered may influence the other performance data**
2	**Transaction response time**	**Min and max show the extremes**; **the average can be skewed by outliers**; **the 90th**

#	Data item	The point
		percentile is often the goal; **100% compliance is not recommended**
3	**Transactions per second**	**How much work was done by the system — system throughput**
4	**Transaction failures**	**Any failure is a cause for concern**; **failures may invalidate the transactions-per-second data because a failed transaction takes far less time**
5	**Hits (requests) per second**	**The number of hits to a server by the simulated users during each second**
6	**Network throughput**	**Bits per second — the amount of data the simulated users receive from the server every second**
7	**HTTP responses**	**Per second, by code: 200, 302, 304, 404**

3.11 One-line answer to every learning objective

LO	If you remember only one sentence
4.1.1	**Stakeholders set objectives, terminology and criteria**; **user-based = satisfaction and business goals**, **technical = ability to scale and conditions of degradation**.
4.1.2	The **PTP has twelve sections**, is written **before testing** and **updated during it**; **response time is a user concern, throughput a business concern, resource utilization a system concern**.
4.1.3	**Cost and risk-to-objective linkage go to business; steps, ramps and schedules go to technical** — and any option is wrong if it is **incomplete**.
4.2.1	**Layers 5 to 7 of the OSI model**; learn the **seven protocol groups**, and remember the official answer for a web service test is **HTTP**.
4.2.2	**Transaction response time + think time = elapsed time**, and **nesting measures discrete steps and the whole process in one test**.
4.2.3	Operational profiles need **personas, tasks and the number of users per role per unit of time** — and the missing item in the official question is **the number of users**.
4.2.4	A load profile applies **quantity and time distribution** (**ramp-up, ramp-down, steps, predefined**) to operational profiles; **check any option by summing its bands against the daily total**.
4.2.5	**Throughput = virtual users ÷ (processing time + think time)**; **concurrency counts sessions, not the work inside them**; **closed = fixed population, open = new arrivals**.
4.2.6	**Initialization, main, clean-up**; **protocol scripts measure server and network time only and scale because the client is bypassed**; **verify via the database**.
4.2.7	**Correlation** replaces context identifiers, and **its absence is the main reason recorded scripts fail to playback**; **caching flatters results, locking punishes them**.
4.2.8	**Performance is a non-linear function of the environment** — less like production means **less reliable projections and more risk**; **monitor the load generators too**.
4.3.1	**Ramp-up exists to reach a steady state before measuring**; **spike tests exercise transient states**; **time-synchronize every tool**.
4.4.1	**Check virtual user status first**; **the 90th percentile is the usual goal and 100% compliance is not recommended**; report **honestly against the stated objective** and recommend **further tuning**.

Section 4 — Demo Questions with Answers

*Questions 1–20 are the **official ISTQB CT-PT Sample Exam questions for Chapter 4** — questions #18 to #37, covering all thirteen learning objectives — reproduced with their official correct answers and rationale. Questions 21–70 are additional items derived strictly from the syllabus text, in the same style and difficulty. Correct options are marked ✔ — cover the green box and self-test first.*

Part A — The 20 official Chapter 4 questions

Q1. OFFICIAL · Sample Exam #18 · K4

You are working on a project that tracks health history information for patients across a region. The number of records handled by the system is in the millions due to the large number of patients in the region. Patient information must be accessible to doctors in offices, hospitals and urgent care facilities. The information should be presented to the requestor within three seconds of request, particularly for patients with critical allergies and preconditions.

Which of the following is a technical objective for performance that could be applicable to this project?

a) The response time must be within three seconds from the time the request is sent when there are 100 concurrent users making similar requests

b) The system must be able to scale to 10 million patient records with no degradation in the performance ✔

c) The system must perform at or above the level of the legacy system when handling a similar load and responding to a similar request

d) The response time must remain the same when the disaster recovery system is in use rather than the primary system and the switchover must cause no discernable downtime

Correct answer: b | LO: PTFL-4.1.1 (K4) | Syllabus §4.1.1

Why: (b) Scalability is a technical objective for performance. (a) is not correct because **this is a user-based objective**. (c) is not correct because **this is also a user-based objective and not a very clear one at that**. (d) is not correct because **this is primarily a robustness test rather than a performance test**. The syllabus distinction: **user-based objectives focus primarily on end-user satisfaction and business goals**, while **technical objectives focus on operational aspects and providing answers to questions regarding a system's ability to scale, or under what conditions degraded performance may become apparent**.

Q2. OFFICIAL · Sample Exam #19 · K4

You are working on a project that tracks health history information for patients across a region. The number of records handled by the system is in the millions due to the large number of patients in the region. Patient information must be accessible to doctors in offices, hospitals and urgent care facilities. The information should be presented to the requestor within three seconds of request, particularly for patients with critical allergies and preconditions.

You have been asked to write a performance test plan for this project. Which of the following is the information you will need to deal with the most critical performance objective?

a) Who can access what data and how often will they do it ✔

b) How is user access authenticated and authorized

c) Where will the data be stored and how much storage is available

d) What is the expected use of the data after it has been presented to the user

Correct answer: a | LO: PTFL-4.1.2 (K4) | Syllabus §4.1.2

Why: (a) Because data retrieval time is critical to the project, you need to know who can access that data (so you can get the user profiles created), what data they will access (so you can be sure you have the right data available) and how often will they access it (so you can set up an operational profile). (b) is not correct because **this is more related to security than performance testing and is not the most critical performance objective**. (c) is not correct because **the data storage may be something to investigate if the data retrieval proves slow, but we don't need to know it to conduct the tests**. (d) is not correct because **that is after retrieval and the scenario just discusses presenting the information to the requestor**.

Q3. OFFICIAL · Sample Exam #20 · K4

You are working on a project that tracks health history information for patients across a region. The number of records handled by the system is in the millions due to the large number of patients in the region. Patient information must be accessible to doctors in offices, hospitals and urgent care facilities. The information should be presented to the requestor within three seconds of request, particularly for patients with critical allergies and preconditions.

You have been asked to prepare a presentation for the business stakeholders regarding your plan for performance testing. Which of the following is an example set of information that should be shared with these stakeholders?

a) Once configured, the performance test system will require data loading. Once loaded, we will next proceed to running a small set of sample scripts to verify the output. When those succeed we will proceed with the performance test script which ramps up users at a rate of 10 per minute until we reach the target number of concurrent users. This load will then be maintained for 2 hours.

b) The performance test system will cost $240,000 which will include the hardware and setup. This system will allow us to create a test system that is representative of the production system and will allow us to simulate production-like conditions. ✔

c) Because the cost of a performance test system is prohibitive, we will conduct the performance testing in the production environment using live data.

d) The product risks include data contention issues, data access issues, locking issues that will reject concurrent requests, network bandwidth constraints, data seek errors, data seek slowdowns or bottlenecks, and difficulty for the user in processing the returned data.

Correct answer: b | LO: PTFL-4.1.3 (K4) | Syllabus §4.1.3

Why: (b) This is information that should be conveyed to the business stakeholders. It addresses the syllabus factor that **stakeholders must be made aware of the balance between the cost of planned performance tests and how representative the performance testing results will be, compared to production conditions.** (a) is not correct because **this is information that should be shared with the technical stakeholders**. (c) is not correct because **it is incomplete. If testing will be conducted in production, mitigation plans must also be made so that any impact to production is understood.** (d) is not correct because **this is a list of risks but doesn't include a list of how the tests will help mitigate these risks.**

Q4. OFFICIAL · Sample Exam #21 · K4

You are working on a project that tracks health history information for patients across a region. The number of records handled by the system is in the millions due to the large number of patients in the region. Patient information must be accessible to doctors in offices, hospitals and urgent care facilities. The information should be presented to the requestor within three seconds of request, particularly for patients with critical allergies and preconditions.

You have been asked to prepare a presentation for the technical stakeholders regarding your plan for performance testing. Which of the following is an example set of information that should be shared with these stakeholders?

a) Once configured, the performance test system will require data loading. Once loaded, we will next proceed to running a small set of sample scripts to verify the output. When those succeed we will proceed with the performance test script which ramps up users at a rate of 10 per minute until we reach the target number of concurrent users. This load will then be maintained for 2 hours. ✔

b) The performance test system will cost $240,000 which will include the hardware and setup. This system will allow us to create a test system that is representative of the production system and will allow us to simulate production-like conditions.

c) Because the cost of a performance test system is prohibitive, we will conduct the performance testing in the production environment using live data.

d) The product risks include data contention issues, data access issues, locking issues that will reject concurrent requests, network bandwidth constraints, data seek errors, data seek slowdowns or bottlenecks, and difficulty for the user in processing the returned data.

Correct answer: a | LO: PTFL-4.1.3 (K4) | Syllabus §4.1.3

Why: (a) This is information that should be conveyed to the technical stakeholders because it shows how the tests will be conducted. It addresses the syllabus factor that **detailed steps in the setup and execution of the performance tests must be explained**. (b) is not correct because **this is information that should be shared with the business stakeholders**. (c) is not correct

because **it is incomplete. If testing will be conducted in production, mitigation plans must also be made so that any impact to production is understood.** (d) is not correct because **this is a list of risks but doesn't include a list of how the tests will help mitigate these risks. Note that questions #20 and #21 use an identical option set — the audience in the stem is the only discriminator.**

Q5. OFFICIAL · Sample Exam #22

If your performance test is testing the speed of the response of a web service, which protocol is being used?

a) ODBC

b) HTTP ✔

c) REST

d) SMTP

Correct answer: b | LO: PTFL-4.2.1 (K2) | Syllabus §4.2.1

Why: (b) This is an example of a web protocol. (a) is **an example of a database protocol**. (c) is **an example of a web service protocol**. (d) is **an example of a network protocol**. **Note the subtlety:** the stem mentions a *web service*, and REST is indeed classified by the syllabus under **Web Service** — but the official answer is **HTTP**, the protocol actually carrying the traffic under test. Answer these from the syllabus's own group labels.

Q6. OFFICIAL · Sample Exam #23

If your performance test is testing the speed with which database requests are sent and received, which protocol is being used?

a) ODBC ✔

b) HTTP

c) REST

d) SMTP

Correct answer: a | LO: PTFL-4.2.1 (K2) | Syllabus §4.2.1

Why: (a) This is an example of a database protocol. (b) is **an example of a web protocol**. (c) is **an example of a web service protocol**. (d) is **an example of a network protocol**. The syllabus also notes that **traditional 2-tier applications built with a client-server model** — GUI client, backend database — **require the use of a protocol such as ODBC to access the database**.

Q7. OFFICIAL · Sample Exam #24

If you are testing from the UI and you need to simulate the amount of time it will take a real user to read a prompt and enter data in a field, what should you implement in your test script?

a) Wait time

b) Think time ✔

c) Latency time

d) Reading time

Correct answer: b | LO: PTFL-4.2.2 (K2) | Syllabus §4.2.2

Why: (b) This is referred to as think time. (a), (c) and (d) **are incorrect** — they are not syllabus terms. The syllabus states that **simulated transactions can include think time to better reflect the timing of a real user taking an action (e.g., pressing the "SEND" button)**, and that **the transaction response time plus the think time equals the elapsed time for that transaction**.

Q8. OFFICIAL · Sample Exam #25

What is the value of nesting transactions for performance testing?

a) It supports the concept of parent and child transactions

b) It allows the tester to measure a series of discrete transactions ✔

c) It speeds the reporting time for the performance results

d) It bypasses the network communication time by sending the transaction directly to the server that will process it

Correct answer: b | LO: PTFL-4.2.2 (K2) | Syllabus §4.2.2

Why: (b) The discrete transactions can be nested together to provide response information for a series of transactions that would normally be performed together while still allowing measurement of the discrete transactions as well. (a) is not correct because **parent and child transactions may not accomplish anything meaningful in terms of system usage**. (c) is not correct because **it will not affect the reporting time, particularly if the discrete transactions are tracked as well — it may increase reporting time**. (d) is not correct because **this has nothing to do with bypassing network communications**.

Q9. OFFICIAL · Sample Exam #26 · K4

You are working on a project that tracks health history information for patients across a region. The number of records handled by the system is in the millions due to the large number of patients in the region. Patient information must be accessible to doctors in offices, hospitals and urgent care facilities. The information should be presented to the requestor within three seconds of request, particularly for patients with critical allergies and preconditions. One of the operational profiles you have identified is an emergency room doctor. You have determined that this person will access the system 10 times per shift (a shift is 10 hours) and that they will normally view 6 patient records for each access. They will print those patient records to be retained in the patient's file at the hospital. They will also enter notes into the database regarding the patient's treatment. For new patients, another user will enter the information into the system.

Given this information, what is missing to construct the operational profile for this user class?

a) No interviews were conducted to better understand the user class

b) The number of users across the system for this role is not known ✔

c) Batch processing of the patient upload information has not been considered

d) Other system components that may be required (x-ray upload, lab results reporting) have not been considered

Correct answer: b | LO: PTFL-4.2.3 (K4) | Syllabus §4.2.3

Why: (b) At this point, we need to know how many users of this type will be accessing the system. We know the concurrency because we can take the number of users and the accesses per shift to figure this out. (a) **is not needed for this instance because we already have the information.** (c) and (d) **are considerations for setting up the load profile but not needed for the operational profile.** The syllabus lists three data items to identify: **user personas and roles · generic tasks · estimated numbers of users for each role/task per unit of time over a given time period** — and it is the third that is missing here.

Q10. OFFICIAL · Sample Exam #27 · K4 calculation

You are working on a project that tracks health history information for patients across a region. The number of records handled by the system is in the millions due to the large number of patients in the region. Patient information must be accessible to doctors in offices, hospitals and urgent care facilities. The information should be presented to the requestor within three seconds of request, particularly for patients with critical allergies and preconditions. One of the operational profiles you have identified is an emergency room doctor. You have determined that this person will access the system 10 times per shift (a shift is 10 hours) and that they will normally update 6 patient records for each access by entering notes into the database regarding the patient's treatment. They will print those patient records to be retained in the patient's file at the hospital. For new patients, another user will enter the information into the system. The shifts these doctors work are: 7am – 5pm (day shift), 2pm – midnight (evening shift), 9pm to 7am (night shift). There are 1000 doctors that work the day shift, 1000 that work the evening shift, and 500 that work the night shift. Assuming an even distribution of the system access across a shift…

…which of the following is the proper load profile for these doctors?

a) 6250 transaction per hour

b) Steady ramp-up increasing the transactions by 15,000 per hour starting at 15,000 transactions and ending at 150,000 transactions

c) Stepped ramp-up with 7 hours at 21,000 transactions, 3 hours at 27,000 transactions, 3 hours at 36,000 transactions, 11 hours at 66,000 transactions ✔

d) Separate tests for 120,000 transactions per hour and 30,000 transactions per hour

Correct answer: c | LO: PTFL-4.2.4 (K4) | Syllabus §4.2.4

Why: (c) is correct. The official calculation: **total transactions should be 2500 doctors × 10 accesses × 6 record updates = 150,000 transactions per day.** Hourly load: **7 hours of 500 doctors = 21,000 transactions · 11 hours of 1000 doctors = 66,000 transactions · 3 hours of 1500 doctors = 27,000 transactions · 3 hours of 2000 doctors = 36,000 transactions.** The doctors-per-hour distribution on a 24-hour clock is **500 at 01:00–06:00, 1000 at 07:00–13:00, 2000 at 14:00–16:00, 1000 at 17:00–20:00, 1500 at 21:00–23:00, and 500 at 24:00**. **The figures in the option are band totals, not hourly rates** — and they sum to **150,000**, which is what confirms the answer.

Q11. OFFICIAL · Sample Exam #28 · K4 calculation

You are working on a project that tracks health history information for patients across a region. The number of records handled by the system is in the millions due to the large number of patients in the region. Patient information must be accessible to doctors in offices, hospitals and urgent care facilities. The information should be presented to the requestor within three seconds of request, particularly for patients with critical allergies and preconditions. One of the operational profiles you have identified is an emergency room doctor. You have determined that this person will access the system 10 times per shift (a shift is 10 hours) and that they will normally update 6 patient records for each access by entering notes into the database regarding the patient's treatment. They will print those patient records to be retained in the patient's file at the hospital. For new patients, another user will enter the information into the system. The shifts these doctors work are: 7am – 5pm (day shift), 2pm – midnight (evening shift), 9pm to 7am (night shift). There are 1000 doctors that work the day shift, 1000 that work the evening shift, and 500 that work the night shift. Assuming an even distribution of the system access across a shift…

…what is the highest number of concurrent accesses on the system from these doctors?

a) 1,000

b) 2,000 ✔

c) 6,000

d) 12,000

Correct answer: b | LO: PTFL-4.2.5 (K4) | Syllabus §4.2.5

Why: (b) The highest concurrent usage is when 2000 doctors are working from 2:00–5:00 in the afternoon. Each doctor has only one access per hour according to the question and that access has up to 6 updates. (a), (c) and (d) are incorrect. **The trap is 12,000** — 2000 doctors × 6 updates — but **concurrency counts simultaneous sessions, not the work performed inside them**. The syllabus defines concurrency as **a measure of the number of simultaneous / parallel threads of execution**.

Q12. OFFICIAL · Sample Exam #29

What is one of the advantages to conducting performance tests at the protocol level?

a) It's the easiest method for manual scripting

b) It's the best way to evaluate the total user experience

c) It's scalable because the client is bypassed ✔

d) It's the best way to handle data correlation

Correct answer: c | LO: PTFL-4.2.6 (K2) | Syllabus §4.2.6

Why: (c) This is a scalable method because it bypasses the UI so you can run many of these tests at the same time without requiring the client. (a) is not correct because **this scripting can be quite difficult without using a tool to capture the protocol level communication**. (b) is not correct because **the UI is bypassed**. (d) is not correct because **data correlation is more difficult because the correlation done at the UI is bypassed**. Recall also that **protocol-level scripts measure server and network response time only, while GUI scripts measure end-to-end time**.

Q13. OFFICIAL · Sample Exam #30

What is the best method to use for verifying that a performance test script added users to a system?

a) Check the error output from the script to verify no errors occurred

b) Manually check via an application to see if the users were created

c) Use the script to verify through the application that the users were created

d) Use the script to verify that the users exist in the database ✔

Correct answer: d | LO: PTFL-4.2.6 (K2) | Syllabus §4.2.6

Why: (d) Scripts should contain their own checks that the process has worked and verifying via the database directly is a more efficient way to do it (and more maintainable) than having the script do the verification through the application (which may also be wrong) as this adds to the maintenance requirements of the script. (a) is not correct because **no errors doesn't mean that it worked — it could just mean that the error handling is also broken**. (b) is not correct because **this could be a very time-consuming approach**. (c) is not correct because **the application may have a defect that causes it to not find the new users**. The syllabus recommends exactly this: **verify that the script is doing what it is supposed to do using indirect methods — for example, checking the database to verify that the proper information was added**.

Q14. OFFICIAL · Sample Exam #31 · K3

You are working on a project that tracks health history information for patients across a region. The number of records handled by the system is in the millions due to the large number of patients in the region. Patient information must be accessible to doctors in offices, hospitals and urgent care facilities. The information should be presented to the requestor within three seconds of request, particularly for patients with critical allergies and preconditions. One of the operational profiles you have identified is an emergency room doctor. You have determined that this person will access the system 10 times per shift (a shift is 10 hours) and that they will normally view 6 patient records for each access. They will print those patient records to be retained in the patient's file at the hospital. They will also enter notes into the database regarding the patient's treatment. For new patients, another user will enter the information into the system. You have created a performance script that logs in as a doctor (from a list of doctors) and then performs the patient look ups. When you run the script, the login works, but then you are not able to perform the patient look ups. You are getting an error that indicates the user is not known to the system.

What is likely your problem?

a) The script is not capturing and re-using the system identifier for the user ✔

b) You cannot use the same user to log in and do the patient look ups because the user has expired

c) The script needs to pass the login username/password for each transaction

d) The login information from the previous user is cached by the system and you need to clear the cache before you can log in as a new user

Correct answer: a | LO: PTFL-4.2.7 (K3) | Syllabus §4.2.7

Why: (a) The problem is most likely with the system identifier that is given to that particular user when they login. When they try to do another transaction, that unique identifier needs to be passed to the system so it can provide access to the transaction. (b) is not correct because **there is no indication that this is the problem**. (c) is not correct because **the question indicates that the doctor performs multiple lookups per access**. (d) is not correct because **if this were happening, it would log in as the previous user, not error out on the login**. This is **correlation** — and the syllabus states that **incorrect correlation or lack of correlation is the main reason why recorded scripts fail to playback**.

Q15. OFFICIAL · Sample Exam #32 · K3

You are testing a sales application for an e-commerce system. You are particularly interested in the response time for when a user enters text to be used to search for an item in the database. You have noticed that the first time you ran the tests it took 5.00 seconds to respond, but subsequent queries with the same data are responding in 0.01 seconds.

What should you have done during your scripting to prevent this issue?

a) There's no issue, the system is just getting faster

b) You need to log in each time to ensure that the transaction is performed again

c) You need to be sure the cache is cleared because the query results are probably being cached ✔

d) You need to use a different user for each test to avoid the user's information being re-used without being restored

Correct answer: c | LO: PTFL-4.2.7 (K3) | Syllabus §4.2.7

Why: (c) This is a symptom of caching and can be avoided by either varying the data or by clearing the cache. (a) is not correct and is **highly unlikely as systems tend to get slower, not faster, unless there is a caching issue**. (b) is not correct because **it's not the user's information that is being re-used but rather the search results**. (d) is not correct **for the same reason**. The syllabus warns that **the data could be completely cached (copied from disk to memory for faster access) and results would be much better than in production (where such data may be read from a disk)**.

Q16. OFFICIAL · Sample Exam #33

What happens when the performance test system is not equivalent to the production environment?

a) Projections become less reliable and risk increases because the results may not be representative ✔

b) The tests will tend to run more quickly because they are not burdened by production data

c) The results will be easier to understand because the system can be configured for a particular test

d) Projects become more reliable because of the targeted focus and this results in risk being reduced

Correct answer: a | LO: PTFL-4.2.8 (K2) | Syllabus §4.2.8

Why: (a) The less like production the performance testing system is, the less reliable are the results and this increases the risk of making incorrect assumptions. (b) is not correct — **it could happen, but not necessarily**. (c) is not correct because **there is no reason to think the results would be easier to understand with a different configuration**. (d) is not correct because **the project risk would actually be increased in this case**. The syllabus sentence: **performance is a non-linear function of the environment, so the further the environment is from production standard, the more difficult it becomes to make accurate projections**.

Q17. OFFICIAL · Sample Exam #34

What is a concern when using a properly configured load generation tool to build the background load for the performance tests?

a) The load generated will contain invalid data

b) The load generator may experience performance problems and will not be able to maintain a steady load ✔

c) The load generated may affect the production system and the production data

d) The load generator log reports may be difficult to read, resulting in problems with interpreting the performance results

Correct answer: b | LO: PTFL-4.2.8 (K2) | Syllabus §4.2.8

Why: (b) A load generator, particularly if you are running multiples on one machine, may slow down enough that it is no longer

maintaining the desired load on the system. (a) and (c) **will not occur if the tool is configured properly**. (d) is not an issue as **the load generator is not usually used to generate the performance results — it just creates the load on the system so the performance tests can be conducted**. The syllabus therefore requires that **machines involved in load generation should be monitored too**.

Q18. OFFICIAL · Sample Exam #35

What is the purpose of having a ramp up period at the beginning of performance tests?

a) To increase the number of users beyond the desired load

b) To allow the system to achieve a steady state before taking measurements ✔

c) To allow the system to achieve an orderly shut down after the tests

d) To ensure the performance monitoring tools are working

Correct answer: b | LO: PTFL-4.3.1 (K2) | Syllabus §4.3

Why: (b) The system must have started up and achieved a steady state for valid steady state performance metrics to be gathered. (a) is not correct because **the goal of the ramp up is to get the number of users to the point desired for the test, not to exceed that number**. (c) is not correct because **ramp up is increasing load, not decreasing it getting ready for shut down**. (d) is not correct because **this has nothing to do with ramp up and should be tested before any official tests are started**.

Q19. OFFICIAL · Sample Exam #36

What is a method for testing transient states?

a) Steady load testing

b) Peak and valley testing

c) Spike testing ✔

d) Scalability testing

Correct answer: c | LO: PTFL-4.3.1 (K2) | Syllabus §4.3 (with §1.2)

Why: (c) This is achieved with spike tests which force the system to transition between a low load to a high load in a short period of time. The syllabus states that **it is sometimes important to test transient states, when the system's behavior is changing**, and gives **the concurrent logging of a large number of users** and **spike tests** as the two examples. It also warns that **some standard approaches — such as monitoring averages — may be very misleading** when testing transient states.

Q20. OFFICIAL · Sample Exam #37 · K4

You are working on a project that tracks health history information for patients across a region. The number of records handled by the system is in the millions due to the large number of patients in the region. Patient information must be accessible to doctors in offices, hospitals and urgent care facilities. The information should be presented to the requestor within three seconds of request, particularly for patients with critical allergies and preconditions. You have conducted your tests and have determined the following metrics: ***< 3 second response time: 85% of the time · < 5 second response time: 90% of the time · < 10 second response time: 95% of the time · < 60 second response time: 100% of the time.***

Given this information, how should you present the results to the stakeholders?

a) The test failed, the system is too slow. The requirements should be reviewed to ensure that <3 seconds is required 100% of the time.

b) 85% of the time the performance goal is met, so the system should be accepted based on industry standards.

c) The response time for 90% of the tests is probably acceptable, but more tuning is needed to bring down the 95% response time. ✔

d) The response time for 100% of the tests is unacceptable and tuning will be required to bring it down to <3 seconds.

Correct answer: c | LO: PTFL-4.4.1 (K4) | Syllabus §4.4

Why: The answer key gives (c). The official rationale is unusually discursive and worth reading in full: "***A is correct but this is also a***

problem with the requirements being unclear. *Does it have to be less than 3 seconds 100% of the time? Or is it acceptable if it's 95%?* ***As stated though, the system has failed the test and the requirements should be revisited. C might be a good result if the requirements had been better defined.*** *Right now, they say that 100% must be less than 3 seconds. Realistically though,* ***the 90% time should be investigated to see if there are a few outliers that are bringing it down or if it is always at the 5 second mark****. More tuning should be attempted though because this is a critical system and performance could be life or death impacting. B is not correct because the goal didn't state 85% of the time was acceptable. D is not correct because tuning to get to below 3 seconds 100% of the time will probably not be possible.*" **The exam-safe reading: report honestly against the stated objective, recommend further tuning, and never invent an acceptance level the requirements did not state.**

Part B — 50 additional syllabus-derived questions

Q21. Recall

According to the syllabus, what do stakeholders set with respect to performance testing?

a) The load generation approach and the tool licence model

b) The test environment configuration and the script structure

c) The objectives, the terminology to be used, and the criteria for determining whether the objective has been achieved ✔

d) The operational profiles and the load profiles

Correct answer: c | LO: PTFL-4.1.1 (K4) | Syllabus §4.1.1

Why: "***Stakeholders set the objectives, the terminology to be used and the criteria for determining whether the objective has been achieved.***" All three come from the stakeholders. Profiles, tooling and environment are the tester's work, informed by those objectives.

Q22. Classification

Which of the following is best described as a user-based performance objective?

a) Determining under what conditions degraded performance becomes apparent

b) Establishing the maximum number of virtual users the load generator can sustain

c) Confirming the system can scale to twice today's data volume

d) Ensuring a search returns within two seconds for a customer browsing the catalogue ✔

Correct answer: d | LO: PTFL-4.1.1 (K4) | Syllabus §4.1.1

Why: User-based objectives focus primarily on end-user satisfaction and business goals — "*users are **less concerned about feature types or how a product gets delivered. They just want to be able to do what they need to do.***" Options (a) and (c) are **technical objectives** (**conditions of degradation** and **ability to scale**); (b) is a property of the test harness, not an objective for the SUT.

Q23. Recall · list

Which of the following is one of the three questions the syllabus says should be answered when gathering information from stakeholders?

a) What performance improvements are expected from these tests compared to previous test cycles? ✔

b) Which protocols will the load generation tool support?

c) How many load generation machines will be required?

d) Which lifecycle model is the project using?

Correct answer: a | LO: PTFL-4.1.1 (K4) | Syllabus §4.1.1

Why: The three questions are: **what transactions will be executed in the performance test and what average response time is expected · what system metrics are to be captured (e.g., memory usage, network throughput) and what values are expected · what performance improvements are expected from these tests compared to previous test cycles**. The other options are implementation concerns from **§4.2.8** and **§3.4**.

Q24. Recall

The syllabus names three key objectives of performance testing. Which of the following is one of them?

a) Certifying the system as defect-free under load

b) Finding opportunities for improvement ✔

c) Replacing functional testing at the system level

d) Establishing the licence cost of the test tools

Correct answer: b | LO: PTFL-4.1.1 (K4) | Syllabus §4.1.1

Why: "***Key objectives of performance testing include identifying potential risks, finding opportunities for improvement, and identifying necessary changes.***" Note that none of the three is *proving the system correct* — consistent with **§2.4**, where **performance testing often lacks a well-defined test oracle**.

Q25. Recall

What does the syllabus say about the relationship between the Performance Test Plan (PTP) and the Test Plan?

a) The PTP replaces the Test Plan for performance projects

b) The PTP is written after performance testing has begun, using real measurements

c) The PTP is created prior to any performance testing occurring, should be referred to by the Test Plan (which also includes relevant scheduling information), and continues to be updated once performance testing begins ✔

d) The PTP is optional if acceptance criteria are already in the contract

Correct answer: c | LO: PTFL-4.1.2 (K4) | Syllabus §4.1.2

Why: All three clauses in (c) come straight from the syllabus. The PTP is **created prior to any performance testing occurring**, is **referred to by the Test Plan** which **also includes relevant scheduling information**, and **continues to be updated once performance testing begins**.

Q26. Tricky · concerns

In the Acceptance Criteria section of the PTP, how does the syllabus allocate response time, throughput and resource utilization to stakeholder concerns?

a) Response time is a system concern, throughput is a user concern, resource utilization is a business concern

b) Response time is a user concern, throughput is a business concern, resource utilization is a system concern ✔

c) All three are business concerns

d) All three are technical concerns

Correct answer: b | LO: PTFL-4.1.2 (K4) | Syllabus §4.1.2

Why: "***In general, response time is a user concern, throughput is a business concern, and resource utilization is a system concern.***" This single sentence connects **PTFL-4.1.1** (user-based versus technical objectives), **PTFL-2.1.1** (technical versus business metrics) and **PTFL-4.4.1** (what to report to whom).

Q27. Definition

How does the syllabus define a baseline in the context of acceptance criteria?

a) A set of metrics used to compare current and previously achieved performance measurements ✔

b) The first test executed after the environment is deployed, regardless of its results

c) The minimum load at which the system is stable

d) The service level agreed with the customer

Correct answer: a | LO: PTFL-4.1.2 (K4) | Syllabus §4.1.2

Why: "*A **baseline** is **a set of metrics used to compare current and previously achieved performance measurements**. This enables*

particular performance improvements to be demonstrated and/or the achievement of test acceptance criteria to be confirmed** **It may be necessary to first create the baseline using sanitized data from a database, where possible. " The SLA is listed separately as another thing acceptance criteria relate back to.

Q28. Tricky · GDPR

What does the syllabus say about data masking or anonymizing in performance testing?

a) It is optional and generally improves test realism

b) It is forbidden in performance testing because it changes the data

c) It is required only for functional testing

d) It is mandatory under GDPR for production data containing personally identifiable information — however, in performance testing it adds risk, as the masked data may not have the same data characteristics as seen in real-world use ✔

Correct answer: d | LO: PTFL-4.1.2 (K4) | Syllabus §4.1.2

Why: The syllabus states both halves: masking **is used on production data that contains personally identifiable information, and is mandatory under General Data Protection Regulations (GDPR)**; **however, in performance testing, data masking adds risk to the performance tests as it may not have the same data characteristics as seen in real-world use**. This connects to **§4.2.8**: **the closer the test data is to the production data in size and structure, the more reliable the test results will be**.

Q29. Recall · list

Which of the following belongs in the Risks section of the PTP?

a) The versions of the operating systems and databases in use

b) The number of virtual users each script will start

c) External interfaces that cannot be simulated, insufficient load, inability to monitor servers, and limitations of the test environment such as insufficient data or a scaled down environment ✔

d) The 90th percentile response time target

Correct answer: c | LO: PTFL-4.1.2 (K4) | Syllabus §4.1.2

Why: "*Risks can include **areas not measured as part of the performance testing** as well as **limitations to the performance testing** (e.g., **external interfaces that cannot be simulated, insufficient load, inability to monitor servers**). **Limitations of the test environment may also produce risks** (e.g., **insufficient data, scaled down environment**).*" Option (a) belongs to **System Configuration**, (b) to **Profiles**, and (d) to **Acceptance Criteria**.

Q30. Recall

What must the Test Environment section of the PTP include when the test environment is a smaller-scale mimic of production?

a) A commitment to repeat all tests in production before release

b) How the results from the performance testing will be extrapolated to apply to the larger production environment ✔

c) A waiver signed by the business stakeholders

d) The load generator licence count

Correct answer: b | LO: PTFL-4.1.2 (K4) | Syllabus §4.1.2

Why: "*The test environment is **often a separate environment that mimics production, but at a smaller scale**. This section of the PTP should include **how the results from the performance testing will be extrapolated to apply to the larger production environment***." The syllabus adds that **with some systems, the production environment becomes the only viable option for testing, but in this case the specific risks of this type of testing must be discussed**, and that **testing tools sometimes reside outside the test environment itself and may require special access rights**.

Q31. Recall

Which three kinds of test data does the PTP section on Test Data name?

a) User account data, user input data, and the pre-populated database ✔

b) Baseline data, masked data, and synthetic data

c) Operational profile data, load profile data, and scenario data

d) Log data, metric data, and report data

Correct answer: a | LO: PTFL-4.1.2 (K4) | Syllabus §4.1.2

Why: "*This data can include the following:* ***user account data*** *(e.g., user accounts available for simultaneous log in)* · ***user input data*** *(e.g., the data a user would enter into the application in order to perform a business process)* · ***database*** *(e.g., the pre-populated database that is populated with data for use in testing).*" The **creation process** separately covers **extraction from production, importing into the SUT, creation of new data, creation of backups for restoration between cycles, and masking or anonymizing**.

Q32. Audience

Which of the following does the syllabus say is generally of secondary interest to stakeholders with a business focus?

a) The connection between product risks and performance test objectives

b) Technical issues concerning tooling, scripting and load generation ✔

c) The balance between cost and how representative the results will be

d) Project risks such as infrastructure requirements and dependencies on key staff

Correct answer: b | LO: PTFL-4.1.3 (K4) | Syllabus §4.1.3

Why: "***Technical issues concerning tooling, scripting and load generation are generally of secondary interest***" to business-focused stakeholders, who are also **less interested in the distinctions between functional and non-functional quality characteristics**. Options (a), (c) and (d) are all explicitly listed as things that **must be communicated** to that audience.

Q33. Audience

Which factor does the syllabus list under communicating with technology-focused stakeholders?

a) A broad plan containing costs, time schedule and milestones

b) The balance between the cost of the tests and their representativeness

c) Where test environments are to be shared, the scheduling of performance tests must be communicated to ensure the test results will not be adversely impacted ✔

d) The connection between product risks and performance test objectives

Correct answer: c | LO: PTFL-4.1.3 (K4) | Syllabus §4.1.3

Why: Shared-environment **scheduling** is one of the six technology-focus factors, alongside **the planned approach to generating required load profiles and the expected involvement of technical stakeholders**, **detailed setup and execution steps showing the relation to architectural risks**, **steps required to make tests repeatable including organizational aspects**, **mitigations if testing must run in production**, and **clarity about their tasks and when they are scheduled**. Options (a), (b) and (d) are business-focus factors.

Q34. Tricky · incompleteness

In the official presentation questions, why are both the "test in production because a test system is prohibitive" option and the "here is the list of product risks" option wrong for either audience?

a) Because both are too technical for business stakeholders

b) Because neither mentions cost

c) Because production testing is never permitted and risk lists are never presented

d) Because each is incomplete — production testing must be accompanied by mitigation plans so any impact to production is understood, and a risk list must be accompanied by how the tests will help mitigate those risks ✔

Correct answer: d | LO: PTFL-4.1.3 (K4) | Syllabus §4.1.3

Why: This is the official rationale, given identically in both questions: option (c) **is incomplete. If testing will be conducted in production, mitigation plans must also be made so that any impact to production is understood.** Option (d) **is a list of risks but doesn't include a list of how the tests will help mitigate these risks.** Note that the syllabus does permit production testing — **PTFL-4.1.2** says **with some systems the production environment becomes the only viable option**, provided **the specific risks are discussed**.

Q35. Recall

What does the syllabus say must be communicated about repeatability, and to whom?

a) To business stakeholders: whether the test will be difficult to repeat or can be repeated with a minimum of effort. To technical stakeholders: the steps required to make performance tests repeatable, which may include organizational aspects as well as technical issues ✔

b) Only to technical stakeholders, and only the technical steps

c) Only to business stakeholders, and only the cost of repetition

d) Repeatability is not a communication topic; it is a property of the scripts

Correct answer: a | LO: PTFL-4.1.3 (K4) | Syllabus §4.1.3

Why: Both audiences get a repeatability message, but a different one. The business version asks **will the test be difficult to repeat or can it be repeated with a minimum of effort?** The technical version specifies **steps required to make performance tests repeatable**, which **may include organizational aspects (e.g., participation of key staff) as well as technical issues**. Repeatability also traces back to **PTFL-1.1.1**, where **reproducibility** is a named principle.

Q36. Recall · OSI

Which OSI model layers does the syllabus say are most commonly used for performance testing?

a) Layer 1 (Physical) to Layer 3 (Network)

b) Layer 5 (Session) to Layer 7 (Application) ✔

c) Layer 2 (Data Link) to Layer 4 (Transport)

d) Only Layer 7 (Application)

Correct answer: b | LO: PTFL-4.2.1 (K2) | Syllabus §4.2.1

Why: "*For performance testing*, ***protocols from Layer 5 (Session Layer) to Layer 7 (Application Layer) are most commonly used for performance testing***." The syllabus adds that **the level of the OSI layer which is most in focus relates to the level of the architecture being tested** — when testing **low-level, embedded architecture**, **the lower numbered layers of the OSI model will be mostly in focus**. The OSI model reference is **ISO/IEC 7498-1**.

Q37. Protocol group

Under which group does the syllabus list LDAP, POP3, Windows Sockets and CORBA?

a) Web Service

b) Remote Access

c) Network ✔

d) SOA

Correct answer: c | LO: PTFL-4.2.1 (K2) | Syllabus §4.2.1

Why: The **Network** group is **DNS · FTP · IMAP · LDAP · POP3 · SMTP · Windows Sockets · CORBA**. **Remote Access** is **Citrix ICA · RTE**; **SOA** is **MQSeries · JSON · WSCL**; **Web Service** is **SOAP · REST**. Learning the group labels is what makes the two official protocol questions automatic.

Q38. Protocol group

Which protocols does the syllabus list under Mobile?

a) TruClient, SMP, MMS ✔

b) Citrix ICA, RTE

c) SOAP, REST

d) ODBC, JDBC

Correct answer: a | LO: PTFL-4.2.1 (K2) | Syllabus §4.2.1

Why: Mobile = TruClient · SMP · MMS. The remaining options are **Remote Access**, **Web Service** and **Database** respectively. Note that the syllabus's protocol catalogue is explicitly a list of **typical protocols encountered in performance testing** — the learning objective is to **give examples**, so recognition is enough.

Q39. Application

The team wants to perform end-to-end testing that emulates user activity from the browser, focusing on the communication and activities of the backend servers rather than the GUI itself.

Which protocol family will be employed, and what is the effect?

a) ODBC, so the database is exercised directly

b) SMTP, so message queues are exercised

c) Citrix ICA, so the remote desktop layer is exercised

d) A web protocol such as HTTP/HTTPS, so that interaction with the GUI can be bypassed and the tests can focus on the communication and activities of the backend servers ✔

Correct answer: d | LO: PTFL-4.2.1 (K2) | Syllabus §4.2.1

Why: "*Therefore, if the need is to* ***perform end-to-end testing emulating user activity from the browser***, ***a web protocol such as HTTP/HTTPS will be employed***. *In this way,* ***interaction with the GUI can be bypassed and the tests can focus on the communication and activities of the backend servers***." The syllabus contrasts this with **traditional 2-tier client-server applications**, which **require the use of a protocol such as ODBC to access the database**.

Q40. Definition

How does the syllabus define a transaction?

a) A single HTTP request and its response

b) The set of activities performed by a system from the point of initiation to when one or more processes (requests, operations, or operational processes) have been completed ✔

c) Any database write that can be rolled back

d) The interval between two synchronization points in a script

Correct answer: b | LO: PTFL-4.2.2 (K2) | Syllabus §4.2.2

Why: The definition in (b) is the syllabus's own. It adds that **the response time of transactions can be measured for the purpose of evaluating system performance**, and that during a performance test these measurements are used to **identify any components that require correction or optimization**.

Q41. Formula

What equals the elapsed time for a simulated transaction?

a) The transaction response time alone

b) The think time alone

c) The transaction response time plus the think time ✔

d) The transaction response time minus the network latency

Correct answer: c | LO: PTFL-4.2.2 (K2) | Syllabus §4.2.2

Why: "***The transaction response time plus the think time equals the elapsed time for that transaction.***" The same sum reappears as the denominator of the throughput formula in **§4.2.5**: **system throughput = [number of virtual users] / ([processing time] + [think time])**.

Q42. Application

A tester wants to know the response time of each step in an online ordering system — search for item, add item to cart, pay for item, confirm order — and also the response time of the order process as a whole.

Which technique does the syllabus recommend, and what does it deliver?

a) Nesting transactions, so that both individual and aggregate activities can be measured in one test ✔

b) Running four separate tests, one per step, then adding the results

c) Setting a synchronization point between each step

d) Removing think time so the steps run consecutively

Correct answer: a | LO: PTFL-4.2.2 (K2) | Syllabus §4.2.2

Why: "*Transactions* ***can also be nested so that individual and aggregate activities can be measured***." The syllabus uses precisely this ordering example and concludes: **by nesting transactions, both sets of information can be gathered in one test**. Synchronization points serve a different purpose — **ensuring a specific action is invoked concurrently or coordinating work between several scripts** (**§4.2.6**).

Q43. Tricky

What do transaction response times collected under increasing load allow a tester to do?

a) Guarantee that no component will degrade

b) Replace resource monitoring entirely

c) Prove the system meets its SLA at any load

d) Correlate the cause of degradation with the response times of one or more transactions, by ramping up load and measuring the underlying transaction times ✔

Correct answer: d | LO: PTFL-4.2.2 (K2) | Syllabus §4.2.2

Why: "*The transaction response times collected during the performance test* ***show how this measurement changes under different loads*** *imposed on the system.* ***Analysis may show no degradation under load while other measurements may show severe degradation. By ramping up load and measuring the underlying transaction times, it is possible to correlate the cause of degradation with the response times of one or more transactions.***" Correlation between metrics recurs in **§4.4** — e.g., **transactions per second when CPU reached 90% capacity**.

Q44. Definition

How does the syllabus define an operational profile?

a) The number of virtual users applied to the system over time

b) Distinct patterns of interaction with an application, such as from users or other system components, which may be combined to create a desired load profile ✔

c) The list of protocols used by the system under test

d) The set of metrics collected during a test run

Correct answer: b | LO: PTFL-4.2.3 (K4) | Syllabus §4.2.3

Why: "*Operational profiles* ***specify distinct patterns of interaction with an application such as from users or other system components. Multiple operational profiles may be specified for a given application. They may be combined to create a desired load profile*** *for achieving particular performance test objectives.*" The PTP section adds that an operational profile provides **a repeatable step-by-step flow through the application for a particular usage of the system**. Option (a) describes the **load profile**.

Q45. Recall · steps

What are the three principal steps for identifying operational profiles?

a) Record, parameterize, correlate

b) Ramp up, hold steady, ramp down

c) Identify the data to be gathered; gather the data using one or more sources; evaluate the data to construct the operational profiles ✔

d) Plan, design, implement

Correct answer: c | LO: PTFL-4.2.3 (K4) | Syllabus §4.2.3

Why: The three steps are exactly as stated in (c). Option (a) is the **script implementation** sequence from **§4.2.7**; option (b) is the **load profile / execution** vocabulary from **§4.2.4** and **§4.3**.

Q46. Recall · data

At what level of abstraction does the syllabus say generic user tasks are generally best modelled?

a) At a high level of abstraction, e.g., at the level of business processes or major user stories ✔

b) At the level of individual HTTP requests

c) At the level of database transactions

d) At the level of UI widget interactions

Correct answer: a | LO: PTFL-4.2.3 (K4) | Syllabus §4.2.3

Why: "*Note that* ***these tasks are generally best modeled at a high level of abstraction*** *(e.g.,* ***at the level of business processes or major user stories****).*" This pairs with the construction rule that **a top-down approach is taken — relatively simple broad operational profiles are initially created and only broken down further if this is needed to achieve performance test objectives**

Q47. Recall · sources

Which source does the syllabus recommend coordinating with planned usability tests, especially if a usability lab is available?

a) Interviews or workshops with stakeholders

b) Functional specifications and requirements

c) Usage data and metrics from similar applications

d) Monitoring the behavior of users when performing predefined tasks with the application ✔

Correct answer: d | LO: PTFL-4.2.3 (K4) | Syllabus §4.2.3

Why: "***Monitoring the behavior of users when performing predefined tasks*** *with the application may give insights into the types of operational profiles to be modeled for performance testing.* ***Coordinating this task with any planned usability tests (especially if a usability lab is available) is recommended.***" This echoes Chapter 1's note that performance testing is **often closely integrated with usability engineering**.

Q48. Recall · construction

Which characteristics cause a particular user profile to be singled out as relevant for performance testing?

a) It uses the greatest number of protocols

b) It involves tasks executed frequently, requires critical (high risk) or frequent transactions between different system components, or potentially demands large volumes of data to be transferred ✔

c) It belongs to the administrator role

d) It can be recorded at the protocol level without correlation

Correct answer: b | LO: PTFL-4.2.3 (K4) | Syllabus §4.2.3

Why: These three criteria are the syllabus's own. The construction rules also require that **operational profiles are reviewed and refined with the principal stakeholders before being used for the creation of load profiles**, and that a **top-down approach** is taken.

Q49. Non-user profiles

For off-line batch processing systems, where does the syllabus say the operational profile focus lies?

a) On the number of concurrent interactive users

b) On the GUI response time of the batch monitoring console

c) Principally on the throughput of the batch processing system and its ability to complete within a given time period, with profiles focusing on the types of processing demanded of the batch processes ✔

d) On the correlation of session identifiers between batch jobs

Correct answer: c | LO: PTFL-4.2.3 (K4) | Syllabus §4.2.3

Why: The syllabus example is a **stock trading system** whose profiles may include **payment transactions, verifying credentials, and checking compliance of legal conditions for particular types of stock transactions — each resulting in different paths being taken through the overall batch process**. It also notes that **the steps outlined for online user-based systems can also be applied in the batch processing context**.

Q50. Non-user profiles

What does the syllabus say may be required when building operational profiles for a system of systems?

a) Modeling several different operational profiles to represent the types of input provided by supplier systems, possibly involving detailed analysis of buffers and queues together with the system architects and based on system and interface specifications ✔

b) A single profile representing the slowest supplier system

c) Crowd testing to represent the supplier systems

d) Ignoring supplier systems, since they are outside the SUT

Correct answer: a | LO: PTFL-4.2.3 (K4) | Syllabus §4.2.3

Why: "*Components within a **multi-system (which may also be embedded) environment** respond to **different types of input from other systems or components**. Depending on the nature of the system under test, this may require **modeling of several different operational profiles** to effectively represent the types of input provided by those **supplier systems. This may involve detailed analysis (e.g., of buffers and queues) together with the system architects and based on system and interface specifications.***"

Q51. Definition

How does the syllabus define a load profile?

a) The pattern of interaction of a single user with the application

b) The set of protocols used to generate load

c) The list of metrics collected during the test

d) The activity a component or system being tested may experience in production, consisting of a designated number of instances that will perform the actions of predefined operational profiles over a specified time period ✔

Correct answer: d | LO: PTFL-4.2.4 (K4) | Syllabus §4.2.4

Why: That is the syllabus definition, and it adds: **where the instances are users, the term "virtual users" is commonly applied**. The PTP section calls the aggregated result a **scenario**. Option (a) describes an **operational profile**.

Q52. Recall · inputs

Which of the following is one of the four pieces of information required to create a realistic and repeatable load profile?

a) The licence model of the load testing tool

b) Known throughput and concurrency issues ✔

c) The programming language of the SUT

d) The number of load generation machines available

Correct answer: b | LO: PTFL-4.2.4 (K4) | Syllabus §4.2.4

Why: The four inputs are: **the performance testing objective** (e.g., to evaluate system behavior under stress loads) · **operational profiles which accurately represent individual usage patterns** · **known throughput and concurrency issues** · **the quantity and time distribution with which the operational profiles are to be executed** so the SUT experiences the desired load.

Q53. Recall · distributions

Which time distribution does the syllabus describe as "instantaneous changes in load", and what example does it give?

a) Ramp-ups; add one virtual user per minute

b) Ramp-downs; steadily decreasing load

c) Steps; add 100 virtual users every five minutes ✔

d) Predefined distributions; volume mimics daily or seasonal business cycles

Correct answer: c | LO: PTFL-4.2.4 (K4) | Syllabus §4.2.4

Why: The four distributions are **ramp-ups** (*steadily increasing load, e.g., add one virtual user per minute*) · **ramp-downs** (*steadily decreasing load*) · **steps** (*instantaneous changes in load, e.g., add 100 virtual users every five minutes*) · **predefined distributions** (*volume mimics daily or seasonal business cycles*).

Q54. Worked example

In the syllabus's example of constructing a stress load profile, what does the top diagram represent?

a) A step input of 100 virtual users performing Operational Profile 1 over the entire duration of the test — typical of load profiles representing a background load ✔

b) A ramp-up to 220 virtual users maintained for two hours

c) The three-hour combined period of stress

d) A ramp-down to zero users after the peak

Correct answer: a | LO: PTFL-4.2.4 (K4) | Syllabus §4.2.4

Why: The top diagram is **a step input of 100 virtual users** performing **Operational Profile 1 over the entire duration of the test** — **typical of many performance load profiles that represent a background load**. The middle diagram is **a ramp-up to 220 virtual users maintained for two hours before ramping down**, performing **Operational Profile 2**, and the combination subjects the SUT to **a three-hour period of stress**.

Q55. Calculation

A call centre has 300 agents on the early shift and 200 on the late shift. Each agent opens the case system 8 times per 8-hour shift, and each opening produces 5 case updates. The shifts do not overlap.

How many case-update transactions does the system process per day from these agents?

a) 4,000

b) 20,000 ✔

c) 2,500

d) 40,000

Correct answer: b | LO: PTFL-4.2.4 (K4) | Syllabus §4.2.4

Why: Apply the official method: **total agents × accesses per shift × transactions per access**. **(300 + 200) × 8 × 5 = 500 × 40 = 20,000.** The same three-factor multiplication produces the **150,000** figure in the official question (**2500 × 10 × 6**). Always compute the daily total first — it is the check that tells you whether a proposed load profile's bands add up.

Q56. Tricky · reading options

In the official load-profile question, the winning option reads "7 hours at 21,000 transactions, 3 hours at 27,000 transactions, 3 hours at 36,000 transactions, 11 hours at 66,000 transactions". How should those figures be interpreted?

a) As hourly rates during each band

b) As the number of virtual users in each band

c) As band totals — the total transactions across all the hours in that band, summing to the 150,000 daily total ✔

d) As percentile response times per band

Correct answer: c | LO: PTFL-4.2.4 (K4) | Syllabus §4.2.4

Why: They are **band totals**. Check: **7 h × 500 doctors × 6 transactions = 21,000 · 11 h × 1000 × 6 = 66,000 · 3 h × 1500 × 6 = 27,000 · 3 h × 2000 × 6 = 36,000**, and **21,000 + 66,000 + 27,000 + 36,000 = 150,000** — the daily total. Reading them as hourly rates makes the arithmetic impossible, which is why this is the most misread question on the paper.

Q57. Definition

How does the syllabus distinguish system throughput from network throughput?

a) System throughput is the number of transactions of a given type the system processes in a unit of time; network throughput is the amount of data moved over the network ✔

b) System throughput is measured in bits per second; network throughput is measured in transactions per second

c) They are two names for the same measure

d) System throughput applies only to batch systems; network throughput only to interactive systems

Correct answer: a | LO: PTFL-4.2.5 (K4) | Syllabus §4.2.5

Why: "***System throughput*** *is a measure of the* ***number of transactions of a given type that the system processes in a unit of time*** *— for example,* ***the number of orders per hour or the number of HTTP requests per second. System throughput should be distinguished from network throughput, which is the amount of data moved over the network.*** **" System throughput defines load on the system.** Network throughput is measured in **bits per second** (§4.4).

Q58. Formula

Which formula does the syllabus give for system throughput?

a) [number of virtual users] × [think time]

b) [transactions completed] / [number of virtual users]

c) [number of virtual users] / ([processing time] + [think time]) ✔

d) [processing time] / [number of virtual users]

Correct answer: c | LO: PTFL-4.2.5 (K4) | Syllabus §4.2.5

Why: "***System throughput = [number of virtual users] / ([processing time] + [think time])***". The syllabus draws the consequence explicitly: **so when the processing time increases, throughput may significantly decrease even if everything else stays the same.** Note the denominator is the **elapsed time** defined in §4.2.2.

Q59. Tricky · 60x

Why does the syllabus say the number of users is not a good measure of load?

a) Because load testing tools cannot count users accurately

b) Because without defining operational profiles — what each user is doing and how intensely — the same 500 users can produce a 60x difference in load, and at least a 60x difference in hardware requirements ✔

c) Because concurrency is always more important than throughput

d) Because users are never concurrent in practice

Correct answer: b | LO: PTFL-4.2.5 (K4) | Syllabus §4.2.5

Why: The syllabus's own arithmetic: **500 users running short queries each minute give a throughput of 30,000 queries per hour; the same 500 users running one query per hour give 500 queries per hour — the same 500 users, but a 60x difference between loads and at least a 60x difference in the hardware requirements for the system**. It notes that user counts are used anyway because **that number is often easier to find** and because **it is the way load testing tools define load**.

Q60. Definition

How does the syllabus contrast closed and open systems?

a) Closed systems have no external network access; open systems are internet-facing

b) Closed systems use protocol-level scripts; open systems use GUI scripts

c) Closed systems are batch; open systems are interactive

d) Closed systems have a fixed population, so if all users are waiting for the system's response no new users can arrive; in open systems new users arrive all the time even if all current users are waiting ✔

Correct answer: d | LO: PTFL-4.2.5 (K4) | Syllabus §4.2.5

Why: The syllabus frames this from **the queuing theory point of view**: **typical test setups are closed systems, where the number of users in the system is set (fixed population)**, whereas **many public systems are open systems — new users are arriving all the time even if all the current users are waiting for the system's response**. This is why a closed-model test can understate the load a public system actually experiences.

Q61. Tricky · concurrency

Why does the syllabus say concurrency matters even when throughput is unchanged?

a) Because concurrency represents the number of parallel sessions, each of which may use its own resources — so the amount of resources used may differ depending on concurrency ✔

b) Because concurrency determines the protocol used

c) Because concurrency is the only measure load testing tools support

d) Because higher concurrency always means higher throughput

Correct answer: a | LO: PTFL-4.2.5 (K4) | Syllabus §4.2.5

Why: "***Concurrency is an important measure. It represents the number of parallel sessions, each of which may use its own resources. Even if throughput is the same, the amount of resources used may differ depending on concurrency.***" This is why the syllabus insists that **to model operational and load profiles properly, both aspects should be taken into consideration**.

Q62. Recall · structure

What is the overall structure of a performance test script, according to the syllabus?

a) Setup, teardown, and reporting

b) An initialization section, main sections that may be executed multiple times, and a clean-up section ✔

c) Ramp-up, steady state, and ramp-down

d) Record, parameterize, and correlate

Correct answer: b | LO: PTFL-4.2.6 (K2) | Syllabus §4.2.6

Why: "*Often the script has **an initialization section (where everything gets prepared for the main part), main sections that may be executed multiple times, and a clean-up section (where necessary steps are taken to finish the test properly)***." Option (c) is the shape of a **test run** (§4.3), not of a script; option (d) is the **implementation** sequence (§4.2.7).

Q63. Tricky · measurement

What is the difference in what protocol-level and GUI-level scripts actually measure?

a) Protocol-level scripts measure end-to-end time; GUI scripts measure server time only

b) Both measure end-to-end time

c) In the case of protocol-level scripts it is server and network response time only, while GUI scripts measure end-to-end time — although what exactly is measured depends on the technology used ✔

d) Neither measures response time; both measure throughput only

Correct answer: c | LO: PTFL-4.2.6 (K2) | Syllabus §4.2.6

Why: That is the syllabus wording. It is also why **§4.3** recommends cross-checking a protocol-level load test by **running several GUI-level (functional) scripts or executing similar operational profiles manually in parallel — response times reported during the test should only differ from those measured manually at the GUI level by the time spent on the client side**.

Q64. Recall

What are synchronization points used for in a performance test script?

a) To make the script wait for an event before proceeding, ensuring a specific action is invoked concurrently or coordinating work between several scripts ✔

b) To align the clocks of the monitoring tools

c) To reset the database between iterations

d) To insert think time between user actions

Correct answer: a | LO: PTFL-4.2.6 (K2) | Syllabus §4.2.6

Why: "*One example is **setting synchronization points**, which is done by **specifying that the script should wait for an event at that point before proceeding**. The synchronization points **may be used to ensure that a specific action is invoked concurrently or to coordinate work between several scripts***." Option (b) describes **time synchronization of tools**, a separate requirement in **§4.3**.

Q65. Tricky · timers

What should the timed requests in a performance test script match?

a) Each individual HTTP request, for maximum granularity

b) The full duration of the test run

c) The think time between actions

d) A meaningful unit of logical work — for example, a business transaction for adding an item to an order or submitting an order ✔

Correct answer: d | LO: PTFL-4.2.6 (K2) | Syllabus §4.2.6

Why: "*To collect response times, **timers should be added to the script to measure how long a request or a combination of requests takes**. **The timed requests should match a meaningful unit of logical work** — for example, **a business transaction for adding an item to an order or submitting an order***." This is the same idea as **transactions** in §4.2.2, and it is what makes **nesting** worthwhile.

Q66. Recall

What does the syllabus conclude about the nature of performance testing scripts?

a) They are configuration, so they need no testing of their own

b) They are software, so creating one is a software development activity that should include quality assurance and tests to verify the script works as expected with the whole range of input data ✔

c) They are documentation, so they belong in the PTP

d) They are disposable, so maintainability is not a concern

Correct answer: b | LO: PTFL-4.2.6 (K2) | Syllabus §4.2.6

Why: "***Performance testing scripts are software, so creating a performance testing script is a software development activity. It should include quality assurance and tests to verify that the script works as expected with the whole range of input data.***" This is also why **default error handling is minimal** and additional checks must be added, and why the **maintainability** of a parameterized script matters (§4.2.7).

Q67. Definition · K3

What is correlation in the context of performance test script implementation?

a) The statistical relationship between two or more measured metrics

b) Matching the think time to the operational profile

c) Replacing recorded internal identifiers that define context with variables that can be changed between runs, using values extracted from the request responses ✔

d) Combining several scripts into one scenario

Correct answer: c | LO: PTFL-4.2.7 (K3) | Syllabus §4.2.7

Why: That is the syllabus definition, with the login-identifier example: **a user identifier that is acquired at login and must be supplied for all subsequent transactions**. The syllabus explicitly warns that **in that context the word correlation has a different meaning than when used in statistics (where it means relationship between two or more things)** — option (a) is the statistical meaning and is a deliberate trap.

Q68. Tricky · K3

What does the syllabus identify as the main reason why recorded scripts fail to play back?

a) Incorrect correlation or lack of correlation ✔

b) Insufficient think time

c) Too many virtual users

d) Use of the wrong protocol group

Correct answer: a | LO: PTFL-4.2.7 (K3) | Syllabus §4.2.7

Why: "***Incorrect correlation or lack of correlation is the main reason why recorded scripts fail to playback.***" The syllabus notes that **advanced load testing tools may do some correlation automatically, so it may be transparent in some cases — but in more complex cases, manual correlation or adding new correlation rules may be required**. This sentence is the direct source of official question #31.

Q69. Tricky · two directions

Running many virtual users with the same user name and the same data can distort results in two opposite directions. What are they?

a) Both directions make results worse than production

b) Both directions make results better than production

c) Caching makes results worse; locking makes them better

d) Caching can make results much better than production because data is copied from disk to memory; concurrency issues such as data locking can make them much worse because the software waits for the lock to free ✔

Correct answer: d | LO: PTFL-4.2.7 (K3) | Syllabus §4.2.7

Why: The syllabus gives both mechanisms explicitly. **The data could be completely cached (copied from disk to memory for faster access) and results would be much better than in production (where such data may be read from a disk).** And: **using the same users and/or data can also cause concurrency issues (e.g., if data is locked when a user is updating it) and results would be much worse than in production as the software would wait for the lock to free before the next user could lock the data for update.**

Q70. Application · K3

A recorded script creates a purchase order. It runs correctly the first time, but every subsequent run fails at the point where the order is saved.

What is the most likely cause according to the syllabus, and what is the fix?

a) The think time is too short; increase it to match the operational profile

b) A value that must be unique — such as the order name — was not parameterized, so the test fails as soon as it tries to create an order with an existing recorded name; parameterize it ✔

c) The load generator is overloaded; add a second generation machine

d) The protocol level is too low; re-record at the GUI level

Correct answer: b | LO: PTFL-4.2.7 (K3) | Syllabus §4.2.7

Why: "*There are cases where **some data must be parameterized for the test to work more than once** — for example, **when an order is created and the order name must be unique**. **Unless the order's name is parameterized, the test will fail as soon as it tries to create an order with an existing (recorded) name.***" This is distinct from correlation: correlation carries **context identifiers extracted from responses**; parameterization supplies **values from a list of possible choices** so that **each virtual user uses a proper set of data**.

Most Important Questions From This Chapter

*Ranking rationale: Chapter 4 is **half the paper**. Seven objectives carry **two marks each** (14 of the 20) and are mostly **K2 recall**; the remaining six marks come one each from the analytical objectives. The ranking below is by **marks available per unit of study effort**.*

Rank	Topic / question focus	LO	Marks	Why it matters
1	Protocol group matching — Q5, Q6, Q36–Q39	4.2.1	**2**	Pure recall from a seven-row table. The cheapest two marks in the chapter.
2	Think time and nested transactions — Q7, Q8, Q40–Q43	4.2.2	**2**	Two definitions and one equation: **response time + think time = elapsed time**.
3	Ramp-up purpose and transient states — Q18, Q19, and the execution items	4.3.1	**2**	Both answers are single sentences, and one of them reaches back to Chapter 1's **spike testing**.
4	Business versus technical audience — Q3, Q4, Q32–Q35	4.1.3	**2**	The **same four options are used twice**. Learn which one belongs to which audience and both marks follow.
5	Correlation and caching — Q14, Q15, Q67–Q70	4.2.7	**2 (K3)**	Two named failure modes with two named fixes. The only K3 objective in the syllabus's own numbering.
6	Protocol-level scalability and database verification — Q12, Q13, Q62–Q66	4.2.6	**2**	Script structure, what each script type measures, and why you verify via the database.
7	Non-production environments and load generator monitoring — Q16, Q17	4.2.8	**2**	**Performance is a non-linear function of the environment**, and **load generators must be monitored too**.
8	The K4 arithmetic: operational profile → load profile → concurrency — Q9, Q10, Q11, Q44–Q61	4.2.3–4.2.5	**3**	Three marks from one worked scenario. Learn the multiplication and the **band totals** reading.
9	Objectives and the PTP — Q1, Q2, Q21–Q31	4.1.1, 4.1.2	**2**	**User-based versus technical**, and the **twelve PTP sections**.
10	Reporting percentile results — Q20	4.4.1	**1**	The **90th percentile is the usual goal**; **100% compliance is not recommended**; report honestly and recommend tuning.

Section 5 — Exam Tricks

5.1 Frequently tested topics (evidence-based)

Chapter 4 is **half the exam**. The Exam Structure Table allocates **20 questions across 13 learning objectives — seven objectives get two questions each** and **six get one**. There is **no selectable group**: every objective appears on every paper.

LO	K	Qs	Sample Exam #	Format used officially
4.1.1	K4	1	#18	Scenario — "which is a **technical objective**?"
4.1.2	K4	1	#19	Scenario — "which **information** will you need for the most critical objective?"
4.1.3	K4/K3	**2**	#20, #21	**Identical four options, asked twice** — once for business, once for technical stakeholders
4.2.1	K2	**2**	#22, #23	"Which **protocol** is being used?" — same four options both times
4.2.2	K2	**2**	#24, #25	**Think time**; **the value of nesting transactions**
4.2.3	K4	1	#26	Scenario — "what is **missing** to construct the operational profile?"
4.2.4	K4	1	#27	Scenario — "which is the **proper load profile**?" (calculation)
4.2.5	K4	1	#28	Scenario — "what is the **highest number of concurrent accesses**?" (calculation)
4.2.6	K2	**2**	#29, #30	**Advantage of protocol-level testing**; **best method to verify a script's effect**
4.2.7	**K3**	**2**	#31, #32	Scenario — **login works then fails** (correlation); **repeat query gets faster** (caching)
4.2.8	K2	**2**	#33, #34	**Non-equivalent environment**; **load generator concern**
4.3.1	K2	**2**	#35, #36	**Purpose of ramp-up**; **method for testing transient states**
4.4.1	K4	1	#37	Scenario — **how to present percentile results**
Chapter 4 total	—	**20**	**#18–#37**	**50% of the paper · all four K3 questions · six of the nine K4 questions**

> **The arithmetic before you sit the exam**
>
> CT-PT is **40 questions worth 40 points in 90 minutes** (**113 minutes** with the 25% non-native-language allowance); the pass mark is **65% = 26 points**. **A perfect score on Chapters 1, 2, 3 and 5 is only 20 marks — you cannot pass without Chapter 4.** At the other extreme, **Chapter 4 alone is 20 marks**, so a strong Chapter 4 plus Chapters 1 and 2 (**10 marks**) already reaches **30**. Budget your time accordingly: Chapter 4 deserves roughly **half your total study effort**.

5.2 Common distractors observed

- **A user-based objective offered as a technical one.** *Three seconds for 100 concurrent users* is **user-based**; *scale to 10 million records* is **technical**.
- **A robustness requirement offered as a performance objective.** *Response time unchanged on the disaster recovery system with no discernible downtime* is **primarily a robustness test rather than a performance test**.
- **Security information offered as performance-planning information.** *How is user access authenticated and authorized* is **more related to security than performance testing**.
- **Storage capacity offered as planning information.** *Where will the data be stored and how much storage is available* **may be something to investigate if retrieval proves slow, but is not needed to conduct the tests**.
- **The technical presentation offered to business stakeholders and vice versa.** The two official questions use an **identical option set**; the only discriminator is the audience.
- **Production testing without mitigation.** Any option proposing production testing **must also include mitigation plans so any impact to production is understood**, or it is **incomplete**.

- **A risk list without mitigations.** A presentation of product risks is incomplete **without a list of how the tests will help mitigate these risks**.
- **REST offered where the answer is HTTP.** For *the speed of the response of a web service*, the official answer is **HTTP** (a **web** protocol).
- **"Wait time", "latency time" or "reading time" offered for think time.** Only **think time** is the syllabus term.
- **Nesting described as "parent and child transactions".** The official rationale: **parent and child transactions may not accomplish anything meaningful in terms of system usage**.
- **Nesting described as speeding reporting.** It **may increase reporting time**.
- **Interviews offered as the missing operational-profile input** when the scenario already supplies the behaviour. What is missing is **the number of users for the role**.
- **Batch processing or additional components offered as missing from an operational profile.** They are **considerations for setting up the load profile but not needed for the operational profile**.
- **12,000 offered for peak concurrency** where the answer is **2,000 — concurrency counts sessions, not the transactions inside them**.
- **Load-profile options whose bands do not sum to the daily total.** Always add them up.
- **Protocol-level testing described as best for user experience. The UI is bypassed**, so it is not.
- **Protocol-level testing described as best for correlation. Data correlation is more difficult because the correlation done at the UI is bypassed.**
- **"No errors occurred" offered as verification. No errors doesn't mean that it worked — it could just mean that the error handling is also broken.**
- **Verification through the application offered instead of the database. The application may have a defect that causes it to not find the new users.**
- **Caching described as the system "getting faster". Systems tend to get slower, not faster, unless there is a caching issue.**
- **"Tests run more quickly in a smaller environment" — it could happen, but not necessarily**; the real consequence is **less reliable projections and increased risk**.
- **Load generator concerns about invalid data or production impact.** These **will not occur if the tool is configured properly**; the real concern is that **the generator may slow down and fail to maintain a steady load**.
- **Ramp-up described as exceeding the target load, as shutdown, or as a monitoring-tool check.** Its purpose is **to allow the system to achieve a steady state before taking measurements**.
- **"Peak and valley testing" offered for transient states.** The syllabus term is **spike testing**.
- **Reporting options that invent an acceptance level. The goal didn't state 85% of the time was acceptable**, and **tuning to below 3 seconds 100% of the time will probably not be possible**.

5.3 Keyword recognition (map words → answers)

Keyword in the stem	Points toward
'ability to scale', 'conditions under which degraded performance becomes apparent', 'operational aspects'	**Technical objective** (4.1.1)
'end-user satisfaction', 'business goals', 'do what they need to do'	**User-based objective** (4.1.1)
'identifying potential risks', 'opportunities for improvement', 'necessary changes'	**Key objectives of performance testing** (4.1.1)
'created prior to any performance testing', 'referred to by the Test Plan', 'continues to be updated'	**The PTP** (4.1.2)
'response time is a user concern, throughput a business concern, resource utilization a system concern'	**Acceptance criteria** in the PTP (4.1.2)
'a set of metrics used to compare current and previously achieved	**Baseline** (4.1.2)

Keyword in the stem	Points toward
performance measurements'	
'mandatory under GDPR', 'may not have the same data characteristics'	**Data masking** (4.1.2)
'external interfaces that cannot be simulated', 'insufficient load', 'scaled down environment'	**Risks section of the PTP** (4.1.2)
'cost', 'representative of the production system', 'milestones', 'product risks and objectives'	**Business-focus communication** (4.1.3)
'ramps up users at a rate of', 'maintained for 2 hours', 'architectural risks', 'shared environment scheduling'	**Technology-focus communication** (4.1.3)
'ODBC', 'JDBC'	**Database** protocol (4.2.1)
'HTTP', 'HTTPS', 'HTML'	**Web** protocol (4.2.1)
'SOAP', 'REST'	**Web Service** protocol (4.2.1)
'DNS', 'FTP', 'IMAP', 'LDAP', 'POP3', 'SMTP', 'Windows Sockets', 'CORBA'	**Network** protocol (4.2.1)
'TruClient', 'SMP', 'MMS'	**Mobile** protocol (4.2.1)
'Citrix ICA', 'RTE'	**Remote Access** protocol (4.2.1)
'MQSeries', 'JSON', 'WSCL'	**SOA** protocol (4.2.1)
'Layer 5 to Layer 7', 'ISO/IEC 7498-1'	**OSI framing for performance testing** (4.2.1)
'read a prompt and enter data', 'pressing the SEND button', 'delays between user actions'	**Think time** (4.2.2)
'measure the discrete steps as well as the process as a whole'	**Nested transactions** (4.2.2)
'personas and roles', 'generic tasks', 'estimated numbers of users per unit of time'	**Operational profile inputs** (4.2.3)
'top-down approach', 'reviewed and refined with the principal stakeholders'	**Constructing operational profiles** (4.2.3)
'nightly batch processing window', 'buffers and queues', 'supplier systems'	**Non-user operational profiles** (4.2.3)
'designated number of instances', 'over a specified time period', 'virtual users', 'scenario'	**Load profile** (4.2.4)
'add one virtual user per minute'	**Ramp-up** distribution (4.2.4)
'add 100 virtual users every five minutes'	**Steps** distribution (4.2.4)
'volume mimics daily or seasonal business cycles'	**Predefined distribution** (4.2.4)
'orders per hour', 'HTTP requests per second', 'defines load on the system'	**System throughput** (4.2.5)
'amount of data moved over the network'	**Network throughput** (4.2.5)
'simultaneous / parallel threads of execution', 'parallel sessions'	**Concurrency** (4.2.5)
'fixed population', 'no new users can arrive'	**Closed system** (4.2.5)
'new users arriving all the time'	**Open system** (4.2.5)
'initialization, main, clean-up'	**Script structure** (4.2.6)

Keyword in the stem	Points toward
'server and network response time only'	**Protocol-level script** (4.2.6)
'end-to-end time'	**GUI-level script** (4.2.6)
'wait for an event before proceeding', 'coordinate work between several scripts'	**Synchronization point** (4.2.6)
'checking the database to verify that the proper information was added'	**Indirect verification** (4.2.6)
'recorded internal identifiers that define context', 'acquired at login and supplied for subsequent transactions'	**Correlation** (4.2.7)
'copied from disk to memory', 'much better than in production'	**Caching hazard** (4.2.7)
'data is locked when a user is updating it', 'much worse than in production'	**Concurrency/locking hazard** (4.2.7)
'the order name must be unique'	**Parameterization for repeat runs** (4.2.7)
'non-linear function of the environment', 'the further the environment is from production standard'	**Environment fidelity** (4.2.8)
'stub', 'third party component responsible for credit card processing'	**Service virtualization** (4.2.8)
'Java heap size', 'memory paging'	**Configuration adjustment** (4.2.8)
'IP spoofing', 'browser emulation parameters'	**Load tool configuration** (4.2.8)
'machines involved in load generation should be monitored too'	**Load generator monitoring** (4.2.8)
'all simulated users initiated and performing work as designed'	**Steady state** (4.3.1)
'the stage of getting to the steady state'	**Ramp-up** (4.3.1)
'when the system's behavior is changing', 'concurrent logging of a large number of users'	**Transient state** (4.3.1)
'time-synchronize all tools used'	**Execution monitoring** (4.3.1)
'differ only by the time spent on the client side'	**Protocol versus GUI cross-check** (4.3.1)
'examined first', 'errors may influence the other performance data'	**Virtual user status** (4.4.1)
'skewed by outliers', '90th percentile', '100% compliance is not recommended'	**Transaction response time analysis** (4.4.1)
'200, 302, 304, 404'	**HTTP responses** (4.4.1)
'date/time of test start, script mix, peak CPU and RAM'	**Test log contents** (4.4.1)
'concise and understandable conclusions for management', 'actionable outcome'	**Executive summary** (4.4.1)

5.4 Typical traps

- **Two questions, one option set — three times.** PTFL-4.1.3 (#20/#21), PTFL-4.2.1 (#22/#23) and Chapter 1's type questions all reuse an option set. **Do not assume the second answer must be a different letter.**
- **The concurrency question tempts you to multiply. 2000 doctors × 6 updates = 12,000** is offered. **Concurrency counts simultaneous accesses, and each doctor makes one access per hour.** The answer is **2,000**.
- **The load-profile option figures are totals, not rates.** Sum the bands; they must equal the daily total.

- **"Correlation" is a false friend.** In this syllabus it means **carrying context identifiers between requests**, **not** the statistical relationship between metrics — even though **§4.4 does use the statistical sense** when discussing **identifying correlation between metrics**. Read which section the question is in.
- **Caching and locking pull in opposite directions.** Faster than expected → **caching**. Slower than expected → **locking / concurrency**.
- **Incompleteness is a valid reason to reject an option.** Two of the four options in the presentation questions are wrong for **both** audiences because they omit **mitigations** or **how the tests mitigate the risks**.
- **"Properly configured" in a stem eliminates configuration-related distractors.** In #34, *properly configured* rules out invalid data and production impact, leaving the generator's own performance.
- **The reporting question has a discursive rationale.** Do not be thrown by it: choose the option that **reports honestly against the stated objective and recommends more tuning**, and reject options that **invent an acceptance threshold** or **demand the impossible**.
- **Eight of the twenty questions share one scenario.** Read the health-records scenario once, extract **millions of records** and **three seconds, especially for critical allergies and preconditions**, and stop re-reading it.

5.5 How to eliminate wrong answers

- Strike any "technical objective" option phrased in terms of **what the user experiences**, and any "performance objective" option that is really about **robustness or failover**.
- Strike any planning-information option about **security, storage or downstream use** when the objective is **retrieval time**.
- Strike any presentation option that proposes **production testing without mitigation** or **risks without mitigations**.
- Strike any protocol option from the **wrong syllabus group**.
- Strike **wait time, latency time and reading time** whenever **think time** is available.
- Strike operational-profile answers that name something the scenario **already provides**, or that belong to the **load profile** stage.
- Strike load-profile options whose **band totals do not sum to the daily transaction total**.
- Strike concurrency answers that **multiply sessions by the work inside them**.
- Strike script options claiming protocol-level testing is best for **user experience** or **correlation**.
- Strike verification options relying on **absence of errors** or on **the application under test**.
- Strike environment options claiming a smaller environment is **faster, clearer or less risky**.
- Strike ramp-up options describing **exceeding the load**, **shutting down**, or **checking the tools**.
- Strike reporting options that **accept a percentage the requirements never stated** or **promise 100% compliance**.

5.6 Scenario-solving shortcuts

- **Objective questions:** ask **who would say this sentence?** A user or the business → **user-based**. An architect or operations engineer → **technical**.
- **Audience questions: money, risk-to-objective linkage, schedule and milestones → business. Steps, ramp rates, durations, environments and tooling → technical.**
- **Protocol questions:** find the protocol in the seven-group table and read off the label. Do not reason architecturally.
- **Operational profile questions:** check off **personas · tasks · numbers of users per unit of time**. Whichever is absent is the answer.
- **Load profile questions:** compute **people × accesses × transactions** for the day, then check each option's bands against it.
- **Concurrency questions:** find the **largest simultaneous headcount**, and ignore the work each one performs.
- **Script questions:** faster than expected → **caching**; fails after login → **correlation**; fails on the second run → **uniqueness / parameterization**.
- **Execution questions: ramp-up reaches steady state · spike exercises transient states · time-synchronize everything**.
- **Reporting questions: check virtual user status first · the 90th percentile is the usual goal · 100% is not recommended · report against the stated objective and recommend tuning**.

5.7 Common candidate mistakes

- Under-investing in Chapter 4 because it is long. It is **half the paper** and **you cannot pass without it**.
- Studying the thirteen objectives evenly. **Seven of them are worth two marks each and are mostly K2 recall** — those seven are **14 of the 20 marks** and are the cheapest.
- Learning the protocol list without the **group labels**. Both protocol marks are pure classification.
- Skipping the **arithmetic**. Three marks (**4.2.3, 4.2.4, 4.2.5**) come from a single worked scenario, and the arithmetic is simple multiplication once you see it.
- Confusing **correlation** with **parameterization**. Correlation carries **context identifiers extracted from responses**; parameterization supplies **varied data from a list** so each virtual user is distinct.
- Forgetting that **performance testing scripts are software** and therefore need **quality assurance across the whole range of input data**.
- Ignoring the **PTP's twelve sections** because the list is long. The official question is answered by knowing what belongs in **Profiles** and **Test Data**.
- Treating the **reporting** question as unanswerable because its official rationale discusses two options. The exam-safe reading is stated in §5.4.
- Not noticing that **PTFL-4.1.3 is worth two marks** — the largest allocation of any single objective on the paper alongside the other two-mark objectives.

5.8 Mnemonics that carry the whole chapter

Mnemonic	Unpacks to
"Users feel, technicians scale"	**User-based objectives = end-user satisfaction and business goals. Technical objectives = ability to scale and conditions of degradation.**
"User, business, system"	**Response time is a user concern · throughput is a business concern · resource utilization is a system concern.**
"Money to the board, steps to the engineers"	**Cost and representativeness → business stakeholders. Setup and execution steps, ramp rates, schedules → technical stakeholders.**
D-W-W-N-M-R-S ("**D**atabase, **W**eb, **W**eb service, **N**etwork, **M**obile, **R**emote, **S**OA")	The seven protocol groups: **ODBC/JDBC · HTTP/HTTPS/HTML · SOAP/REST · DNS/FTP/IMAP/LDAP/POP3/SMTP/Sockets/CORBA · TruClient/SMP/MMS · Citrix ICA/RTE · MQSeries/JSON/WSCL.**
"Response plus think equals elapsed"	**Transaction response time + think time = elapsed time** — and the same sum is the **denominator of the throughput formula**.
"Nest to get both"	Nesting transactions measures **the discrete steps and the whole process in one test**.
P-T-N ("**P**ersonas, **T**asks, **N**umbers")	The three operational-profile inputs. **Numbers** — *estimated numbers of users for each role/task per unit of time* — is the one the official question removes.
"Profile is what, load is how many and when"	**Operational profile** = the pattern one kind of user follows. **Load profile** = how many instances, over what time distribution.
R-R-S-P ("**R**amp-up, **R**amp-down, **S**teps, **P**redefined")	The four load-profile time distributions.
"Users over processing plus think"	**System throughput = [virtual users] / ([processing time] + [think time])** — so **when processing time rises, throughput falls**.
"Sessions, not transactions"	**Concurrency counts simultaneous sessions.** 2000 doctors with one access each = **2,000**, not 12,000.
"Closed is fixed, open keeps arriving"	**Closed system** = fixed population, no new arrivals while everyone waits. **Open system** = new users arrive regardless.
I-M-C ("**I**nit, **M**ain, **C**lean-up")	The three script sections. **Protocol scripts time server and network only; GUI scripts time**

Mnemonic	Unpacks to
	end-to-end.
"Correlate context, parameterize data"	**Correlation** carries identifiers extracted from responses. **Parameterization** varies the data so each virtual user is distinct — and some data (an **order name**) must be unique for the test to run twice.
"Cache flatters, locks punish"	**Caching → results much better than production. Locking → results much worse than production.**
"Non-linear"	**Performance is a non-linear function of the environment** — the further from production, the **less reliable the projections and the higher the risk**.
"Watch the watchers"	**Machines involved in load generation should be monitored too**, or the load may quietly fail to be maintained.
"Ramp to steady, spike for transient"	**Ramp-up exists to reach a steady state before measuring. Spike tests exercise transient states.**
V-R-T-F-H-N-H	The seven items analysed in **§4.4**, in order: **V**irtual user status (first) · **R**esponse time · **T**ransactions per second · **F**ailures · **H**its per second · **N**etwork throughput · **H**TTP responses.
"Ninety, not a hundred"	**The 90th percentile is often used as a goal**; **100% compliance is not recommended as the resources required may be too large and the net effect to the users will often be minor.**

5.9 Chapter 4 mind map

CHAPTER 4 — Performance Testing Tasks · 475 minutes · 20 exam marks · K2 = 10, K3 = 4, K4 = 6 · all thirteen objectives guaranteed

Branch	Node (K)	Everything hanging off it
4.1 Planning	**4.1.1** Deriving objectives (**K4**) — 1 mark	**Stakeholders set objectives, terminology and achievement criteria · user-based = end-user satisfaction and business goals** (users **just want to do what they need to do**) **· technical = operational aspects, ability to scale, conditions of degraded performance · key objectives: identify potential risks, find opportunities for improvement, identify necessary changes · three stakeholder questions: which transactions and what average response time · which system metrics and what values · what improvements versus previous cycles**
	4.1.2 The Performance Test Plan (**K4**) — 1 mark	**Created before testing, referred to by the Test Plan (which holds scheduling), updated as testing begins · twelve sections**: Objective (**quantifiable answer to adequacy and readiness under load**) · Test Objectives (**per stakeholder type**) · System Overview · Types of Performance Testing · **Acceptance Criteria** (**response time = user · throughput = business · resource utilization = system**; tied to **objectives, SLAs, baselines**; **baseline = metrics comparing current with previously achieved measurements**, possibly from **sanitized data**) · **Test Data** (**user accounts · user input · pre-populated database**; **extraction, import, creation, backups, masking — GDPR-mandatory but adds risk**) · System Configuration (**servers, tiers, hardware, software, external systems, versions, SUT build identifier**) · **Test Environment** (**mimics production at smaller scale**; **how results will be extrapolated**; **production is a last resort with risks discussed**; **tools may need special access rights**; **component-level testing is cheaper and earlier**) · Test Tools (**simulation, points of presence, monitoring**) · **Profiles** (**operational profiles aggregate into a load profile / scenario**) · **Relevant Metrics** (**too many measurements make analysis difficult and can degrade the application**) · **Risks** (**unmeasured areas, testing limitations, environment limitations**)
	4.1.3 Communicating	**Business focus**: less interested in **functional versus non-functional distinctions** ·

Branch	Node (K)	Everything hanging off it
	(**K4/K3**) — **2 marks**	**tooling, scripting and load generation are secondary · product risks linked to performance test objectives · cost versus representativeness · repeatability — difficult or minimum effort · project risks: constraints, dependencies, infrastructure (hardware, tools, data, bandwidth, environment, resources), key staff · high-level activities with costs, schedule and milestones · Technology focus: the load-profile generation approach and their expected involvement · detailed setup and execution steps related to architectural risks · repeatability steps including organizational aspects · scheduling of shared environments · mitigations if testing runs in production · clarity about their tasks and timing**
4.2 Analysis, design, implementation	**4.2.1** Protocols (K2) — **2 marks**	**OSI (ISO/IEC 7498-1); Layers 5–7 most common; lower layers for low-level embedded · Database: ODBC, JDBC · Web: HTTP, HTTPS, HTML · Web Service: SOAP, REST · Network: DNS, FTP, IMAP, LDAP, POP3, SMTP, Windows Sockets, CORBA · Mobile: TruClient, SMP, MMS · Remote Access: Citrix ICA, RTE · SOA: MQSeries, JSON, WSCL · 2-tier client-server = GUI client + backend database, needs ODBC · browser end-to-end uses HTTP/HTTPS, bypassing GUI interaction to focus on backend servers**
	4.2.2 Transactions (K2) — **2 marks**	**A transaction = activities from the point of initiation to when one or more processes have been completed · response time measured to identify components needing correction or optimization · think time reflects a real user's timing · response time + think time = elapsed time · response times under increasing load allow degradation to be correlated with specific transactions · nesting measures discrete steps and the aggregate in one test** (search, add to cart, pay, confirm)
	4.2.3 Operational profiles (**K4**) — 1 mark	**Distinct patterns of interaction, from users or other system components**; **multiple profiles combine into a load profile · identify: personas and roles · generic tasks at a high level of abstraction (business processes, major user stories) · estimated numbers of users per role/task per unit of time · gather: interviews and workshops (product owners, sales managers, end users) · functional specifications and requirements (user stories give the user type; UML use cases give the actor) · usage data from similar applications (web master's tool, monitoring logs) · monitoring user behavior (coordinate with usability tests) · construct: top-down, broken down only if needed · single out frequent, high-risk or data-heavy profiles · review and refine with principal stakeholders · non-user: off-line batch (throughput and completion within a period; stock-trading example) and systems of systems (supplier inputs, buffers and queues, system architects)**
	4.2.4 Load profiles (**K4**) — 1 mark	**The activity a system may experience in production — a designated number of instances performing predefined operational profiles over a specified time period**; instances that are users are **virtual users · four inputs: the objective · accurate operational profiles · known throughput and concurrency issues · quantity and time distribution · four distributions: ramp-ups (one user per minute) · ramp-downs · steps (100 users every five minutes) · predefined (daily or seasonal cycles) · worked example: a 100-user step background load plus a ramp to 220 users held two hours = a three-hour stress period · the official calculation: 2500 × 10 × 6 = 150,000/day, split into bands of 21,000 / 66,000 / 27,000 / 36,000**
	4.2.5 Throughput and concurrency (**K4**) — 1 mark	**System throughput = transactions of a given type per unit of time; it defines load · distinct from network throughput (data moved over the network) · user counts are a poor load measure without operational profiles — the 60x example · throughput = [virtual users] / ([processing time] + [think time])**, so **rising processing time cuts throughput · batch throughput = transactions within a given time frame (a nightly window) · concurrency = simultaneous / parallel threads of**

Branch	Node (K)	Everything hanging off it
		execution, modeled by **setting the number of virtual users; equal throughput can still mean different resource usage · closed system = fixed population; open system = continuous arrivals**
	4.2.6 Script structure (K2) — **2 marks**	**Simulates user or component activity contributing load; initiates requests in a proper order and at a given pace · three creation approaches: protocol-level recording (traditional; parameterization gives scalability and maintainability but is time consuming) · GUI-level recording · programming (the exact sequence must be determined) · structure: initialization · main sections executed multiple times · clean-up · timers must match a meaningful unit of logical work · protocol scripts measure server and network response time only; GUI scripts measure end-to-end time · default error handling is minimal (HTTP return code) — add checks; manual cleanup on error; verify indirectly, e.g., check the database · synchronization points wait for an event to force concurrency or coordinate scripts · scripts are software and need quality assurance across the whole range of input data**
	4.2.7 Implementing scripts (**K3**) — **2 marks**	**Implemented from the PTP and the load profiles, using an IDE or script editor, usually one operational profile per script (or several with conditional statements) · recording exactly simulates the real system; programming relies on knowing the proper request sequence · correlation replaces recorded context identifiers with variables extracted from responses — different from the statistical meaning; tools may automate simple cases; incorrect or missing correlation is the main reason recorded scripts fail to playback · re-using one user and one dataset misleads twice: caching makes results much better than production; locking makes them much worse · parameterize so each virtual user has a proper set of data, different enough to avoid caching and concurrency; some data must be unique (order name) · insert or adjust think times to hit the target throughput · combine scripts into a scenario implementing the whole load profile**
	4.2.8 Preparing for execution (K2) — **2 marks**	**Three activities: set up the SUT · deploy the environment · set up load generation and monitoring tools · be as close to production as possible; otherwise understand the differences and how results will be projected · performance is a non-linear function of the environment — less like production means less reliable projections and more risk · most important parts: data, hardware and software configuration, network configuration · data size and structure affect results dramatically; restore original data between cycles · service virtualization uses stubs (credit card processing) to isolate the SUT · deployment: labs for spotting small optimization effects · SaaS/cloud for end-to-end production load testing (if reachable from a cloud) · IaaS/cloud to minimize costs for time-limited testing · compensate configuration differences (Java heap size to avoid memory paging) · network emulation for global and mobile systems; deploy generators where users are · more than one generation machine may be needed and generators must be monitored · IP spoofing and browser emulation · validate the environment with a controlled set of tests, log analysis and database verification; restore state before repeating**
4.3 Execution	**4.3.1** Running the scripts (K2) — **2 marks**	**Generate load per the load profile · monitor all parts of the environment · collect and keep all results · advanced tools automate this and provide a console for monitoring and adjustment; some manual steps may remain · tests focus on the steady state (all users initiated and working as designed); ramp-up gets there, ramp-down finishes · transient states matter for concurrent login and spike tests, where monitoring averages may be very misleading · implement incremental load states during ramp-up; check load and responses are stable and random variation is not substantial · specify failure handling (logout to release resources) · start stand-alone monitoring separately; time-synchronize all tools · use the console and real-time log analysis to avoid needlessly continuing expensive tests**

Branch	Node (K)	Everything hanging off it
		that might impact other systems; stop or adjust on the fly · cross-check protocol tests with GUI scripts or manual profiles — differences should only be client-side time · in CI, checks must be automatic with alerting; easier for regression than exploratory or large-scale tests
4.4 Analysis and reporting	**4.4.1** Analyzing and reporting (**K4**) — 1 mark	**Metrics defined up front determine what is measured; analysis first compares to the objective; conclusions produce a summary report with recommended actions — physical components (hardware, routers), software (applications, database calls), network (load balancing, routing) · seven data items: virtual user status (first, because errors influence everything else) · transaction response time (min, max, average — skewed by outliers — and the 90th percentile as the usual goal; 100% compliance not recommended) · transactions per second (system throughput) · transaction failures (invalidate TPS because failures are faster) · hits per second · network throughput in bits per second · HTTP responses (200, 302, 304, 404) · graphical representations make trends easier to see · seven techniques: compare to requirements · observe trends · statistical quality control · identify errors · compare expected and actual · compare to prior results · verify component functioning · correlation between metrics locates the degradation point (TPS when CPU hit 90%); confirmation testing checks the fix · report sections: Executive Summary (actionable outcome for management) · Test Results (summary, baseline snapshot with date/time, concurrent user goal, throughput and key findings, architecture diagram, detailed tables and charts including where a stable application became unstable and the source of failures) · Test Logs (date/time, duration, scripts and script mix, data files, created files, HW/SW configuration and changes, average and peak CPU and RAM on web and database servers, notes, defects) · Recommendations (technical changes, areas for further analysis, additional monitoring of gateways, servers and networks)**

Section 6 — Flashcards

Cover the BACK row and recall it from the FRONT. All 94 cards are drawn from Chapter 4 of the CT-PT 2018 syllabus — the chapter that decides half the paper. Target: three clean passes before exam day.

FRONT	**1. What do stakeholders set with respect to performance testing?**
BACK	**The objectives, the terminology to be used, and the criteria for determining whether the objective has been achieved.**
FRONT	**2. Define user-based objectives.**
BACK	They **focus primarily on end-user satisfaction and business goals**. Users are **less concerned about feature types or how a product gets delivered — they just want to be able to do what they need to do**.
FRONT	**3. Define technical objectives.**
BACK	They **focus on operational aspects and provide answers to questions regarding a system's ability to scale, or under what conditions degraded performance may become apparent**.
FRONT	**4. Name the three key objectives of performance testing.**
BACK	**Identifying potential risks · finding opportunities for improvement · identifying necessary changes.**
FRONT	**5. List the three questions to answer when gathering information from stakeholders.**
BACK	**What transactions will be executed and what average response time is expected · what system metrics are to be captured and what values are expected · what performance improvements are expected compared to previous test cycles.**
FRONT	**6. When is the PTP created, and how does it relate to the Test Plan?**
BACK	**Created prior to any performance testing occurring**; **should be referred to by the Test Plan**, which **also includes relevant scheduling information**; and it **continues to be updated once performance testing begins**.
FRONT	**7. Name the twelve PTP sections.**
BACK	**Objective · Test Objectives · System Overview · Types of Performance Testing to be Conducted · Acceptance Criteria · Test Data · System Configuration · Test Environment · Test Tools · Profiles · Relevant Metrics · Risks.**
FRONT	**8. What does the PTP Objective section enable?**
BACK	**A quantifiable answer to the central question of the adequacy and the readiness of the system to perform under load.**
FRONT	**9. How does the syllabus allocate response time, throughput and resource utilization?**
BACK	**Response time is a user concern, throughput is a business concern, and resource utilization is a system concern.**
FRONT	**10. Define a baseline.**
BACK	**A set of metrics used to compare current and previously achieved performance measurements** — enabling **performance improvements to be demonstrated and/or acceptance criteria to be confirmed. It may be necessary to first create the baseline using sanitized data from a database.**
FRONT	**11. Name the three kinds of test data in the PTP.**
BACK	**User account data · user input data · database (the pre-populated database).**
FRONT	**12. What must the test data creation process address?**
BACK	**Data extraction from production data · importing data into the SUT · creation of new data · creation of backups that can be used to restore the data when new cycles of testing are performed · data masking or anonymizing.**
FRONT	**13. What is the syllabus's warning about data masking?**
BACK	It is **mandatory under GDPR** for production data with personally identifiable information, **however, in performance testing, data masking adds risk to the performance tests as it may not have the same data characteristics as seen in real-world use**.

FRONT	**14. What must the Test Environment section include when the environment is a smaller-scale mimic of production?**
BACK	**How the results from the performance testing will be extrapolated to apply to the larger production environment.**
FRONT	**15. When is production an acceptable test environment?**
BACK	**With some systems the production environment becomes the only viable option for testing, but in this case the specific risks of this type of testing must be discussed.**
FRONT	**16. What belongs in the PTP Risks section?**
BACK	**Areas not measured · limitations to the performance testing** (external interfaces that cannot be simulated, insufficient load, inability to monitor servers) · **limitations of the test environment** (insufficient data, scaled down environment).
FRONT	**17. Name three things that must be communicated to business-focused stakeholders.**
BACK	Any three of: **the connection between product risks and performance test objectives · the balance between cost and how representative the results will be · repeatability · project risks (constraints, dependencies, infrastructure, key staff) · high-level activities with costs, time schedule and milestones**.
FRONT	**18. What is of secondary interest to business-focused stakeholders?**
BACK	**Technical issues concerning tooling, scripting and load generation** — and they are **less interested in the distinctions between functional and non-functional quality characteristics**.
FRONT	**19. Name three things that must be communicated to technology-focused stakeholders.**
BACK	Any three of: **the planned approach to generating required load profiles and their expected involvement · detailed setup and execution steps showing the relation to architectural risks · steps required to make tests repeatable, including organizational aspects · scheduling of shared test environments · mitigations if testing must run in production · clarity about their tasks and when they are scheduled**.
FRONT	**20. Which OSI layers are most commonly used in performance testing?**
BACK	**Layer 5 (Session Layer) to Layer 7 (Application Layer)**, per **ISO/IEC 7498-1. The layer most in focus relates to the level of the architecture being tested** — lower layers for low-level embedded architectures.
FRONT	**21. Name the database and web protocols.**
BACK	**Database: ODBC · JDBC (and other vendor-specific). Web: HTTP · HTTPS · HTML.**
FRONT	**22. Name the web service, network and SOA protocols.**
BACK	**Web Service: SOAP · REST. Network: DNS · FTP · IMAP · LDAP · POP3 · SMTP · Windows Sockets · CORBA. SOA: MQSeries · JSON · WSCL.**
FRONT	**23. Name the mobile and remote access protocols.**
BACK	**Mobile: TruClient · SMP · MMS. Remote Access: Citrix ICA · RTE.**
FRONT	**24. Which protocol do traditional 2-tier client-server applications require, and why?**
BACK	**A protocol such as ODBC**, because the **"client" is the GUI and primary user interface** and the **"server" is the backend database**.
FRONT	**25. Which protocol family is used for end-to-end testing from the browser, and what does it bypass?**
BACK	**A web protocol such as HTTP/HTTPS — interaction with the GUI can be bypassed and the tests can focus on the communication and activities of the backend servers**.
FRONT	**26. Define a transaction.**
BACK	**The set of activities performed by a system from the point of initiation to when one or more processes (requests, operations, or operational processes) have been completed.**
FRONT	**27. What is think time, and what equation does it appear in?**

BACK	**Delays between user actions**, included to **better reflect the timing of a real user taking an action**. **Transaction response time + think time = elapsed time.**
FRONT	**28. What is the value of nesting transactions?**
BACK	**Individual and aggregate activities can be measured** — e.g., **search for item, add item to cart, pay for item, confirm order** as discrete steps **and** the order process as a whole, **gathered in one test**.
FRONT	**29. What can be learned by ramping up load while measuring transaction times?**
BACK	**It is possible to correlate the cause of degradation with the response times of one or more transactions** — and **analysis may show no degradation under load while other measurements show severe degradation**.
FRONT	**30. Define an operational profile.**
BACK	**Distinct patterns of interaction with an application, such as from users or other system components — a repeatable step-by-step flow through the application for a particular usage of the system. Multiple profiles may be combined to create a load profile.**
FRONT	**31. What are the three principal steps for identifying operational profiles?**
BACK	**1 Identify the data to be gathered · 2 Gather the data using one or more sources · 3 Evaluate the data to construct the operational profiles.**
FRONT	**32. What three data items must be identified for a user-based operational profile?**
BACK	**User personas and their roles · the generic tasks they perform (best modeled at a high level of abstraction, e.g., business processes or major user stories) · estimated numbers of users for each role/task per unit of time over a given time period.**
FRONT	**33. Name the four sources for gathering operational profile data.**
BACK	**Interviews or workshops with stakeholders · functional specifications and requirements · usage data and metrics from similar applications · monitoring the behavior of users performing predefined tasks.**
FRONT	**34. What do user stories and UML use cases contribute to profile identification?**
BACK	The user story format **As a <type of user>, I want <some capability> so that <some benefit> directly enables types of users to be identified**; **UML Use Case diagrams and descriptions identify the "actor"**.
FRONT	**35. State the three rules for constructing operational profiles.**
BACK	**A top-down approach** — broad profiles first, **broken down further only if needed to achieve performance test objectives · single out profiles involving frequent tasks, critical (high risk) or frequent inter-component transactions, or large data volumes · review and refine with the principal stakeholders before creating load profiles**.
FRONT	**36. What is the focus for off-line batch processing operational profiles?**
BACK	**Principally the throughput of the batch processing system and its ability to complete within a given time period**, with profiles focusing on **the types of processing demanded of the batch processes**.
FRONT	**37. What may be required for systems-of-systems operational profiles?**
BACK	**Modeling several different operational profiles** to represent inputs from **supplier systems**, possibly with **detailed analysis (e.g., of buffers and queues) together with the system architects and based on system and interface specifications**.
FRONT	**38. Define a load profile.**
BACK	**The activity which a component or system being tested may experience in production — a designated number of instances that will perform the actions of predefined operational profiles over a specified time period**. Instances that are users are **virtual users**.
FRONT	**39. Name the four inputs required to create a realistic and repeatable load profile.**
BACK	**The performance testing objective · operational profiles which accurately represent individual usage patterns · known throughput and concurrency issues · the quantity and time distribution with which the operational profiles are to be executed.**

FRONT	**40. Name the four time distributions, with an example of each.**
BACK	**Ramp-ups** (steadily increasing load, e.g., **add one virtual user per minute**) · **ramp-downs** (steadily decreasing load) · **steps** (instantaneous changes, e.g., **add 100 virtual users every five minutes**) · **predefined distributions** (e.g., **volume mimics daily or seasonal business cycles**).
FRONT	**41. Describe the syllabus's stress load profile example.**
BACK	A **step input of 100 virtual users** running **Operational Profile 1 for the whole test** (**a background load**), combined with a **ramp-up to 220 virtual users maintained for two hours before ramping down**, running **Operational Profile 2** — subjecting the SUT to **a three-hour period of stress**.
FRONT	**42. Compute the daily transaction total in the official doctors scenario.**
BACK	**2500 doctors × 10 accesses × 6 record updates = 150,000 transactions per day.**
FRONT	**43. Give the four hourly bands in the official doctors scenario.**
BACK	**7 hours of 500 doctors = 21,000 · 11 hours of 1000 doctors = 66,000 · 3 hours of 1500 doctors = 27,000 · 3 hours of 2000 doctors = 36,000.** They sum to **150,000**.
FRONT	**44. What is the highest concurrency in the official doctors scenario, and why is it not 12,000?**
BACK	**2,000** — the headcount from **14:00 to 16:00**. **Each doctor has only one access per hour**; the **six updates happen inside that access**. **Concurrency counts sessions, not the work inside them.**
FRONT	**45. Define system throughput and distinguish it from network throughput.**
BACK	**System throughput = the number of transactions of a given type that the system processes in a unit of time** (orders per hour, HTTP requests per second) and **defines load on the system**. **Network throughput = the amount of data moved over the network.**
FRONT	**46. State the throughput formula.**
BACK	**System throughput = [number of virtual users] / ([processing time] + [think time])** — so **when the processing time increases, throughput may significantly decrease even if everything else stays the same**.
FRONT	**47. Explain the 60x example.**
BACK	**500 users running short queries each minute = 30,000 queries per hour. The same 500 users running one query per hour = 500 queries per hour. Same users, 60x difference in load and at least 60x difference in hardware requirements** — which is why **user counts are not a good measure of load without operational profiles**.
FRONT	**48. Define concurrency, and say why it matters even at constant throughput.**
BACK	**A measure of the number of simultaneous / parallel threads of execution**, modeled by **setting the number of virtual users**. **It represents the number of parallel sessions, each of which may use its own resources** — so **even if throughput is the same, the amount of resources used may differ**.
FRONT	**49. Contrast closed and open systems.**
BACK	**Closed: the number of users is set (fixed population); if all users are waiting for the system's response, no new users can arrive** — the typical test setup. **Open: new users arrive all the time even if all current users are waiting** — many public systems.
FRONT	**50. How is throughput measured for batch systems?**
BACK	**According to the number of transactions that can be accomplished within a given time frame** — e.g., **a nightly batch processing window**.
FRONT	**51. What must a performance test script do?**
BACK	**Simulate a user or component activity that contributes to the load on the SUT**, and **initiate requests to the server in a proper order and at a given pace**.
FRONT	**52. Name the three ways to create a performance test script.**

BACK	**Recording at the protocol level** (traditional; **parameterization gives a scalable and maintainable script but may be time consuming**) · **recording at the GUI level** · **programming** (using **protocol requests, GUI actions or API calls**; **the exact sequence must be determined, which may be not trivial**).
FRONT	**53. Give the overall structure of a script.**
BACK	**An initialization section** (everything prepared for the main part) · **main sections that may be executed multiple times** · **a clean-up section** (steps to finish the test properly).
FRONT	**54. What should timed requests match, and why?**
BACK	**A meaningful unit of logical work** — e.g., **a business transaction for adding an item to an order or submitting an order**.
FRONT	**55. What do protocol-level and GUI scripts each measure?**
BACK	**Protocol-level: server and network response time only. GUI: end-to-end time** — although **what exactly is measured depends on the technology used**.
FRONT	**56. What does the syllabus say about default error handling?**
BACK	**Even in the best load testing tools, default error handling tends to be minimal (such as checking the HTTP request return code)**, so **add additional checks to verify what the requests actually return**; **cleanup in case of an error likely needs to be implemented manually**.
FRONT	**57. What is the recommended way to verify that a script did what it should?**
BACK	**Indirect methods — for example, checking the database to verify that the proper information was added.**
FRONT	**58. What are synchronization points for?**
BACK	**Specifying that the script should wait for an event at that point before proceeding** — used **to ensure that a specific action is invoked concurrently or to coordinate work between several scripts**.
FRONT	**59. What is the syllabus's conclusion about the nature of scripts?**
BACK	**Performance testing scripts are software, so creating one is a software development activity. It should include quality assurance and tests to verify that the script works as expected with the whole range of input data.**
FRONT	**60. What is one advantage of protocol-level performance testing?**
BACK	**It is scalable because the client is bypassed** — many tests can run at the same time **without requiring the client**.
FRONT	**61. What are the disadvantages of protocol-level testing?**
BACK	**Scripting can be quite difficult without a capture tool**; **it is not the best way to evaluate the total user experience because the UI is bypassed**; and **data correlation is more difficult because the correlation done at the UI is bypassed**.
FRONT	**62. Define correlation as used in this syllabus.**
BACK	**Replacing all recorded internal identifiers that define context with variables that can be changed between runs, with values extracted from the request responses** — e.g., **a user identifier acquired at login and supplied for all subsequent transactions. It has a different meaning than in statistics.**
FRONT	**63. What is the main reason recorded scripts fail to play back?**
BACK	**Incorrect correlation or lack of correlation.**
FRONT	**64. What do advanced load testing tools do about correlation?**
BACK	**They may do some correlation automatically, so it may be transparent in some cases — but in more complex cases, manual correlation or adding new correlation rules may be required.**
FRONT	**65. Name the two opposite hazards of re-using one user and one data set.**
BACK	**Caching** — data **copied from disk to memory** makes results **much better than in production**. **Concurrency / locking** — **if data is locked when a user is updating it**, results are **much worse than in production**, as the software **waits for the lock to free**.

FRONT	**66. What does parameterization mean, and what does "proper" data mean?**
BACK	**Replacing fixed or recorded data with values from a list of possible choices**, so **each virtual user uses a proper set of data**. **"Proper" means different enough to avoid problems with caching and concurrency** — specific to **the system, data and test requirements**.
FRONT	**67. Give the syllabus's example of data that must be parameterized for a test to run twice.**
BACK	**An order name that must be unique — unless it is parameterized, the test will fail as soon as it tries to create an order with an existing (recorded) name**.
FRONT	**68. Why are think times inserted or adjusted during implementation?**
BACK	**To match operational profiles** and **generate a proper number of requests / throughput**.
FRONT	**69. How do individual scripts become a whole test?**
BACK	**Scripts for separate operational profiles are combined into a scenario implementing the whole load profile. The load profile controls how many virtual users are started using each script, when, and with what parameters.**
FRONT	**70. Name the three main activities for preparing to execute performance tests.**
BACK	**Setting up the system under test · deploying the environment · setting up the load generation and monitoring tools and making sure that all the necessary information will be collected.**
FRONT	**71. State the non-linearity principle.**
BACK	**Performance is a non-linear function of the environment**, so **the further the environment is from production standard, the more difficult it becomes to make accurate projections for production performance**. **The lack of reliability of the projections and the increased risk level grow as the test system looks less like production.**
FRONT	**72. Name the three most important parts of the test environment.**
BACK	**Data · hardware and software configuration · network configuration.**
FRONT	**73. What does the syllabus say about test data size and structure?**
BACK	**The size and structure of the data could affect load test results dramatically**; **using a small sample set or one with different data complexity can give misleading results**; **it is difficult to predict how much data size affects performance before real testing**; and **the closer the test data is to production data in size and structure, the more reliable the results**.
FRONT	**74. What is service virtualization?**
BACK	Implementing **a workaround such as a stub to replace and emulate an unavailable component** — e.g., **a third party component responsible for credit card processing**. **Special tools are available and highly recommended to isolate the system under test.**
FRONT	**75. Name the three environment deployment options and when each is preferred.**
BACK	**Traditional internal (and external) test labs** — best **to see even small variations introduced by a performance optimization**. **Cloud as SaaS (vendor-provided load testing service)** — best **to load test the whole production environment end-to-end**, if the SUT is **reachable from a cloud**. **Cloud as IaaS** — best **to minimize costs when performance testing is limited in time**.
FRONT	**76. Give the syllabus's example of compensating for a configuration difference.**
BACK	**If test machines have less physical memory than the production machines, software memory parameters (such as Java heap size) may need to be adjusted to avoid memory paging.**
FRONT	**77. What are the two network approaches for global and mobile systems?**
BACK	For **global systems**, **deploy load generators in places where users are located**. For **mobile systems**, **network emulation remains the most viable option due to the variances in the network types that can be used**.
FRONT	**78. Why must load generation machines be monitored?**
BACK	**To avoid a situation where the load is not maintained properly because one of the load generators is running slowly.**

	More than one generation machine may be needed.
FRONT	**79. What is IP spoofing in this context?**
BACK	**Simulating that each virtual user has a different IP address** — a load tool configuration option, alongside **browser emulation parameters**.
FRONT	**80. How is the environment and setup validated before execution?**
BACK	**By conducting a controlled set of tests and verifying the outcome, and checking that the monitoring tools are tracking the important information** — using **log analysis and verifying database content**, and **returning the system to its original state before repeating a test that changed it**.
FRONT	**81. What does performance test execution involve?**
BACK	**Generating a load against the SUT according to a load profile · monitoring all parts of the environment · collecting and keeping all results and information related to the test.**
FRONT	**82. Define steady state, ramp-up and ramp-down.**
BACK	**Steady state: the system's behavior is stable — all simulated users/threads initiated and performing work as designed. Ramp-up: the stage of getting to the steady state. Ramp-down: the stage of finishing the test.**
FRONT	**83. When is it important to test transient states, and what is the warning?**
BACK	For **the concurrent logging of a large number of users** and **spike tests**. The warning: **careful monitoring and analysis are needed, as some standard approaches — such as monitoring averages — may be very misleading**.
FRONT	**84. What is advisable during the ramp-up?**
BACK	**Implement incremental load states to monitor the impact of the steadily increasing load**, ensuring **sufficient time is allocated** and **the system is able to handle the load**. Once steady, **monitor that load and responses are stable and that random variations are not substantial**.
FRONT	**85. Why must failure handling be specified?**
BACK	**To make sure that no system issues are introduced** — e.g., **it may be important for the user to logout when a failure occurs to ensure that all resources associated with that user are released**.
FRONT	**86. Why is time synchronization essential?**
BACK	**So that all information related to a specific test execution cycle can be located** — this applies to **stand-alone monitoring tools and log analysis** as well as the load tool.
FRONT	**87. Describe the cross-check technique for protocol-level tests.**
BACK	**Run several GUI-level (functional) scripts, or execute similar operational profiles manually, in parallel to the running load test. This checks that response times reported during the test only differ from the response times measured manually at the GUI level by the time spent on the client side.**
FRONT	**88. What extra requirement applies when performance testing runs automatically in CI?**
BACK	**Checks must be done automatically, since manual monitoring and intervention may not be possible** — the setup **should recognize any deviations or problems and issue an alert (usually while properly completing the test). Easier for regression performance tests than for exploratory or large-scale expensive tests.**
FRONT	**89. Which data item is analysed first, and why?**
BACK	**The status of simulated (virtual) users — any errors encountered may influence the other performance data**, and **it is normally expected that all simulated users have been able to accomplish the tasks specified in the operational profile**.
FRONT	**90. What does the syllabus say about averages and percentiles?**
BACK	**The average performance is not necessarily indicative of anything other than the mathematical average and can often be skewed by outliers. The 90th percentile is often used as a goal since it represents the majority of users attaining a specific performance threshold. It is not recommended to require 100% compliance with the performance objectives as the resources required may be too large and the net effect to the users will often be minor.**

FRONT	**91. Why can transaction failures invalidate transactions-per-second data?**
BACK	**Because a failed transaction will take far less time than a completed one. Any failures encountered are a cause for concern and the root cause must be investigated.**
FRONT	**92. List the seven analysis techniques.**
BACK	**Comparing results to stated requirements · observing trends in results · statistical quality control techniques · identifying errors · comparing expected and actual results · comparing the results to prior test results · verifying proper functioning of components.**
FRONT	**93. What are the four sections of a typical performance testing report?**
BACK	**Executive Summary · Test Results · Test Logs / Information Recorded · Recommendations.**
FRONT	**94. What does the test log typically include?**
BACK	**Date/time of test start · test duration · scripts used (including script mix) and relevant script configuration data · test data file(s) used · name and location of data/log files created · HW/SW configuration tested (especially changes between runs) · average and peak CPU and RAM utilization on web and database servers · notes on achieved performance · defects identified.**

Section 7 — Revision Notes

7.1 One-page chapter summary

Chapter 4 — Performance Testing Tasks

Planning. Stakeholders set the objectives, terminology and achievement criteria, and it is good practice to separate **user-based objectives** (**end-user satisfaction and business goals**) from **technical objectives** (**operational aspects, the ability to scale, and the conditions under which degraded performance appears**). The **Performance Test Plan** is written **before testing begins**, **referred to by the Test Plan**, and **updated as testing proceeds**; its **twelve sections** run from **Objective** through **Acceptance Criteria** — where **response time is a user concern, throughput a business concern and resource utilization a system concern**, and a **baseline** is **a set of metrics comparing current with previously achieved measurements** — via **Test Data** (**masking is GDPR-mandatory but adds risk**), **Test Environment** (**mimics production at smaller scale, with extrapolation stated**) and **Profiles**, to **Relevant Metrics** and **Risks**. Communication splits by audience: **cost versus representativeness, risk-to-objective linkage, project risks and milestones go to the business; load-profile approach, detailed setup and execution steps, repeatability steps, shared-environment scheduling and production-testing mitigations go to the technical audience. Analysis, design and implementation.** Protocols sit at **OSI layers 5 to 7** and fall into **seven groups** — **database, web, web service, network, mobile, remote access and SOA**. A **transaction** runs **from initiation to completion**, and **response time + think time = elapsed time**; **nesting** measures **discrete steps and the whole process in one test**. An **operational profile** describes **what one kind of user does**, built from **personas, tasks and numbers of users per unit of time**, gathered from **interviews, specifications, comparable usage data and user observation**, and constructed **top-down and reviewed with stakeholders**. A **load profile** describes **how many instances, over what time distribution — ramp-ups, ramp-downs, steps or predefined distributions** — and aggregating operational profiles produces the **scenario**. **System throughput = transactions per unit of time = [virtual users] / ([processing time] + [think time])**, and **user counts alone are a poor measure of load** — the syllabus's **60x example** proves it. **Concurrency counts simultaneous sessions**, and **closed systems have a fixed population while open systems see continuous arrivals**. Scripts have **initialization, main and clean-up** sections, must **time meaningful units of logical work**, and are **software requiring quality assurance**; **protocol scripts measure server and network time and scale because the client is bypassed**, while **GUI scripts measure end-to-end time**. In implementation, **correlation** carries **context identifiers extracted from responses — its absence is the main reason recorded scripts fail to play back** — and **parameterization** varies data so each virtual user is distinct, avoiding **caching** (**results much better than production**) and **locking** (**much worse**). Preparation covers **the SUT, the environment and the tools**, remembering that **performance is a non-linear function of the environment**, that **service virtualization isolates the SUT**, and that **load generators must themselves be monitored**. **Execution** generates load per the load profile while **monitoring everything and keeping all results**; **ramp-up exists to reach a steady state before measuring**, **spike tests exercise transient states**, and **every tool must be time-synchronized**. **Analysis and reporting** starts with **virtual user status**, then **response times** — where **averages are skewed by outliers**, **the 90th percentile is the usual goal** and **100% compliance is not recommended** — then **transactions per second, failures, hits, network throughput and HTTP responses**; results are consolidated into an **Executive Summary, Test Results, Test Logs and Recommendations**, with **correlation between metrics** locating the point where performance begins to degrade and **confirmation testing** proving the fix.

7.2 Key definitions to memorize

- **User-based objective — end-user satisfaction and business goals. Technical objective — operational aspects, ability to scale, conditions of degraded performance.**
- **PTP** — created **before** testing, **referred to by the Test Plan**, **updated during** testing; **twelve sections.**
- **Baseline — a set of metrics used to compare current and previously achieved performance measurements.**
- **Transaction — activities from the point of initiation to when one or more processes have been completed.**
- **Think time — delays between user actions; response time + think time = elapsed time.**
- **Operational profile — distinct patterns of interaction with an application**, a **repeatable step-by-step flow for a particular usage.**
- **Load profile (scenario) — a designated number of instances performing predefined operational profiles over a specified time period.**
- **System throughput — transactions of a given type per unit of time; = [virtual users] / ([processing time] + [think time]).**

- **Concurrency — the number of simultaneous / parallel threads of execution.**
- **Correlation — replacing recorded context identifiers with variables extracted from request responses.**
- **Parameterization — replacing fixed or recorded data with values from a list of possible choices.**
- **Service virtualization — implementing a stub to replace and emulate an unavailable component.**
- **Steady state / ramp-up / ramp-down — stable behaviour / getting there / finishing the test.**

7.3 High-priority concepts (study first)

Priority	Concept	LO	Marks
★★★	The **seven protocol groups**	4.2.1	**2**
★★★	**Think time**, the **elapsed-time equation**, and the **value of nesting**	4.2.2	**2**
★★★	**Ramp-up's purpose** and **spike testing for transient states**	4.3.1	**2**
★★★	**Business versus technical audience** — and why two options are wrong for both	4.1.3	**2**
★★★	**Correlation** and **caching** — the two K3 scenarios	4.2.7	**2**
★★★	**Protocol-level scalability**, **script structure**, **database verification**	4.2.6	**2**
★★★	**Non-linear environment** and **load generator monitoring**	4.2.8	**2**
★★	The **operational → load → concurrency** arithmetic	4.2.3–4.2.5	**3**
★★	**User-based versus technical objectives**; the **twelve PTP sections**	4.1.1, 4.1.2	**2**
★★	**Percentile reporting** and the **seven data items**	4.4.1	**1**

7.4 Last-minute revision sheet

- **20 of 40 questions · 20 of 40 points · K2 = 10, K3 = 4, K4 = 6 · all thirteen objectives guaranteed · 475 minutes of teaching.** Sample Exam questions **#18–#37**.
- **Seven objectives are worth two marks each: 4.1.3 · 4.2.1 · 4.2.2 · 4.2.6 · 4.2.7 · 4.2.8 · 4.3.1** — that is **14 of the 20 marks**, and most of it is K2 recall.
- **Objectives:** stakeholders set **objectives, terminology and criteria**; **user-based = satisfaction and business goals**; **technical = scale and degradation conditions**.
- **PTP:** twelve sections; **response time = user · throughput = business · resource utilization = system**; **baseline compares current with previously achieved measurements**; **masking is GDPR-mandatory but adds risk**.
- **Audience: money and milestones → business**; **steps, ramps and schedules → technical**; **production testing needs mitigations** and **risk lists need mitigations**.
- **Protocols: Database ODBC/JDBC · Web HTTP/HTTPS/HTML · Web Service SOAP/REST · Network DNS/FTP/IMAP/LDAP/POP3/SMTP/Sockets/CORBA · Mobile TruClient/SMP/MMS · Remote Access Citrix ICA/RTE · SOA MQSeries/JSON/WSCL. Layers 5–7.**
- **Transactions: response time + think time = elapsed time**; **nesting gives discrete and aggregate in one test**.
- **Operational profile** = **personas · tasks · numbers of users per unit of time**, built **top-down**, **reviewed with stakeholders**. **Load profile** = **instances × time distribution** (**ramp-up · ramp-down · steps · predefined**).
- **Throughput = [virtual users] / ([processing time] + [think time]). Concurrency = simultaneous sessions. Closed = fixed population; open = continuous arrivals. The 60x example** proves user counts alone are not load.
- **Scripts: initialization · main · clean-up**; **protocol = server and network time, scalable, client bypassed**; **GUI = end-to-end**; **verify via the database**; **scripts are software**.
- **Implementation: correlation carries context identifiers — its absence is the main playback failure**; **caching flatters, locking punishes**; **unique data (order name) must be parameterized**.

- **Preparation: performance is a non-linear function of the environment**; **data, hardware/software config and network config matter most**; **service virtualization isolates the SUT**; **labs for optimizations · SaaS for end-to-end · IaaS for cost**; **monitor the load generators**.
- **Execution: ramp-up reaches steady state before measuring**; **spike tests exercise transient states**; **averages mislead in transient states**; **time-synchronize all tools**; **cross-check protocol tests with GUI scripts**.
- **Reporting: virtual user status first**; **90th percentile is the usual goal**; **100% compliance is not recommended**; **failed transactions invalidate TPS**; **HTTP codes 200, 302, 304, 404**; four report sections: **Executive Summary · Test Results · Test Logs · Recommendations**.

7.5 Memorization checklist

1. I can name the thirteen learning objectives and say which seven are worth **two marks**.
2. I can separate a **user-based** from a **technical** objective in one reading.
3. I can list the **twelve PTP sections** and say what goes in **Test Data**, **Profiles** and **Risks**.
4. I can quote **response time = user, throughput = business, resource utilization = system**.
5. I can define a **baseline** and state the **GDPR masking** caveat.
6. I can assign any presentation content to **business** or **technical** stakeholders, and explain why two of the official options are wrong for both.
7. I can place any named protocol in its **syllabus group** and state the **OSI layers** used.
8. I can define a **transaction**, state the **elapsed-time equation**, and explain **nesting**.
9. I can list the three **operational profile** data items and the four **gathering sources**.
10. I can state the three **construction rules** and the two **non-user** profile types.
11. I can list the four **load profile inputs** and the four **time distributions**.
12. I can reproduce the **150,000-transaction calculation** and the **four hourly bands**.
13. I can state the **throughput formula** and explain the **60x example**.
14. I can define **concurrency** and explain why the answer is **2,000, not 12,000**.
15. I can contrast **closed** and **open** systems.
16. I can give the **three script sections** and say what **protocol** and **GUI** scripts each measure.
17. I can explain **synchronization points** and **indirect verification via the database**.
18. I can define **correlation**, state the **playback failure** sentence, and separate it from **parameterization**.
19. I can explain the **caching** and **locking** hazards and their opposite directions.
20. I can state the **non-linearity** principle and the **three most important parts** of the environment.
21. I can explain **service virtualization** and choose between **lab, SaaS and IaaS** deployment.
22. I can say why **load generators must be monitored** and what **IP spoofing** does.
23. I can define **steady state, ramp-up, ramp-down and transient state**, and give the **purpose of ramp-up**.
24. I can describe the **protocol-versus-GUI cross-check**.
25. I can list the **seven data items** analysed and say which is examined first and why.
26. I can state the **percentile rule** and why **100% compliance is not recommended**.
27. I can name the **four report sections** and list the **test log** contents.

7.6 Exam-day quick review

60-second mental checklist before answering any Chapter 4 question

1) Which of the thirteen objectives is this? Planning · communication · protocols · transactions · profiles · throughput and concurrency · script structure · script implementation · preparation · execution · reporting. **2)** If it is a **classification** question — objective type, protocol group, audience — answer straight from the table and do not reason around it. **3)** If it is **arithmetic**, compute **people × accesses × transactions** for the day first, then check the option's bands sum to it; and for **concurrency**, count **sessions**, never the work inside them. **4)** If a **script misbehaves**: fails after login → **correlation**; faster on repeat → **caching**; fails

on the second run → **uniqueness / parameterization**; slower than production → **locking**. **5)** If it is about the **environment**, remember **performance is a non-linear function of the environment** and that **the load generators need monitoring too**. **6)** If it is about **execution**, **ramp-up reaches steady state before measuring** and **spike exercises transient states**. **7)** If it is about **reporting**, check **virtual user status first**, favour the **90th percentile**, reject **100% compliance**, and never accept a threshold the requirements did not state. **8)** Reject any option that is **incomplete** — production testing without mitigations, risks without mitigations. **9)** If two options survive, choose the one that repeats **syllabus wording**.

Sources: ISTQB® Certified Tester Foundation Level Specialist Syllabus — Performance Testing, Version 2018 (9 December 2018), Chapter 4 and Keywords; ISTQB® CT-PT Sample Exam Questions and Sample Exam Answers, Version 2018 (9 December 2018); ISTQB® Exam Structure Tables v1.16 (CT-PT) and ISTQB® Exam Structures and Rules v1.2. All content is traceable to these references; no external knowledge was used.